

UNIVERSIDAD POLITÉCNICA DE MADRID

Escuela Técnica Superior de Ingeniería de Sistemas Informáticos (ETSISI)

Grado en Ingeniería de Computadores

Curso académico: 2025-2026

TRABAJO DE FIN DE GRADO

Atalaya: desarrollo de un asistente para re-valorizar veredictos de sandbox para la priorización de incidentes en un SOC

Autor: Iván de Diego Benítez

Tutor: Borja Bordel Sánchez

Madrid, septiembre de 2026

Resumen

Los centros de operaciones de seguridad (SOC) afrontan una sobrecarga estructural de alertas: los analistas dedican la mayor parte de su jornada a descartar ruido en lugar de investigar amenazas reales. El *sandbox* dinámico CAPEv2, referencia en el análisis de *malware*, agrava el problema de priorización porque emite una puntuación de maliciosidad (*malscore*, 0-10) que satura: sobre los 48.976 informes reales del dataset Avast-CTU, el 56,28 % alcanza el valor máximo (10,0), de modo que familias con perfiles operativos muy distintos quedan empatadas en «urgencia máxima» y la cola se vuelve inoperable para priorizar.

Este trabajo diseña, implementa y evalúa **Atalaya**, un asistente de triaje post-*sandbox* que no reemplaza a CAPE, sino que re-valora su veredicto con señales independientes del *malscore*. El sistema encadena, tras un parser multi-esquema de informes CAPE, cuatro etapas de decisión auditables: un motor de severidad intrínseca determinista (tipo de amenaza, firmas CAPE, técnicas MITRE ATT&CK y una lista de notoriedad basada en *advisories* CISA), un detector de anomalía conductual (Isolation-Forest sobre 14 *features*), un corrector de falsos positivos y una fusión multiplicativa de urgencia (0-10). Genera una ficha SecOps accionable mediante síntesis dual: plantilla determinista por defecto y modelo de lenguaje local (Ollama) opcional, con degradación automática. Un principio anti-*leakage* verificado por contrato garantiza que ningún componente emplee el *malscore* para decidir.

Sobre el dataset completo, el sistema reduce el bloque crítico del 60,9 % (que resultaría de bucketizar el *malscore*) al 0,74 % (364 casos): un factor 82 veces menor de carga crítica, con cada posición justificada componente a componente. Una validación cualitativa de 64 casos (57 coherentes, 7 con salvedades, 0 absurdos) y un análisis de sensibilidad (correlación de Spearman $\geq 0{,}9887$ bajo perturbaciones de ±25 % en todas las constantes) respaldan la robustez del orden propuesto. La aportación principal es una priorización explicable y trazable que respeta el trabajo del *sandbox* en lugar de duplicarlo.

Palabras clave: SecOps, priorización de incidentes, *sandbox* dinámico, CAPEv2, explicabilidad, aprendizaje automático, detección de anomalías, MITRE ATT&CK, modelo de lenguaje local, triaje de seguridad.

Abstract

Security Operations Centers (SOCs) face a structural alert overload: analysts spend most of their shift discarding noise instead of investigating real threats. CAPEv2, a reference dynamic sandbox for malware analysis, worsens the prioritization problem because it emits a maliciousness score (*malscore*, 0-10) that saturates: across the 48,976 real reports of the Avast-CTU dataset, 56.28 % reach the maximum value (10.0), so families with very different operational profiles end up tied at "maximum urgency" and the queue becomes useless for prioritization.

This work designs, implements and evaluates **Atalaya**, a post-sandbox triage assistant that does not replace CAPE but re-assesses its verdict with signals independent of the *malscore*. After a multi-schema parser for CAPE reports, the system chains four auditable decision stages (a deterministic intrinsic-severity engine covering threat type, CAPE signatures, MITRE ATT&CK techniques and a notoriety list based on CISA advisories; a behavioral anomaly detector, IsolationForest over 14 features; a false-positive corrector; and a multiplicative urgency fusion, 0-10) and produces an actionable SecOps summary card through dual synthesis: a deterministic template by default and an optional local language model (Ollama) with automatic fallback. A contract-verified anti-leakage principle guarantees that no component uses the *malscore* to decide.

Over the full dataset, the system shrinks the critical block from 60.9 % (which would result from bucketizing the *malscore*) down to 0.74 % (364 cases): an 82-fold reduction in critical load, with every position justified component by component. A qualitative validation of 64 cases (57 consistent, 7 with caveats, 0 absurd) and a sensitivity analysis (Spearman correlation ≥ 0.9887 under ±25 % perturbations of every constant) support the robustness of the proposed ordering. The main contribution is an explainable, traceable prioritization that respects the sandbox's work instead of duplicating it.

Keywords: SecOps, incident prioritization, dynamic sandbox, CAPEv2, explainability, machine learning, anomaly detection, MITRE ATT&CK, local language model, security triage.

Índice general

Resumen . . . 1
Abstract . . . 2

Capítulo 1. Introducción **10**
1.1 Contexto y motivación . . . 10
1.2 Problema que aborda este trabajo . . . 11
1.3 Objetivos del trabajo . . . 11
1.4 Estructura de la memoria . . . 12

Capítulo 2. Estado del arte **13**
2.1 Fatiga de alertas en Centros de Operaciones de Seguridad . . . 13
2.2 Sandboxing dinámico: de Cuckoo a CAPEv2 . . . 13
2.3 Evasión adversarial de sandbox y robustez del sistema . . . 14
2.4 Detección de anomalías en logs mediante Deep Learning . . . 15
DeepLog y la línea Transformer . . . 15
2.4.1 Enriquecimiento con inteligencia de amenazas: CISA y MITRE ATT&CK . . . 16
2.4.2 Síntesis narrativa y evaluación de resúmenes SOC . . . 17
2.5 Trabajos académicos relacionados . . . 17
TFGs nacionales . . . 17
Trabajos internacionales . . . 18
2.6 Síntesis: posicionamiento de este TFG . . . 19

Capítulo 3. Objetivos y alcance **21**
3.1 Objetivo general . . . 21
3.2 Objetivos específicos . . . 21
3.3 Preguntas de investigación . . . 23
3.4 Hipótesis . . . 24
3.5 Alcance, restricciones y criterios de éxito . . . 24
3.5.1 Alcance del sistema . . . 24
3.5.2 Exclusiones explícitas . . . 25
3.5.3 Criterios de éxito (hitos del MVP v2, H1–H4) . . . 25

Capítulo 4. Metodología **27**
4.1 Enfoque metodológico general . . . 27
4.2 Gestión del proyecto . . . 27
4.2.1 Estructura por puertas de calidad . . . 27
4.2.2 Control de versiones y trazabilidad . . . 28
4.2.3 Gestión de riesgos . . . 28
4.3 Estrategia de datos . . . 29
4.3.1 Fuentes y política de datos . . . 29
4.3.2 CapeFeatures: el contrato de extracción del parser . . . 29

4.3.3 Anonimización . . . 30
4.3.4 Validación de calidad . . . 30
4.3.5 División del dataset . . . 30
4.4 Protocolo experimental . . . 30
4.4.1 Configuración experimental . . . 30
4.4.2 Features por componente del pipeline . . . 31
4.4.3 Métricas de evaluación . . . 31
4.4.4 Estudios adicionales . . . 32
4.4.5 Amenazas a la validez . . . 32
4.5 Herramientas y entorno . . . 33
4.5.1 Stack tecnológico . . . 33
4.5.2 Entorno de ejecución . . . 33
4.5.3 Entorno de sandbox: CAPEv2 . . . 33
4.5.4 Reproducibilidad . . . 34

Capítulo 5. Diseño y arquitectura del sistema **35**
5.1 Visión general . . . 35
Fundamento de la arquitectura: qué hace útil a una alarma . . . 36
5.2 Contrato de datos: de EventSchema v1 a `CapeFeatures` . . . 37
5.3 Componente 1: parser CAPE multi-schema (`cape_parser_v2.py`) . . . 37
Variantes de `ttps` manejadas . . . 38
Extracción defensiva del resto de bloques . . . 38
Garantía anti-leakage . . . 38
5.4 Componente 2: motor de severidad intrínseca (`severity_engine.py`) . . . 39
Escala ordinal de tipo (`type_part`) . . . 39
Ponderación de TTPs por táctica (`ttp_part`) . . . 40
Lista CISA (`cisa_part`) . . . 40
Garantía anti-leakage . . . 40
5.5 Componente 3: anomalía conductual y corrección de falsos positivos . . . 41
5.5.1 Motor de anomalía (`anomaly_engine.py`) . . . 41
5.5.2 Corrector de falsos positivos (`fp_correction.py`) . . . 41
5.6 Componente 4: motor de fusión de urgencia (`urgency_fusion.py`) . . . 42
Buckets categóricos para la UI . . . 43
Garantía anti-leakage . . . 43
5.7 Componente 5: síntesis SecOps (`secops_summary.py` / `ollama_client.py`) . . . 43
Interfaz dual . . . 44
Contrato de salida . . . 44
Hallazgo de ingeniería: el system prompt defensivo . . . 44
Métrica de fidelidad: `payload_fidelity` . . . 44
Anti-leakage por contrato . . . 45
5.8 Componente 6: API REST y UI Atalaya . . . 45
Orquestador (`triage_service.py`) . . . 45
API v2 (`routes_triage.py`) . . . 46
UI Atalaya (`/ui/triage`, `/ui/triage/{rank}`) . . . 46
Estado de la integración . . . 46
5.9 Integración con CAPEv2 . . . 47
5.10 Decisiones de diseño relevantes . . . 47

Capítulo 6. Implementación **49**
6.1 Estructura del repositorio y stack . . . 49
6.2 El pipeline de 6 componentes . . . 50
6.2.1 `cape_parser_v2.py`: parser multi-schema (463 líneas) . . . 50

6.2.2 severity_engine.py: severidad intrínseca (378 líneas) . . . 51
6.2.3 anomaly_engine.py: detección de anomalías conductuales (251 líneas) . . . 52
6.2.4 fp_correction.py: corrección de falsos positivos (208 líneas) . . . 53
6.2.5 urgency_fusion.py: fusión multiplicativa (236 líneas) . . . 53
6.2.6 secops_summary.py y ollama_client.py: síntesis dual (414 + 192 líneas) . . . 54
6.3 Orquestación: triage_service.py . . . 55
6.4 API: rutas v2 . . . 56
6.5 UI Atalaya (/ui/triage y /ui/triage/{rank}) . . . 57
6.6 Pruebas . . . 58
6.6.1 Estrategia . . . 58
6.6.2 Conteo de tests por fase . . . 58
6.6.3 Demo end-to-end . . . 59
6.7 Garantías de ingeniería . . . 59
6.7.1 Anti-leakage de extremo a extremo (verificado en código) . . . 59
6.7.2 Degradación elegante del LLM . . . 60
6.7.3 Ingesta sin leakage: la etiqueta viene del CSV, no del informe . . . 60
6.7.4 Relación con Gate 3 y la máquina CAPE . . . 60

Capítulo 7. Evaluación experimental **61**
7.1 Configuración experimental . . . 61
7.1.1 Dataset . . . 61
7.1.2 Componentes evaluados . . . 62
7.1.3 Preguntas e hipótesis evaluadas . . . 63
7.2 Tabla estrella 1: saturación del malscore frente a severity_v2 . . . 63
7.3 Tabla estrella 2: separación inter-familia y descomposición del η² (H1) . . . 64
Por qué el η² agregado es circular . . . 65
Descomposición del η² inter-familia (n = 48.976) . . . 65
Lectura de los resultados . . . 65
La aportación del sistema: reordenar el bloque saturado . . . 66
Disclaimer metodológico: Spearman vs type_ordinal . . . 66
7.4 Tabla estrella 3: anomalía conductual por familia y caso Swisyn (H2) . . . 66
Caso de uso del componente de anomalía: una muestra Swisyn (prueba de existencia) 67
Corrector de falsos positivos . . . 68
7.5 Tabla estrella 4: cola priorizada, distribución por bucket y rank_shift . . . 68
Distribución por bucket . . . 68
Estadísticos globales . . . 68
Cola por familia . . . 69
Divergencias frente al orden por malscore (rank_shift) . . . 69
Correlación global y robustez . . . 70
Qué demuestran (y qué no) estos casos . . . 70
Validación cualitativa de la cola . . . 70
7.6 Tabla estrella 5: fidelidad de la síntesis SecOps (plantilla vs LLM) . . . 73
7.7 Amenazas a la validez . . . 75
7.7.1 Validez interna . . . 75
7.7.2 Validez externa . . . 75
7.7.3 Constructo . . . 75
7.7.4 Dos defectos de calibración conocidos en el motor de severidad . . . 76
7.8 Validación pendiente: informes CAPE 2.5 reales (Gate 3) . . . 76
7.9 Evaluación del prototipo v1 (histórico, contexto del pivote) . . . 77
7.9.1 Resultados obtenidos (v1) . . . 77
7.9.2 Por qué se descarta como resultado principal . . . 77
7.9.3 Qué se conserva de la evaluación v1 . . . 77

Capítulo 8. Conclusiones y trabajo futuro **79**
8.1 Resumen de aportaciones . . . 79
8.2 Cumplimiento de objetivos específicos . . . 80
8.3 Respuesta a las preguntas de investigación . . . 81
RQ1. ¿Produce la re-valoración contextual una priorización más discriminativa que el `malscore` bruto? . . . 81
RQ2. ¿Reduce el sistema el rango de los falsos positivos conocidos? . . . 81
RQ3. ¿Es el resumen generado completo, preciso y accionable? . . . 81
8.4 Limitaciones . . . 82
8.4.1 Dataset único: una sola fuente (Avast-CTU) . . . 82
8.4.2 Dependencia del campo `type` como entrada . . . 82
8.4.3 Motor de anomalía sin partición train/test . . . 82
8.4.4 Corrector de FP sin recall evaluado . . . 82
8.4.5 Evaluación de síntesis sobre n = 2 y con sesgo estructural . . . 82
8.4.6 Validación con CAPE 2.5 en vivo no realizada . . . 83
8.4.7 Lectura adversarial de las muestras rebajadas . . . 83
8.5 Trabajo futuro . . . 83
8.5.1 Validación con CAPE 2.5 en vivo (trabajo futuro declarado) . . . 83
8.5.2 Integración de un dataset de benignos en el corrector de FP . . . 83
8.5.3 Evaluación cualitativa de la síntesis con rúbrica MESSALA . . . 84
8.5.4 Validación con el dataset completo Avast-CTU: completada (2026-06-27) . . . 84
8.5.5 Robustez del campo `type` ante familias no etiquetadas . . . 84
8.5.6 Consolidación pre-entrega del repositorio . . . 84
8.5.7 Cableado de IOCs y alineación display-vs-motor en la síntesis . . . 84
8.6 Impactos y responsabilidad . . . 85
8.6.1 Impacto social . . . 85
8.6.2 Impacto ambiental . . . 85
8.6.3 Responsabilidad ética y profesional . . . 86
8.6.4 Relación con los Objetivos de Desarrollo Sostenible . . . 86
8.7 Cierre . . . 87

Bibliografía **88**
Referencias principales (ordenadas por categoría) . . . 88
Análisis dinámico de malware / Sandbox . . . 88
Fatiga de alertas y operación de SOC . . . 88
Detección de anomalías en logs . . . 88
Datasets de análisis dinámico . . . 89
Priorización de amenazas y síntesis SOC . . . 89
TFGs nacionales relacionados . . . 89
Trabajos internacionales . . . 90
Referencias técnicas y normativas . . . 90
Frameworks y estándares . . . 90
Herramientas y software empleado . . . 90
Datasets del proyecto (D1 evaluado en cap. 7; D2-D3 en estado del arte cap. 2) . . . 91

Anexos **92**
Anexo A. Manual de reproducción del pipeline (sin CAPE/VM) . . . 92
Anexo B. Estructura del repositorio del prototipo . . . 93
Anexo C. Catálogo de endpoints de la API v2 . . . 94
Anexo D. Capturas de la interfaz Atalaya . . . 94

Índice de tablas

2.1. Comparativa de métodos de detección de anomalías en logs (F1 en HDFS, BGL y Thunderbird). . . . 16
2.2. Posicionamiento de este TFG frente a los trabajos relacionados. . . . 19

3.1. Hitos de verificación del MVP v2 (H1–H4) y criterios de calidad. . . . 25

4.1. Fases del proyecto y puertas de salida. . . . 28
4.2. Riesgos del proyecto y mitigaciones principales. . . . 28
4.3. Features por componente del pipeline y su origen en el informe CAPE. . . . 31
4.4. Amenazas a la validez y mitigaciones. . . . 32
4.5. Stack tecnológico del prototipo. . . . 33

5.1. Propiedades de una alarma útil según Alahmadi et al. [20] y cómo las aborda el sistema. 36
5.2. Contrato de datos `CapeFeatures`: bloques y origen en el informe CAPE. . . . 37
5.3. Variantes del campo `ttps` manejadas por el parser multi-schema. . . . 38
5.4. Componentes del motor de severidad: rango y cálculo. . . . 39
5.5. Escala ordinal de tipo de amenaza (`type_part`). . . . 39
5.6. Pesos por táctica MITRE ATT&CK (`ttp_part`). . . . 40
5.7. Buckets categóricos de urgencia para la interfaz. . . . 43
5.8. Endpoints de la API v2. . . . 46

6.1. Rutas de la API v2 en la implementación. . . . 57
6.2. Conteo de tests por fase del pivote v2. . . . 58

7.1. Configuración experimental. . . . 61
7.2. Distribución del dataset de evaluación por familia. . . . 62
7.3. Componentes evaluados y su naturaleza. . . . 62
7.4. Preguntas de investigación e hipótesis, y sección que las responde. . . . 63
7.5. Saturación del `malscore` frente a `severity_v2`. . . . 64
7.6. Saturación por familia: `malscore` medio frente a `severity_v2` medio. . . . 64
7.7. Descomposición del η^2 inter-familia por señal. . . . 65
7.8. Correlación de Spearman frente a `type_ordinal`. . . . 66
7.9. Anomalía conductual por familia. . . . 67
7.10. Caso Swisyn: features de la muestra atípica. . . . 67
7.11. Distribución de la cola priorizada por bucket. . . . 68
7.12. Estadísticos globales: `urgency_score` frente a `malscore`. . . . 68
7.13. Cola priorizada por familia. . . . 69
7.14. Divergencias `rank_shift` medias por familia. . . . 70
7.15. Validación cualitativa: muestra ilustrativa de casos inspeccionados. . . . 71
7.16. Fidelidad de la síntesis SecOps: plantilla frente a LLM. . . . 73
7.17. Resultados del prototipo v1 (histórico). . . . 77

8.1. Cumplimiento de los objetivos específicos. 80

Índice de figuras

D.1. Cola priorizada (`/ui/triage`). Arriba, la distribución por bucket sobre los 48.976 casos (364 críticos, 22.562 altos, 22.201 medios y 3.849 bajos) frente a las 27.565 muestras que CAPE situaba empatadas en `malscore` = 10. Debajo, el panel «Divergencia real frente a CAPE», restringido a `malscore` < 10 para que la comparación de posiciones sea 1:1: promueve familias que el sandbox infra-puntúa (njRAT, +37.231 posiciones) y rebaja las que sobre-puntúa (HarHar *coinminer*, −16.896). 95
D.2. Ficha explicable de un caso (`/ui/triage/1`, njRAT). La urgencia 8,58 se descompone en la fórmula multiplicativa auditable (severidad 7,45 × *boost* de anomalía 1,1514 × penalización de FP 1,0 × contexto 1,0) y en el aporte de cada componente del motor de severidad (tipo 3,0; firmas CAPE 2,2; TTPs MITRE 1,25; notoriedad 1,0). Abajo, la ficha SecOps en modo plantilla con los IOCs de red anonimizados y la acción recomendada. 96
D.3. Dashboard del prototipo v1, accesible en `/ui/legacy` fuera de la navegación principal y con su aviso de *legacy*. Se conserva sin modificar como evidencia de que el pivote a v2 no rompió el contrato anterior. 97

Capítulo 1. Introducción

1.1 Contexto y motivación

La superficie de ataque de las organizaciones modernas crece a un ritmo que supera la capacidad de respuesta humana. Los centros de operaciones de seguridad (SOC, *Security Operations Center*) reciben decenas de miles de alertas diarias procedentes de fuentes heterogéneas (sistemas de detección de intrusiones, cortafuegos, plataformas SIEM, herramientas de análisis de endpoint) y deben decidir en segundos cuáles requieren atención inmediata y cuáles pueden ignorarse sin riesgo. Este problema, conocido en la industria como *alert fatigue*, constituye uno de los principales cuellos de botella en la operación de seguridad: analistas cualificados invierten la mayor parte de su jornada descartando ruido en lugar de investigar amenazas reales.

El análisis del sector apunta a un patrón generalizado: la función de detección representa uno de los principales cuellos de botella del ciclo de gestión de incidentes, no por falta de visibilidad, sino por exceso de señal sin priorizar. Las arquitecturas de tipo SOAR (*Security Orchestration, Automation and Response*), popularizadas por plataformas como Google SecOps, intentan paliar este problema mediante la automatización de flujos de trabajo, pero dependen de reglas de correlación predefinidas que no generalizan bien ante tácticas de ataque nuevas o variantes de amenazas conocidas.

La aplicación de técnicas de aprendizaje automático a las operaciones de seguridad, campo que suele denominarse *AI-driven SecOps* o *MLSecOps*, ofrece una vía complementaria a las reglas estáticas. En el ámbito de la detección de anomalías en logs, trabajos como LogBERT [10] y LogLLM [11] han demostrado que los modelos Transformer superan sistemáticamente a las líneas base tradicionales como DeepLog. En el análisis dinámico de malware, el uso de CAPEv2 como sandbox de referencia para extraer señales comportamentales está consolidado en la comunidad [08]. Sin embargo, ninguno de estos enfoques afronta el problema concreto que origina este TFG: los sandboxes de análisis como CAPEv2 emiten una puntuación de maliciosidad (*malscore*, 0-10) que, en la práctica, satura en el valor máximo para una fracción elevada de las muestras analizadas. Cuando la mayoría de las alertas activas reciben el mismo techo de urgencia, la cola del SOC queda plana e inoperable para priorizar, independientemente de la calidad de la detección.

Este Trabajo de Fin de Grado nace de esa observación concreta. El objetivo no es construir un detector mejor que CAPE, que sería redundante, sino diseñar un **asistente de triaje post-sandbox**: un sistema que asuma el veredicto del sandbox y añada lo que éste no aporta, una valoración de urgencia operativa basada en el tipo de amenaza, su notoriedad en listas de inteligencia internacional y las técnicas MITRE ATT&CK observadas en el análisis dinámico. Los trabajos existentes raramente integran estas tres dimensiones en un único sistema: **priorización trazable** (¿qué alerta es más urgente dentro del bloque saturado, y por qué?), **trazabilidad de la decisión** (¿qué componentes generaron esa urgencia, con qué valores?) y **síntesis accionable** (¿cómo resumirlo para que el analista pueda actuar sin leer el JSON técnico de CAPE?).

1.2 Problema que aborda este trabajo

El problema central que este TFG aborda puede formularse de la siguiente manera:

> *Dado el informe de análisis dinámico generado por el sandbox CAPEv2 para una muestra de malware, ¿es posible construir un sistema que re-valore ese veredicto con señales independientes de su puntuación bruta (tipo de amenaza, técnicas MITRE ATT&CK observadas, presencia en listas de inteligencia internacional) para producir una cola de urgencia priorizada, trazable componente a componente y presentada al analista en forma de ficha narrativa accionable?*

Este problema presenta tres dificultades técnicas concretas que este trabajo debe resolver:

Saturación del score de sandbox. El sandbox CAPEv2 emite una puntuación de maliciosidad (*malscore*, 0-10) que, en datasets de análisis reales, satura en el valor máximo para una fracción elevada de muestras: familias de malware con perfiles operativos muy distintos reciben el mismo techo de urgencia. Un sistema útil debe introducir señales adicionales, independientes del malscore, que discriminen la urgencia dentro del bloque saturado.

Circularidad y leakage. Un sistema que re-predice la puntuación del sandbox a partir de features extraídas del mismo informe CAPE no aporta valor informativo: aprende una función de su propia entrada. El diseño debe garantizar que la valoración de urgencia se funde en fuentes externas al sandbox (tipología de amenaza según clasificación SOC, listas de inteligencia internacional, técnicas MITRE ATT&CK) y verificar ese principio mediante tests de contrato en cada componente del pipeline.

Trazabilidad de la decisión. En un contexto de seguridad, donde la decisión de aislar un activo o escalar una alerta tiene consecuencias operativas directas, un número de urgencia sin justificación auditable tiene escasa aceptación práctica. El sistema debe exponer, para cada alerta, la descomposición completa del score: qué componente contribuyó cuánto, con qué valor de entrada, de forma que el analista pueda cuestionar o confirmar la priorización propuesta.

La magnitud del problema es medible: en el dataset de evaluación de este trabajo (48.976 informes reales de CAPEv2), el 56,28 % de las muestras alcanza la puntuación máxima del sandbox (malscore = 10,0). Si un SOC convirtiera esa puntuación directamente en niveles de urgencia, el 60,9 % de la cola (29.809 casos) quedaría etiquetado como crítico: una cola inoperable. El sistema propuesto reduce ese bloque a 364 casos críticos (0,74 %): un factor 82 veces menor de carga crítica, manteniendo trazable la justificación de cada posición (capítulo 7).

El sistema desarrollado en este trabajo, denominado **Atalaya** (la torre de vigilancia desde la que el SOC observa y prioriza), aborda las tres dificultades mediante un motor de decisión determinista y auditable que codifica política SOC explícita (tipología de amenaza, lista de notoriedad propia basada en advisories CISA, técnicas MITRE ATT&CK) en una función con componentes acotados y verificados por contrato en cada módulo.

1.3 Objetivos del trabajo

El objetivo general de este trabajo es diseñar, implementar y evaluar un asistente de triaje post-sandbox que re-valore el veredicto de CAPEv2 con señales independientes del malscore y produzca, para cada informe de análisis, una urgencia priorizada, trazable y accionable para el analista SOC.

Los objetivos específicos, las preguntas de investigación y las hipótesis de trabajo se desarrollan en detalle en el Capítulo 3. En síntesis, el sistema debe:

- Construir un parser multi-schema de informes CAPE reales sin leakage circular con el malscore.
- Implementar un motor de severidad intrínseca determinista y auditable (tipo de amenaza, firmas CAPE, técnicas MITRE ATT&CK, lista de notoriedad propia basada en advisories CISA).
- Detectar anomalía conductual intra-familia mediante IsolationForest sobre features conductuales independientes del malscore.
- Calcular una urgencia de fusión multiplicativa (0-10) con buckets categóricos para la cola SOC.
- Generar una ficha narrativa accionable mediante síntesis estructurada en dos modos: plantilla determinista (modo por defecto, sin dependencias externas) y LLM local (Ollama, modo opcional), con degradación automática al modo plantilla si Ollama no está disponible.
- Evaluar cuantitativamente la reordenación respecto al malscore bruto de CAPE sobre informes reales (Spearman, η^2 inter-familia, rank_shift bidireccional, fidelidad de síntesis).
- Ofrecer una API REST y una interfaz de triaje reproducible demostrable sin infraestructura externa.

1.4 Estructura de la memoria

La memoria se organiza en ocho capítulos:

- **Capítulo 1** introduce el contexto del problema, la motivación del trabajo y la estructura general de la memoria.
- **Capítulo 2** presenta el estado del arte en SecOps, análisis dinámico de malware con sandbox, saturación de scores como problema de priorización, enriquecimiento con inteligencia de amenazas (CISA, MITRE ATT&CK), síntesis narrativa con LLM local y evaluación de resúmenes en contexto SOC.
- **Capítulo 3** define los objetivos específicos, las preguntas de investigación, las hipótesis y los criterios de éxito (puertas de calidad).
- **Capítulo 4** describe la metodología: enfoque, gestión del proyecto, estrategia de datos (dataset Avast-CTU Public CAPE Dataset) y protocolo experimental de evaluación v2 (ranking, η^2, divergencia bidireccional y fidelidad de síntesis).
- **Capítulo 5** detalla el diseño y la arquitectura del sistema.
- **Capítulo 6** documenta la implementación: módulos, decisiones de código y suite de tests.
- **Capítulo 7** presenta los resultados experimentales y su análisis.
- **Capítulo 8** recoge las conclusiones, las limitaciones del trabajo y las líneas de trabajo futuro.

Capítulo 2. Estado del arte

2.1 Fatiga de alertas en Centros de Operaciones de Seguridad

Los Centros de Operaciones de Seguridad (SOC) modernos enfrentan un problema estructural conocido como *alert fatigue*: el volumen de alertas generadas por las herramientas de detección supera con creces la capacidad de análisis humano. Los sistemas SIEM (*Security Information and Event Management*) tradicionales, cuyo paradigma se basa en reglas correlacionales estáticas, tienden a producir tasas elevadas de falsos positivos, que los analistas señalan de forma recurrente como una de las principales causas de la fatiga de alertas en entornos reales. El estudio cualitativo de Alahmadi et al. [20], basado en entrevistas a analistas de SOC en activo, recoge esa percepción de forma elocuente: uno de los participantes cifra en un 99 % las alarmas que resultan no ser incidentes. Conviene interpretar ese dato con precisión, y los propios autores lo acotan: **es la percepción declarada por un analista, no una tasa medida**, y responde en su mayor parte a disparadores benignos más que a un fallo de rendimiento de la tecnología de detección. Su valor no es estadístico sino sintomático: describe cómo vive el analista una cola que no discrimina, que es exactamente el problema que aborda este trabajo. La consecuencia directa es la trivialización del trabajo del analista: al revisar alertas mayoritariamente benignas, la atención se degrada y los incidentes reales pasan inadvertidos.

Los stacks de monitorización basados en ELK (*Elasticsearch, Logstash, Kibana*) han avanzado en la capa de visualización y centralización de logs, proporcionando cuadros de mando útiles para el análisis forense posterior al incidente. Sin embargo, no resuelven el problema de priorización: acumulan señales pero no las ponderan [06]. La evolución natural hacia plataformas SOAR (*Security Orchestration, Automation and Response*) introduce flujos de trabajo automatizados, pero exige una inversión en configuración manual de playbooks que resulta prohibitiva para equipos pequeños.

Google SecOps (anteriormente Chronicle SOAR) representa el estado del arte en este espacio: una plataforma *cloud-native* que combina SIEM de alta escala, análisis de inteligencia de amenazas y capacidades SOAR integradas, con motor de correlación basado en UDM (*Unified Data Model*) y modelos de detección de anomalías a nivel de flota. La inspiración directa de este TFG es reproducir, a escala académica y sin dependencias de infraestructura propietaria, el flujo de priorización automatizada que plataformas como Chronicle implementan de forma comercial.

2.2 Sandboxing dinámico: de Cuckoo a CAPEv2

El análisis dinámico de malware mediante sandbox consiste en ejecutar muestras sospechosas en un entorno aislado e instrumentado, capturando su comportamiento en tiempo de ejecución: llamadas al sistema, tráfico de red, modificaciones de registro, capturas de pantalla y volcados de memoria. Este

enfoque complementa el análisis estático (firmas, heurísticas PE) al revelar comportamiento ofuscado que solo se manifiesta en ejecución.

Cuckoo Sandbox fue durante años la referencia de código abierto en este dominio. Gazdík (2023), en una tesina sobre análisis de malware en cracks de videojuegos, confirma que Cuckoo extrae más de 2.300 características del análisis dinámico frente a las 92 obtenidas del análisis estático de cabeceras PE, con precisiones del 94,64 % y 99,36 % respectivamente sobre sus corpus experimentales. Los experimentos del mismo trabajo, con 800 muestras benignas y 2.200 maliciosas analizadas en Cuckoo, validan que la información comportamental supera cuantitativamente a la información estructural para la clasificación de malware.

Los experimentos con Cuckoo de Gazdík son especialmente reveladores: cinco muestras de cracks de videojuegos sometidas a análisis dinámico recibieron puntuación 10/10 (*very suspicious*) en todos los casos. Entre los indicadores comportamentales detectados destacan: uso de UPX packer con secciones de alta entropía, llamadas a `NtProtectVirtualMemory` para cambiar permisos de memoria de RW a RX (técnica de evasión que evita el flag RWX simultáneo), consultas a `GetAdaptersAddresses` (posible reconocimiento de red), desactivación del proxy del sistema vía registro (`ProxyEnable=0`), y creación de un ejecutable con nombre en chino (`开始游戏.exe`) en el directorio temporal. El análisis profundo de la muestra más peligrosa registró 1.928 llamadas a la API y alertas de Suricata (*ETPRO USER_AGENTS Suspicious User-Agent*, *ETPRO HUNTING Request for config.json*) [08, Gazdík, 2023, VŠB-TUO].

CAPEv2 (*Config And Payload Extraction v2*) es el sucesor mantenido activamente de Cuckoo, con capacidades adicionales de extracción de configuraciones de malware y payloads directamente desde memoria durante la ejecución. A diferencia de Cuckoo (cuyo mantenimiento activo decayó hacia 2019-2020), CAPEv2 incorpora módulos de extracción específicos para familias de malware conocidas (RATs, ransomware, stealers) y continúa recibiendo actualizaciones de la comunidad de seguridad.

El uso de CAPEv2 como fuente de datos central de este TFG no es arbitrario: el autor desarrolló previamente un sistema equivalente en entorno empresarial, y la recreación académica del flujo sandbox→triaje replica una arquitectura probada en producción. Los informes JSON que genera CAPEv2 (puntuaciones, firmas de comportamiento, técnicas MITRE ATT&CK, IOCs, capturas de red) constituyen la materia prima del pipeline v2.

El análisis de este TFG sobre el dataset Avast-CTU revela un **fenómeno de saturación del malscore** sin precedente documentado en la literatura revisada: en datasets de análisis reales, el score de maliciosidad de CAPEv2 satura en su valor máximo para una fracción elevada de las muestras. El dataset público Avast-CTU Public CAPE Dataset [arXiv:2209.03188], que agrupa 48.976 informes reales de CAPEv2 procedentes de diez familias de malware, ilustra este problema con claridad. En el subconjunto evaluado en este TFG, siete de las diez familias presentan mediana de malscore = 10,0. La consecuencia operativa es directa: ordenar la cola del SOC por malscore produce una lista plana donde familias con perfiles de peligrosidad muy distintos (un RAT de espionaje persistente y un troyano básico) reciben la misma prioridad máxima. Esta observación empírica es el punto de partida del sistema propuesto: re-valorar ese veredicto con señales independientes del malscore que permitan discriminar urgencia dentro del bloque saturado.

2.3 Evasión adversarial de sandbox y robustez del sistema

El análisis en sandbox no es inmune a técnicas de evasión. Bouhnine (2024), en una tesis de máster en ciberseguridad, estudia sistemáticamente la efectividad de técnicas de evasión desarrolladas con el framework AVET contra CAPEv2. Sus conclusiones son directamente relevantes para el diseño del sistema:

1. **Inyección de shellcode** consigue evadir el sistema de monitorización de CAPEv2, ocultando las actividades del payload subsequente.
2. **Direct Syscalls** (implementados con SysWhispers3) reducen significativamente los IOCs generados por CAPEv2, al saltarse la instrumentación a nivel de API de Windows.
3. **Dynamic Loading de NTAPIs** resulta efectivo para reducir los indicadores de compromiso registrados por el sandbox.
4. Se observa una discrepancia inesperada: muestras compiladas con Visual Studio 2022 evadieron la detección de Windows Defender mientras que las compiladas con MinGW fueron detectadas, ilustrando la importancia del compilador en la superficie de detección.

[09, Bouhnine, 2024, Cylab]

Estas limitaciones tienen implicaciones directas para el diseño del sistema propuesto en este TFG. Un sistema robusto no puede depender exclusivamente del score de CAPEv2 como señal de verdad; debe combinar el veredicto del sandbox con señales independientes de él (tipología de la amenaza, técnicas MITRE ATT&CK observadas en el análisis dinámico, presencia en listas de inteligencia) que son más difíciles de manipular artificialmente porque no dependen de los mecanismos de instrumentación del sandbox. El motor de decisión determinista del sistema propuesto en este TFG responde precisamente a este requisito: cada componente de la valoración de urgencia opera sobre features que el adversario no puede suprimir simplemente evadiendo la instrumentación de CAPEv2, y la ausencia del malscore como input de ningún componente está verificada por tests de contrato.

2.4 Detección de anomalías en logs mediante Deep Learning

La detección automática de anomalías en flujos de logs de sistema constituye uno de los problemas centrales del MLSecOps moderno. Landauer et al. (2023), en una revisión sistemática publicada en *Machine Learning with Applications*, analizan 62 publicaciones (de 2.925 candidatas iniciales) sobre técnicas de *deep learning* para este problema y establecen una taxonomía de retos que resulta estructuralmente relevante para este TFG:

Tipos de anomalías identificados en la literatura: - *Point anomalies*: eventos individuales que se desvían del comportamiento normal. - *Contextual anomalies*: eventos normales en abstracto pero anómalos en su contexto de ocurrencia (momento, secuencia, host). - *Collective anomalies*: grupos de eventos que individualmente son normales pero cuya combinación es sospechosa.

Retos principales extraídos del survey: inestabilidad del formato de logs, desbalance de clases severo, diversidad de artefactos anómalos, disponibilidad limitada de etiquetas, necesidad de procesamiento en stream, volumen masivo de datos e interleaving de logs concurrentes. Cabe destacar que la **explicabilidad del modelo** aparece explícitamente como reto abierto en la revisión, lo que refuerza la elección de Random Forest (con importancia de variables nativa) sobre modelos de caja negra más potentes pero opacos [07, Landauer et al., 2023, AIT Vienna / *Machine Learning with Applications* 12, 100470].

DeepLog y la línea Transformer

El trabajo de Du et al. (2017), *DeepLog* [12], es la publicación más citada en el campo (Landauer et al. lo identifican como la más influyente de los 62 papers revisados) y estableció el paradigma de modelar logs como secuencias mediante LSTM, detectando anomalías como desviaciones del siguiente evento predicho.

LogBERT (Guo et al., 2021) lleva este enfoque al paradigma Transformer: aplica BERT con dos tareas de pre-entrenamiento autosupervisadas (predicción de log keys enmascarados y minimización

del volumen de hiperesfera) sobre secuencias de logs normales. Los resultados sobre tres datasets públicos (HDFS, BGL, Thunderbird) muestran mejoras consistentes frente a DeepLog y LogAnomaly:

Tabla 2.1: Comparativa de métodos de detección de anomalías en logs (F1 en HDFS, BGL y Thunderbird).

Método	HDFS F1	BGL F1	Thunderbird F1
DeepLog	77,34	86,12	93,08
LogAnomaly	56,19	74,08	92,73
LogBERT	**82,32**	**90,83**	**96,64**

[10, Guo et al., 2021, arXiv:2103.04475]

LogLLM (Guan et al., 2025) extiende este planteamiento al régimen de LLMs: combina BERT como encoder semántico de mensajes individuales con Llama (decoder-based) como clasificador de secuencias, enlazados mediante un proyector de alineación de espacios vectoriales. A diferencia de LogBERT, LogLLM no requiere un parser de logs previo (usa expresiones regulares), lo que simplifica el pipeline. Su entrenamiento en tres etapas (fine-tuning de Llama, entrenamiento del embedder BERT+proyector y fine-tuning del modelo completo) reporta mejoras sobre el estado del arte en cuatro datasets, incluyendo logs inestables [11, Guan et al., 2025, arXiv:2411.08561, SJTU/QUT].

Posicionamiento de este TFG respecto a esta línea: el sistema propuesto no detecta anomalías en logs de texto en bruto (ese es el dominio de LogBERT y LogLLM), sino que parte de los informes estructurados que emite CAPEv2 tras el análisis dinámico. El problema que aborda es distinto: no detectar una anomalía en la secuencia de eventos, sino re-valorar un veredicto ya emitido por el sandbox con señales de inteligencia externas (tipo de amenaza, TTPs MITRE, listas CISA). Para la síntesis narrativa de la ficha SOC, el sistema adopta un enfoque complementario al de LogLLM: un LLM local de tamaño reducido (Phi-4-mini 3.8B, Llama 3.2 3B) ejecutado sobre CPU mediante Ollama, con degradación garantizada a plantilla determinista cuando el modelo no está disponible, lo que hace el pipeline operable sin dependencia de infraestructura de inferencia permanente.

2.4.1 Enriquecimiento con inteligencia de amenazas: CISA y MITRE ATT&CK

La re-valoración del veredicto del sandbox requiere fuentes de inteligencia externas que permitan contextualizar una muestra más allá de su puntuación de maliciosidad. Dos marcos de referencia consolidados en la industria SOC aportan esta señal:

La **CISA** (*U.S. Cybersecurity and Infrastructure Security Agency*) publica periódicamente advisories que identifican las familias de malware más prevalentes y de mayor impacto operativo a nivel global. El advisory AA22-216A (*2021 Top Malware Strains*, publicado en agosto de 2022) es el punto de partida de referencia para este TFG. La presencia de una familia en estas listas constituye una señal de inteligencia **independiente** del análisis dinámico: indica que la familia es objeto de seguimiento activo por agencias de seguridad nacional y que su impacto en infraestructuras críticas está documentado.

El motor de severidad del sistema propuesto incorpora una **lista de notoriedad propia** construida sobre el advisory AA22-216A y otras alertas CISA del período, que añade familias con presencia documentada en alertas CISA pero no nombradas en ese advisory específico (Emotet, njRAT, gh0st, asyncrat, raccoon) y omite familias del advisory sin representación en el dataset de evaluación (mouseisland, nanocore, gootloader). La pertenencia a esta lista añade un componente fijo (+1,0 sobre la escala 0-10) a la valoración de urgencia, independientemente de cómo se comporte la muestra concreta en el sandbox. De las diez familias del dataset Avast-CTU, seis activan este bonus: Emotet, Qakbot, Trickbot, Lokibot, Ursnif y njRAT.

MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) es el marco de referencia de facto para la descripción de técnicas de ataque observadas. CAPEv2 mapea automáticamente el comportamiento detectado a técnicas ATT&CK (campo `signatures[].ttp` en el JSON de salida), lo que permite valorar la urgencia en función del impacto táctico de lo observado: las técnicas de impacto (*Impact*), exfiltración (*Exfiltration*) y movimiento lateral (*Lateral Movement*) pesan significativamente más que las de descubrimiento (*Discovery*) o ejecución simple (*Execution*), siguiendo la lógica de priorización por daño potencial habitual en SecOps.

La priorización de alertas por TTPs, en lugar de por un score monolítico de maliciosidad, tiene respaldo directo en la literatura reciente. Rashid et al. (2025) identifican explícitamente este vacío y proponen AIM (*Adversarial Impact Metric*), un sistema de puntuación proactiva de técnicas ATT&CK para la priorización ante el analista SOC, ante la ausencia de un marco base para ponderarlas [14, Rashid et al., 2025, ICSCA '25, ACM. DOI:10.1145/3731806.3731846]. Como precedente seminal de la idea de «severidad por táctica y secuencia frente a veredicto único», el sistema RapSheet (Hassan et al., 2020) demuestra en entornos EDR que el orden causal de las alertas aporta información de riesgo que el score aislado de cada alerta no captura [15, Hassan et al., 2020, IEEE S&P 2020, Xplore:9152771].

2.4.2 Síntesis narrativa y evaluación de resúmenes SOC

La generación de fichas narrativas accionables (resúmenes en lenguaje natural que comunican al analista qué hizo una muestra, por qué importa, qué IOCs contiene y qué acción se recomienda) es un área de investigación activa en el cruce de LLMs y SecOps. El trabajo de LogLLM mencionado en §2.4 apunta en esa dirección para el dominio de logs; el sistema propuesto en este TFG la aplica al dominio de informes de sandbox.

La evaluación de la calidad de resúmenes generados automáticamente presenta retos metodológicos específicos: las métricas de referencia en NLP como ROUGE-L y BERTScore miden solapamiento léxico o semántico con una referencia humana, pero no verifican específicamente que el resumen contenga los datos operativos correctos del informe (valores numéricos de urgencia, familia exacta, TTPs detectados, IOCs). La rúbrica MESSALA, propuesta como marco de evaluación de resúmenes de incidentes de seguridad con dimensiones de completitud, precisión, accionabilidad, concisión y ausencia de alucinaciones, ofrece un instrumento más apropiado para este dominio [16, Okada et al., 2026, arXiv:2601.03013] *(preprint arXiv; sin peer-review confirmado a la fecha de redacción de este TFG)*.

2.5 Trabajos académicos relacionados

TFGs nacionales

La colección de trabajos de fin de grado relacionados cubre tres pilares complementarios al enfoque de este TFG.

Pilar IDS con ML (Universidad de Zaragoza, 2019-2024)

Cinco TFGs de la Universidad de Zaragoza abordan la detección de intrusiones con machine learning y constituyen el contexto académico nacional más directo.

Entrena Rodríguez (2023) estudia la selección de atributos para IDS en entornos IoT, comparando técnicas FFR, FSS y WFS sobre los datasets LATAM-DoS y BOT-IoT. Su resultado más relevante: con solo dos atributos (reducción del 90 %) se alcanza F1 próximo a 1 en LATAM-DoS/DDoS, y F1 ≈ 0,9 en BOT-IoT. El trabajo establece que, en ciberseguridad, optimizar el tiempo de reacción es tan crítico como la precisión [01].

Huarte Albás (2019) aplica Distribución Gaussiana e Isolation Forest sobre capturas de tráfico propio y los datasets KDD'99 y NSL-KDD, distinguiendo entre tráfico benigno y ataques de fuerza bruta, HTTP-flood y TCP-SYN. Es el único trabajo de la colección que emplea Isolation Forest como algoritmo principal, lo que lo convierte en referencia directa para justificar su uso en el componente de anomalía de este TFG [02].

Varea Palacios (2024) aborda la clasificación de familias de malware a partir de trazas de ejecución (llamadas al sistema), extrayendo n-gramas y comparando KNN, SVM, Gradient Boosting y Regresión Logística. Concluye que los clasificadores múltiples superan a los simples, y que filtrar por «categorías de interés» (llamadas al sistema asociadas a comportamiento malicioso) mejora sistemáticamente los resultados. Comparte con este TFG el foco en señales de ejecución como fuente primaria de detección [03].

Calvera Tonin (2022) desarrolla la herramienta ERTMD y dos prototipos de IDS (uno interactivo y uno como daemon Linux) evaluados sobre KDD'99 y SSH/FTP_ISCX con varias técnicas de minería de datos. El trabajo es especialmente útil como referencia de diseño experimental reproducible y como ejemplo de IDS operativo sobre Linux [04].

Rodríguez García (2022) es el trabajo más cercano en metodología experimental: evalúa diez técnicas de ML sobre CICIDS2017 usando Weka (Naive Bayes, Logistic, MLP, SMO, k-NN, AdaBoost, OneR, J48, PART y Random Forest), con clasificación binaria y multiclase. Random Forest, J48 y PART alcanzan F1 > 0,999 (binario). El uso de Zeek para extraer atributos de flujos de red replica el tipo de ingeniería de variables que este TFG aplica sobre eventos SOC. La separación explícita binario/multiclase es una referencia directa para el diseño experimental del capítulo 7 [05].

Pilar SIEM/ELK (Universidad Carlos III de Madrid, 2018)

Mateos López (2018) implementa un sistema de monitorización de aulas informáticas basado en el stack ELK (Elasticsearch, Logstash, Kibana) desplegado con Docker. El sistema recoge y visualiza logs de Linux, detecta intentos de acceso por fuerza bruta y cifra la intercomunicación entre componentes. Representa el estado del arte operativo inmediatamente anterior al enfoque de ML: acumula y visualiza señales pero no las puntúa ni las prioriza. La diferencia con este TFG es precisamente esa capa de inteligencia añadida sobre la canalización de logs [06].

Caso más próximo: análisis de malware con sandbox (VŠB-TUO, 2023)

El trabajo de Gazdík (2023) [08] es el más cercano al núcleo técnico de este TFG por su uso de Cuckoo Sandbox para análisis dinámico. Gazdík extrae más de 2.300 features dinámicas y 92 estáticas, alcanzando 94,64 % de precisión dinámica y 99,36 % estática. La diferencia fundamental de alcance: Gazdík se centra en la detección binaria malicioso/benigno, mientras que este TFG conecta el análisis de sandbox con una capa de priorización operacional (cola SOC) con explicabilidad y contexto del host.

Trabajos internacionales

Diploma thesis AI-IDS con TOGAF (TU Wien, 2025): Alsudani (2025) propone un framework de integración de IDS basado en LSTM con hiperparámetros optimizados mediante el algoritmo *Cheetah Optimization* (COA), evaluado sobre NSL-KDD, CIC IoT-DIAD 2024 y UM-NIDS. La contribución más relevante para este TFG no es el modelo en sí sino el marco arquitectónico: el trabajo mapea los servicios de detección sobre capas de negocio, aplicación, tecnología y motivación (TOGAF/ArchiMate), garantizando trazabilidad vertical desde los requisitos hasta la implementación. El propio trabajo identifica la *explicabilidad y la automatización SOC* como líneas de trabajo futuro abierto, lo que valida que el enfoque de este TFG opera en la frontera del estado del arte académico [13, Alsudani, 2025, TU Wien].

Masterarbeit NIDS para OPC-UA (TU Darmstadt, 2017): Bortoli (2017) desarrolla un plugin para Bro NIDS que soporta el protocolo OPC-UA, utilizado en sistemas de control industrial (ICS/IoT). El trabajo es relevante para este TFG como referencia de despliegue en entornos industriales y por su análisis de las limitaciones de los NIDS basados en firmas frente a exploits de día cero. La observación de que «la mayoría del soporte para ICS en NIDS está ausente» sigue siendo pertinente en 2025 y justifica el valor del enfoque agnóstico al protocolo de este TFG, que opera sobre eventos normalizados independientemente del vector de ataque [17, Bortoli, 2017, TU Darmstadt]. *Nota: el fichero lleva el sufijo 2024 pero el documento es de 2017 (fecha en portada: 19 September 2017).*

2.6 Síntesis: posicionamiento de este TFG

La tabla siguiente resume las dimensiones de comparación entre los trabajos más relevantes y el sistema propuesto:

Tabla 2.2: Posicionamiento de este TFG frente a los trabajos relacionados.

Trabajo	Dataset / sandbox	Enfoque principal	Trazabilidad SOC	Cola priorizada	CAPEv2	Síntesis LLM
Gazdík 2023 [08]	Cuckoo (propio)	RF + features dinámicas	No	No	No	No
Bouhnine 2024 [09]	CAPEv2	Evaluación de evasión	No	No	Sí	No
LogBERT 2021 [10]	Logs (HDFS/BGL)	BERT autosupervisado	Parcial	No	No	No
LogLLM 2025 [11]	Logs (HDFS/BGL)	BERT + Llama	No	No	No	Sí
Landauer et al. 2023 [07]	Survey (62 métodos)	Deep learning para logs	—	No	No	No
Entrena 2023 [01]	LATAM-DoS/BOT-IoT	Weka ML (selección features)	No	No	No	No
Huarte 2019 [02]	KDD99/NSL-KDD	Gauss. + Isolation Forest	No	No	No	No
Varea 2024 [03]	Llamadas a sistema	KNN/SVM/GB/LR (n-gramas)	No	No	No	No
Calvera 2022 [04]	KDD99/SSH/FTP	Data mining	No	No	No	No
Rodríguez 2022 [05]	CICIDS2017	RF/J48/PART	Parcial	No	No	No

Trabajo	Dataset / sandbox	Enfoque principal	Trazabilidad SOC	Cola priorizada	CAPEv2	Síntesis LLM
Mateos 2018 [06]	Logs Linux (propio)	ELK (sin ML)	No	No	No	No
Alsudani 2025 [13]	NSL-KDD/IoT	LSTM+COA	No (futuro)	No	No	No
Bortoli 2017 [17]	OPC-UA (propio)	Bro NIDS	No	No	No	No
Este TFG	**Avast-CTU (CAPE reales)**	**Motor determinista (tipo+CISA+TTPs) + IsolationForest aux.**	**Sí (por componente)**	**Sí**	**Sí**	**Sí**

El valor diferencial de este trabajo respecto al estado del arte académico reside en la **combinación de tres contribuciones** que la literatura aborda de forma separada: (1) partir de informes reales de CAPEv2, no de datos sintéticos ni de logs de red, para re-valorar el veredicto del sandbox en lugar de repetirlo, (2) un motor de decisión determinista y auditable que codifica política SOC explícita (tipología de amenaza, lista de notoriedad propia basada en advisories CISA, técnicas MITRE ATT&CK) con trazabilidad componente a componente verificada por contrato, y (3) síntesis narrativa accionable mediante LLM local que convierte el JSON técnico de CAPE en una ficha legible por el analista sin acceso al informe original.

El sistema no pretende superar las métricas de detección de malware de los trabajos especializados de la literatura, que resuelven un problema distinto, sino demostrar que la re-valoración trazable del veredicto de sandbox es viable, evaluable sobre datos públicos y demostrable sin infraestructura externa.

Capítulo 3. Objetivos y alcance

3.1 Objetivo general

El objetivo general de este Trabajo de Fin de Grado es diseñar, implementar y evaluar un **asistente de triaje post-sandbox** que, a partir de los informes generados por el sandbox de análisis dinámico CAPEv2, produzca una valoración de urgencia operativa para el Centro de Operaciones de Seguridad (SOC) y una ficha narrativa accionable que el analista pueda interpretar sin acceder al JSON técnico de CAPE. «Operativa» significa aquí accionable y trazable, no demostrablemente óptima: no existe un *ground truth* de urgencia contra el que validar el orden propuesto, como se discute en §7.5.

El punto de partida del sistema es el veredicto ya emitido por CAPE, su puntuación de maliciosidad (`malscore`), y el contenido del informe de análisis dinámico: firmas detectadas, técnicas MITRE ATT&CK observadas, comportamiento del proceso, actividad de red y metadatos de la muestra. El sistema no reclasifica si la muestra es malware (CAPE ya lo hace); en cambio, responde a una pregunta distinta: **¿con qué urgencia debe atenderse este hallazgo en el SOC, teniendo en cuenta el tipo de amenaza, su comportamiento observado y el contexto del activo afectado?** Y, complementariamente: **¿cómo resumir ese hallazgo en un texto legible, completo y accionable para el analista?**

El prototipo resultante debe ser técnicamente defendible en un contexto académico, reproducible experimentalmente sobre datos públicos de CAPE (dataset Avast-CTU Public CAPE Dataset), y demostrable en una presentación sin dependencia de infraestructura externa.

3.2 Objetivos específicos

Los objetivos específicos desglosan el objetivo general en contribuciones verificables, cada una asociada a una puerta de calidad concreta (§ 3.5.3).

OE1. Construir un dataset etiquetado de reportes CAPE reales sin leakage circular. Adaptar el parser de informes CAPE (módulo `cape_parser_v2.py`) al esquema real de Avast-CTU (CAPE 2.2) y al esquema de la propia VM de análisis (CAPE 2.5), de manera que las etiquetas de familia y tipo de amenaza provengan de la fuente externa (`public_labels.csv` de Avast-CTU), nunca del propio `malscore` del informe. El dataset resultante debe incluir manifiesto con metadatos de versión, y el split temporal (70 % entrenamiento / 15 % validación / 15 % test) debe preservar el orden cronológico sin contaminación hacia atrás. *Criterio de verificación: Hito H1. Dataset versionado ingestado sobre ≥ 200 informes reales; manifiesto publicado en* `data/processed/`*; test* `test_avast_no_label_derivation` *verde.*

OE2. Implementar un motor de severidad intrínseca determinista y auditable. Construir la función `compute_severity(family, type_, sig_max_severity, sig_sum_weight, sig_high_severity_count, ttp_codes)`

que calcule una puntuación 0-10 a partir de cuatro componentes independientes del `malscore`: la escala ordinal del tipo de amenaza (ransomware/rat > banker > trojan > pws/keylogger > coinminer > adware), el peso agregado de las firmas CAPE, el impacto de las técnicas MITRE ATT&CK observadas (ponderadas por táctica: impact, exfiltration, lateral movement… > discovery, reconnaissance) y la presencia de la familia en la lista de amenazas de alto impacto de CISA (Joint Advisory AA22-216A). El motor debe ser completamente auditable, sin cajas negras, y libre de circularidad con el `malscore`. *Criterio de verificación: Hito H2. η² inter-familia de* `severity_v2` *mayor que η² de* `malscore` *sobre el dataset de Hito H1; test* `TestNoLeakage` *verde.*

OE3. Implementar un motor de anomalía conductual y un corrector de falsos positivos. Entrenar un `IsolationForest` sobre 14 features puramente conductuales, de red y estáticas (operaciones de fichero, registro y mutex; llamadas a API; conteos TCP/UDP/HTTP/DNS; imports PE) que detecte muestras intra-familia cuyo comportamiento se desvíe significativamente de su grupo. En paralelo, implementar el módulo `fp_correction.py` con mecanismos de registro de evidencia benigna (imphash, sha256, nombres de firma, patrones funcionales) que reduzca la urgencia de verdaderos benignos sobre-puntuados por el sandbox. Ambos componentes deben operar sin acceso al `malscore` ni a la familia de la muestra. *Criterio de verificación: Hito H2. Modelo entrenado y persistido en* `data/models/anomaly_v2.joblib`*; 0 falsos benignos en el dataset de entrenamiento 100 % malware; tests de anti-leakage verdes.*

OE4. Diseñar e implementar un motor de fusión de urgencia multiplicativo 0-10. Combinar los tres componentes anteriores (severidad, anomalía, corrección de FP) en una puntuación única de urgencia para el SOC mediante la fórmula: `urgencia = clip(severity × anomaly_boost × fp_penalty × context_multiplier, 0, 10)`, donde cada factor tiene interpretación directa (boost del +20 % máximo si la muestra es conductualmente atípica; penalización de hasta −80 % si hay evidencia de benigno; multiplicador de contexto configurable por criticidad de host y privilegios de usuario). La puntuación numérica 0-10 se mapea a cuatro buckets categóricos (crítico / alto / medio / bajo) para la visualización en la interfaz de analista. *Criterio de verificación: Hito H3. Tests de monotonicidad, caps y anti-leakage verdes; cola de 500 muestras Avast-CTU generada con distribución no plana (no más del 5 % en el bucket máximo).*

OE5. Construir un componente de síntesis SecOps accionable con interfaz dual. Implementar el módulo `secops_summary.py` que genere para cada informe una ficha narrativa en español con los campos {qué hizo, por qué importa, IOCs, acción recomendada, narrativa}. El módulo debe soportar dos modos con el mismo contrato de salida: modo plantilla (NLG determinista, sin dependencias externas) y modo LLM (Ollama local con Phi-4-mini o Llama 3.2 3B, con encuadre de tarea como análisis defensivo SOC para evitar rechazos por alineación de seguridad), con degradación automática al modo plantilla si Ollama no está disponible. Medir la fidelidad de cada modo mediante cinco comprobaciones automáticas (mención de familia, valor numérico de urgencia, bucket categórico, al menos una técnica MITRE, campos completos). *Criterio de verificación: Hito H3. Modo plantilla con fidelidad ≥ 0,60 verificada sobre ≥ 2 muestras reales; degradación automática testeada; anti-leakage de* `malscore` *en la firma del generador.*

OE6. Evaluar cuantitativamente el sistema frente al malscore bruto de CAPE. Medir con métricas de ranking (η² inter-familia, correlación de Spearman, rank_shift por muestra) la mejora del sistema v2 respecto al malscore como único criterio de priorización. Documentar los casos extremos de divergencia (muestras que el malscore sobre-puntúa y el sistema rebaja, y viceversa) con justificación en términos de familia, TTPs y comportamiento observado. Incluir la comparativa empírica del componente de síntesis (plantilla vs LLM) con las métricas de fidelidad automática y, en la medida de lo posible, con evaluación cualitativa según la rúbrica MESSALA (completitud, precisión, accionabilidad, concisión, ausencia de alucinaciones). *Criterio de verificación: Hito H4. Tablas de η², distribución de buckets y rank_shift presentes en los reportes de evaluación; al menos 5 casos de divergencia documentados.*

OE7. Desarrollar una API REST y una UI Atalaya operativos con trazabilidad completa. Im-

plementar y documentar los endpoints del sistema (`POST /api/v1/events/ingest`, `POST /api/v1/score`, `GET /api/v1/incidents`, `GET /api/v1/incidents/{id}`, `GET /api/v1/status/model`) y una interfaz de usuario que muestre la cola de incidentes ordenada por urgencia descendente, la ficha detallada de cada incidente (incluyendo el desglose de los componentes de urgencia, la comparativa con el `malscore` original de CAPE y el resumen narrativo de síntesis) y los metadatos de trazabilidad dato→urgencia→ficha. La suite de pruebas automatizadas debe cubrir todos los módulos del pipeline v2. *Criterio de verificación: Hito H4. Flujo end-to-end funcional; 199 tests automatizados pasando (parser v2, severity, anomaly, fp, fusion, synthesis, API, calidad y baseline Gate 5).*

3.3 Preguntas de investigación

Los objetivos específicos anteriores se articulan en torno a tres preguntas de investigación que estructuran el diseño experimental del trabajo.

Las tres no se eligen por conveniencia: se corresponden con las propiedades que la literatura atribuye a una alarma útil. Alahmadi et al. [20], entrevistando a analistas de SOC en activo, identifican cinco: que la alarma sea **fiable**, **explicable**, **analítica**, **contextual** y **transferible**. RQ1 interroga la propiedad **contextual** (¿aporta el contexto una priorización más discriminativa que la puntuación aislada del sandbox?); RQ2, la propiedad **fiable** (¿baja el sistema el rango de las alarmas que no merecen la atención que reclaman?); y RQ3, la propiedad **analítica** (¿entrega la ficha lo necesario para actuar?). La propiedad **explicable** no se formula como pregunta porque no es una hipótesis a contrastar sino un requisito de diseño no negociable, verificado por construcción: la urgencia es una función determinista descompuesta componente a componente (§5.6). La **transferible** queda fuera del alcance empírico de este trabajo, porque requeriría un segundo entorno, y se discute como limitación (§8.4.1).

RQ1. ¿Produce la re-valoración contextual una priorización de alertas más discriminativa que el `malscore` bruto de CAPEv2? El `malscore` de CAPE es una puntuación global de maliciosidad que refleja la intensidad del análisis dinámico pero no captura información sobre el tipo de amenaza, su relevancia para el entorno afectado ni el impacto potencial de las técnicas observadas. Esta pregunta evalúa si el motor de urgencia v2, que combina familia, TTPs MITRE, comportamiento y contexto del activo, distribuye las alertas de forma más discriminativa, medido mediante la varianza inter-familia explicada (η^2) y la correlación de Spearman con la gravedad real de cada tipo de amenaza.

RQ2. ¿Reduce el sistema el rango en la cola de priorización de los falsos positivos conocidos (muestras benignas sobre-puntuadas por el sandbox)? Los sandboxes de análisis dinámico tienden a sobre-puntuar muestras que exhiben comportamiento técnicamente visible pero sin intención maliciosa (binarios de administración legítimos, instaladores, herramientas de diagnóstico). Esta pregunta evalúa si los componentes de corrección de FP y fusión de urgencia rebajan el rango en la cola de este tipo de muestras respecto al orden que impondría el `malscore` solo, cuantificado mediante el desplazamiento de ranking (`rank_shift`) de los casos documentados.

RQ3. ¿Es el resumen generado por el componente de síntesis completo, preciso y accionable para un analista de SOC? La utilidad operativa del sistema depende en parte de si la ficha narrativa que entrega al analista incluye los datos relevantes (familia, urgencia, TTPs, IOCs, acción recomendada) sin inventar información ni perder campos clave. Esta pregunta evalúa la fidelidad del componente de síntesis mediante el protocolo `payload_fidelity` (cinco comprobaciones automáticas sobre la cobertura de campos) y, cualitativamente, mediante la rúbrica MESSALA aplicada por el propio autor sobre una muestra representativa de fichas generadas.

3.4 Hipótesis

Las hipótesis de trabajo son falsables, directamente vinculadas a las preguntas de investigación y verificables sobre el dataset Avast-CTU Public CAPE Dataset con los métodos descritos en el capítulo 7.

H1 *(asociada a RQ1)*: La puntuación de severidad producida por el motor v2 (que integra tipo ordinal de amenaza, peso de firmas CAPE, impacto de técnicas MITRE ATT&CK y presencia en la lista CISA) presenta una varianza inter-familia significativamente mayor que la del `malscore` de CAPE, expresada como razón η^2 (varianza entre grupos / varianza total). En particular, se espera $\eta^2(\text{severity_v2}) > \eta^2(\text{malscore})$ con una ventaja de al menos 20 puntos porcentuales sobre el dataset de evaluación, reflejando que el motor v2 discrimina entre familias cuyo `malscore` es indistinguible (por ejemplo, saturación en 10.0 para Qakbot, Emotet y Trickbot simultáneamente).

H2 *(asociada a RQ2)*: El motor de fusión de urgencia desplaza hacia posiciones más bajas en la cola de priorización las muestras que el `malscore` sobre-puntúa (familias de bajo impacto real con comportamiento poco activo), y hacia posiciones más altas las muestras que el `malscore` infra-puntúa por ser de familia de alto impacto con comportamiento conductualmente marcado. Este efecto es documentable mediante el indicador `rank_shift` calculado para cada muestra como la diferencia entre su posición en la cola ordenada por urgencia v2 y su posición en la cola ordenada por `malscore`.

H3 *(asociada a RQ3)*: El componente de síntesis en modo plantilla determinista alcanza una fidelidad automática de al menos 0,60 sobre el conjunto de verificación {menciona familia, menciona valor numérico de urgencia, menciona bucket categórico, menciona al menos una técnica MITRE, campos completos en el JSON de salida}. En la comparativa empírica, los modelos LLM de tamaño reducido (3-4B parámetros en CPU sin aceleración por GPU) mostrarán fidelidad inferior a la de la plantilla, reflejando que la fluidez narrativa de los LLMs pequeños viene a expensas de la precisión en la cita de valores numéricos y de la cobertura de todos los campos requeridos.

3.5 Alcance, restricciones y criterios de éxito

3.5.1 Alcance del sistema

El prototipo abarca las siguientes capacidades verificables en el entorno de laboratorio:

- Ingesta de informes CAPE en formato JSON (esquemas 2.2 y 2.5) mediante el parser v2, con extracción de features de comportamiento, red, firmas y TTPs.
- Anonimización obligatoria de campos sensibles (IPs, hashes, paths) antes de cualquier persistencia, mediante hash SHA-256 con sal configurable.
- Motor de severidad intrínseca 0-10 con cuatro componentes auditables y sin dependencia del `malscore`.
- Motor de anomalía conductual basado en `IsolationForest` entrenado sobre 14 features puramente conductuales.
- Corrector de falsos positivos extensible con registros de evidencia benigna por imphash, sha256, nombre de firma y patrones funcionales.
- Motor de fusión de urgencia 0-10 con fórmula multiplicativa y bucketing categórico configurable.
- Componente de síntesis SecOps con interfaz dual (plantilla determinista / LLM local Ollama) y degradación automática.
- API REST documentada con OpenAPI automático (FastAPI) y UI Atalaya con cola priorizada y ficha detallada.

- Pipeline de ingesta del dataset Avast-CTU (versión completa 48.976 reportes / versión reducida 566 MB para desarrollo) con split temporal estricto.
- Registro de experimentos con MLflow local y trazabilidad completa dato→urgencia→ficha.
- Modo offline-first: la demo de defensa puede ejecutarse sin conectividad de red, con modelos preentrenados y dataset local.

3.5.2 Exclusiones explícitas

Quedan fuera del alcance de este TFG los siguientes elementos:

Integración de logs del sistema operativo en tiempo real con el sandbox. La correlación entre el informe de sandbox y los registros de red o de endpoint del mismo incidente requiere un dataset que combine ambas fuentes sobre las mismas muestras. A fecha del trabajo, no existe un dataset público que lo proporcione (confirmado en el análisis de fuentes del proyecto). La arquitectura del sistema contempla una interfaz de fuente multi-origen preparada para esta extensión, pero su implementación y evaluación quedan documentadas como trabajo futuro.

Integración de un dataset de benignos en el corrector de falsos positivos. El módulo `fp_correction.py` dispone de la interfaz para alojar listas de imphashes y sha256 de binarios benignos conocidos, pero la carga de las 11.735 muestras benignas del dataset Cuckoo de Ilić et al. [21] como tabla de referencia se ha aplazado al ciclo de desarrollo posterior a la evaluación inicial.

Validación con muestras propias en CAPE 2.5. La ejecución de binarios de MalwareBazaar en la VM de análisis de la Universidad (tower Intel cedida por el tutor) está condicionada a que la infraestructura de la torre esté operativa. El grueso de la evaluación se realiza sobre el dataset Avast-CTU, que son informes CAPE reales ya generados. La validación propia con CAPE 2.5 queda pendiente y, si se cierra antes de la entrega, se incorpora al capítulo 7 como evidencia complementaria.

Evaluación con analistas SOC externos. La evaluación de la ficha narrativa mediante la rúbrica MESSALA se realiza por el propio autor sobre una muestra representativa. Los estudios de usuario con analistas reales quedan fuera del alcance académico de este TFG.

Escalabilidad en producción. El prototipo está diseñado para entornos de laboratorio y defensa académica; no se evalúa su rendimiento bajo cargas de producción reales ni su integración con plataformas SIEM/SOAR externas.

3.5.3 Criterios de éxito (hitos del MVP v2, H1–H4)

El avance entre fases del proyecto se controla mediante cuatro hitos de verificación del MVP v2 (denominados H1-H4 para distinguirlos del Gate 3 de validación CAPE en vivo, tratado en el capítulo 7) con criterios de aceptación medibles. Ningún hito comienza sin que el anterior esté cerrado.

Tabla 3.1: Hitos de verificación del MVP v2 (H1–H4) y criterios de calidad.

Puerta	Criterio principal	Criterio de calidad
Hito H1. Ingesta sin leakage	Dataset Avast-CTU ingestado ≥ 200 muestras; etiquetas de familia desde fuente externa	Split temporal 70/15/15; test `test_avast_no_label_derivation` verde
Hito H2. Motores autónomos	η^2(severity_v2) > η^2(malscore) inter-familia; `anomaly_v2.joblib` entrenado; 0 falsos benignos en dataset malware	Tests de anti-leakage verdes en los tres motores; suite ≥ 142 tests

Puerta	Criterio principal	Criterio de calidad
Hito H3. Cola priorizada y síntesis	Cola priorizada con urgency_score; distribución no plana (≤ 5 % en bucket crítico); síntesis plantilla fidelidad ≥ 0,60	Tests de monotonicidad, caps y fusión verdes; modo auto-degradable funcional; suite ≥ 199 tests
Hito H4. Evaluación completa	Tablas η^2, rank_shift y fidelidad exportadas; ≥ 5 casos de divergencia documentados; UI Atalaya operativa	Flujo end-to-end funcional; demo reproducible sin Ollama; informe de evaluación generado

El cierre de Hito H4 constituye la condición de entrega del prototipo para la defensa del TFG. La validación complementaria con muestras propias en CAPE 2.5 (torre de la Universidad) se incorpora al capítulo 7 si está disponible antes del depósito de la memoria; en caso contrario, se documenta como trabajo futuro pendiente con el protocolo de transferencia ya redactado.

Capítulo 4. Metodología

4.1 Enfoque metodológico general

Este trabajo adopta un enfoque de **ingeniería aplicada con experimentación empírica**. No se trata de investigación básica orientada a la teoría, ni de desarrollo de software puro sin hipótesis verificables: el objetivo es construir un sistema funcional cuyo comportamiento se evalúa de forma rigurosa mediante un protocolo experimental definido antes de la implementación, siguiendo el principio de preregistro implícito habitual en ingeniería de sistemas inteligentes.

El ciclo de trabajo se articula en cuatro fases iterativas que se refuerzan mutuamente:

1. **Diseño**: definición de contratos de datos, arquitectura del sistema y protocolo experimental antes de escribir código de producción.
2. **Implementación**: construcción incremental por puertas de calidad, sin avanzar de fase sin haber cerrado la puerta anterior.
3. **Evaluación**: medición de métricas primarias y secundarias sobre datos reservados, con reportes exportables y reproducibles.
4. **Validación**: contraste de hipótesis con los resultados, análisis de errores y documentación de limitaciones.

La elección de un motor de decisión determinista y auditable frente a un clasificador de caja negra responde a un criterio deliberado: en un contexto de seguridad donde el analista necesita comprender y poder cuestionar cada posición en la cola, la **trazabilidad de la decisión** es un requisito funcional, no solo una preferencia metodológica. El aprendizaje automático ocupa un rol auxiliar (un Isolation-Forest detecta comportamiento atípico intra-familia; un LLM local genera la síntesis narrativa), pero no determina la urgencia. La urgencia final es el resultado de una función aritmética con cuatro componentes acotados y verificados por contrato, lo que garantiza que cualquier evaluador externo pueda reproducir el cálculo dado los mismos inputs.

4.2 Gestión del proyecto

4.2.1 Estructura por puertas de calidad

El proyecto se organiza en seis fases separadas por puertas de calidad del prototipo v1 (Gates 0–5; hoy hitos históricos, que no deben confundirse con los hitos H1-H4 del MVP v2 de cap. 3 ni con el Gate 3 de validación CAPE en vivo de cap. 7). Cada puerta define un conjunto de entregables verificables que deben estar completos y validados antes de iniciar la fase siguiente. Este mecanismo previene dos riesgos habituales en proyectos de TFG: la acumulación de deuda técnica no documentada y el avance sobre fundamentos frágiles.

Tabla 4.1: Fases del proyecto y puertas de salida.

Fase	Semanas	Puerta de salida
Fase A. Dataset real	1–3	Parser CAPE v2 ingestando informes Avast-CTU sin leakage; manifiesto versionado publicado
Fase B. Motor de severidad	3–5	`compute_severity()` con 4 componentes; tests `TestNoLeakage` verdes
Fase C. Anomalía + FP	5–7	IsolationForest sobre features conductuales; `fp_correction.py` con interfaz de benignos
Fase D. Fusión de urgencia	7–8	`urgency_fusion.py`; buckets categóricos; cola priorizada en API
Fase E. Síntesis LLM	8–10	`secops_summary.py` con modo LLM y plantilla; ficha estructurada validada
Fase F. Evaluación + UI	10–13	Métricas de ranking (Spearman, η^2, rank_shift); UI Atalaya; demo reproducible

Las fases A-F del pivote v2 se completaron entre los meses de mayo y junio de 2026 (cierre de la fase F en junio de 2026, con 225 tests en verde en ese momento). La decisión de diseño que origina esta estructura de fases se documenta en el diseño de arquitectura v2 y se motiva en §7.9.2, donde se detallan los defectos del prototipo v1 que la originaron.

4.2.2 Control de versiones y trazabilidad

Todo el código, la documentación y los artefactos de datos se gestionan en un único repositorio Git. La política de ramas establece ramas de trabajo temáticas (desarrollo técnico y documentación) que convergen en la rama principal mediante revisión explícita. Los scripts de entrenamiento y evaluación generan artefactos versionados (`dataset_version`, `model_version`, `feature_set_version`) que se registran en MLflow y se persisten en `data/models/` y `reports/`, garantizando la reproducibilidad completa de cualquier corrida anterior.

4.2.3 Gestión de riesgos

Se identificaron siete categorías de riesgo antes del inicio del desarrollo. La tabla siguiente resume los de mayor impacto y sus estrategias de mitigación activas:

Tabla 4.2: Riesgos del proyecto y mitigaciones principales.

Riesgo	Impacto	Mitigación principal
Saturación del malscore → sistema sin discriminación real	Alto	Señales independientes del malscore (tipo+CISA+TTPs); evaluación cuantitativa de la divergencia
Leakage circular entre etiqueta y score de CAPE	Alto	Etiquetas de `public_labels.csv` (fuente externa); 13 tests `TestNoLeakage` en 6 módulos
Dataset insuficiente para anomalía intra-familia	Medio	IsolationForest con 14 features conductuales; subconjunto Avast-CTU estratificado

Riesgo	Impacto	Mitigación principal
LLM local sin disponibilidad en demo	Medio	Degradación garantizada a plantilla determinista (`DEFAULT_SUMMARY_MODE="template"`)
Dependencia de VM/CAPE para validación en vivo	Medio	Dataset Avast-CTU (reales de CAPE); Gate 3 declarado trabajo futuro con protocolo escrito
Complejidad de alcance	Alto	MVP defendible, multi-fuente y Gate 3 fuera del MVP
Riesgo legal/ético de datos	Alto	Solo datos públicos (Avast-CTU, licencia propia revisada), sin PII en artefactos

---

4.3 Estrategia de datos

4.3.1 Fuentes y política de datos

Fuente principal: Avast-CTU Public CAPE Dataset [arXiv:2209.03188]: conjunto de informes JSON reales generados por CAPEv2, procedentes de diez familias de malware (Emotet, Qakbot, Trickbot, Lokibot, njRAT, Zeus, Ursnif, Adload, Swisyn, HarHar). El dataset está disponible en dos versiones: `reduced` (566 MB, subconjunto de desarrollo) y `full` (13,97 GB, 48.976 informes; versión de ingesta `20260626_225446`). Las etiquetas de familia y tipo de amenaza se obtienen exclusivamente de `public_labels.csv` (una fuente **externa** al contenido de cada informe), lo que es la base de la garantía anti-leakage del sistema. La verificación de que ningún módulo del pipeline deriva etiquetas del malscore interno del informe se realiza mediante el test `test_avast_no_label_derivation` y el conjunto de 13 tests `TestNoLeakage`.

Fuente secundaria para FP: dataset de Ilić et al. [21]: 11.735 muestras benignas para poblar el módulo `fp_correction.py`. Pendiente de integración como trabajo futuro (§4.4.4 y cap. 8).

Fuente de validación en vivo: VM CAPEv2 propia (Gate 3): la VM `ubuntu-cape` (Ubuntu 22.04.5, CAPEv2 esquema 2.5) permitiría generar informes nuevos sobre muestras controladas. El parser v2 fue validado contra 11 informes de esta VM; la integración E2E está documentada pero declarada trabajo futuro (hardware parado, ver §8.5.1).

La política de datos prohíbe la persistencia de información de identificación personal (PII). Todos los identificadores sensibles presentes en los informes CAPE se someten a anonimización antes de cualquier persistencia (módulo `anonymizer.py`, hash SHA-256 con sal configurable).

4.3.2 CapeFeatures: el contrato de extracción del parser

El parser `cape_parser_v2.py` extrae del informe JSON de CAPEv2 un objeto `CapeFeatures` con los campos necesarios para el pipeline v2. Los campos se organizan en cuatro grupos:

- **Metadatos de identificación**: `sha256`, `family`, `type_` (de `public_labels.csv`), `malscore` (conservado como referencia comparativa, nunca como input de valoración).
- **Firmas de comportamiento**: `sig_max_severity`, `sig_sum_weight`, `sig_high_severity_count` (agregados de `signatures[]`).
- **TTPs MITRE ATT&CK**: lista de `ttp_codes` (de `signatures[].ttp`, si presentes en el informe).
- **Features conductuales para IsolationForest** (14 fields): operaciones de fichero, registro y mutex; llamadas a API; conteos TCP/UDP/HTTP/DNS; imports PE estáticos.

El `EventSchema v1` del prototipo v1 (con 21 features numéricas) sigue disponible en la API `/api/v1/*` para preservar la compatibilidad con el clasificador v1. El pipeline v2 opera sobre `CapeFeatures`, no sobre `EventSchema v1`.

Los campos `sha256`, `src_ip` y `dst_ip` se someten a anonimización (módulo `anonymizer.py`) antes de cualquier persistencia.

4.3.3 Anonimización

La anonimización se ejecuta obligatoriamente en el módulo `anonymizer.py` como primer paso del pipeline de ingesta, antes de la validación de calidad y la persistencia. Los campos sensibles se sustituyen por un hash SHA-256 con sal (formato `sha256(sal:valor)`) prefijado con `anon:`, generado con una sal configurable (`TFG_ANON_SALT`). El módulo devuelve el evento anonimizado junto con un metadato (`anonymization_meta`) que registra qué campos fueron transformados, garantizando la trazabilidad del proceso sin exponer los valores originales.

4.3.4 Validación de calidad

El módulo `quality.py` aplica en ingesta las siguientes reglas mínimas:

- Unicidad de `event_id` dentro del lote y respecto a eventos ya persistidos.
- Coherencia temporal: el timestamp no puede ser nulo ni estar en el futuro remoto.
- Presencia de al menos un indicador de actividad (red, comportamiento o IOC no vacío).

Los eventos que no superan estas reglas se rechazan con motivo explícito y se incluyen en la respuesta de ingesta (`rejected_items`), sin afectar al procesamiento del resto del lote.

4.3.5 División del dataset

Los componentes deterministas del motor de severidad y fusión de urgencia no requieren partición: son funciones sin parámetros entrenados evaluadas sobre el conjunto completo (n = 48.976).

El IsolationForest, al ser un modelo no supervisado sin etiquetas de clase, se ajusta y evalúa sobre las mismas 48.976 muestras (n_train = n_eval = 48.976); no existe partición train/test separada para este componente. Esta decisión es una consecuencia de la naturaleza no supervisada del modelo (ajuste sobre la distribución completa de comportamientos observados) y se declara explícitamente como amenaza a la validez interna (§4.4.5, §7.7.1): los porcentajes de muestras flagged (capítulo 7) describen el comportamiento del modelo sobre sus propios datos de entrenamiento, no su capacidad de generalización a muestras nuevas.

Cada versión del dataset se registra con un manifiesto que incluye: identificador de versión, fecha de generación, número de informes por familia, hash del fichero `public_labels.csv` utilizado y versión del parser.

4.4 Protocolo experimental

4.4.1 Configuración experimental

La evaluación del sistema v2 no compara configuraciones de clasificador (no hay modelo entrenado a elegir), sino que contrasta la cola de urgencia producida por el motor determinista con el `malscore` bruto de CAPE como línea base. Esto responde a la pregunta central: ¿añade la re-valoración propuesta valor real frente a ordenar simplemente por la puntuación del sandbox?

Línea base, el malscore de CAPE: orden de la cola determinado exclusivamente por `malscore` (campo nativo del informe CAPEv2). Sirve para cuantificar la saturación de partida y la mejora aportada por el sistema (relevante para RQ1).

Sistema v2, el motor de urgencia: orden determinado por `urgency_score`, resultado de la fusión multiplicativa de cuatro componentes (§5.6):

```
urgency_score = clip(severity_v2 × (1 + 0,20·anomaly_score) × fp_penalty × context_mul, 0, 10)
```

donde `severity_v2` es la salida del motor determinista (tipo + firmas CAPE + TTPs MITRE + CISA), `anomaly_score` es la señal auxiliar del IsolationForest, y `fp_penalty` y `context_mul` son ganchos de producción (= 1,0 en el corte offline del MVP; ver §5.6).

El `malscore` de CAPE aparece en la cola v2 como columna comparativa, visible al analista, pero **no es input de ningún componente** del motor de urgencia.

4.4.2 Features por componente del pipeline

El pipeline v2 utiliza grupos de features distintos en cada componente, garantizando la ausencia de circularidad con el malscore en todos ellos:

Tabla 4.3: Features por componente del pipeline y su origen en el informe CAPE.

Componente	Features de entrada	Fuente en el informe CAPE
Motor de severidad (`severity_engine.py`)	`family`, `type_`, `sig_max_severity`, `sig_sum_weight`, `sig_high_severity_count`, `ttp_codes`	`info.category`, firmas, técnicas ATT&CK
Motor de anomalía (`anomaly_engine.py`)	14 features conductuales: operaciones de fichero/registro/mutex, llamadas a API, conteos TCP/UDP/HTTP/DNS, imports PE	Secciones `behavior`, `network`, `static` del informe
Corrector de FP (`fp_correction.py`)	`imphash`, `sha256`, nombres de firma, patrones funcionales	Secciones `static`, `signatures`
Fusión de urgencia (`urgency_fusion.py`)	`severity_v2`, `anomaly_score`, `fp_penalty`, `context_mul`	Salidas de los componentes anteriores

El `malscore` del informe CAPE es accesible al parser pero **no se propaga como input a ninguno de los cuatro componentes**. Su presencia en la salida final es exclusivamente comparativa (columna `malscore_cape` en la UI). Esta restricción está verificada por 13 tests `TestNoLeakage` distribuidos en los 6 módulos del pipeline.

4.4.3 Métricas de evaluación

El sistema v2 no produce una clasificación binaria malicioso/benigno; produce una cola ordenada por urgencia. Las métricas de evaluación reflejan este cambio de paradigma.

Métricas de priorización (primarias): - **Correlación de Spearman (ρ)** entre el ranking por `urgency_score` v2 y el ranking por `malscore` de CAPE, calculada sobre el conjunto completo de informes evaluados. Un ρ alto indica que el sistema es coherente con el sandbox; una divergencia controlada ($\rho < 1$) indica que el sistema efectivamente re-ordena el bloque saturado. - **η^2 inter-familia de `severity_v2`** (eta-cuadrado de la ANOVA), como medida de la capacidad del motor para discriminar entre familias. Se descompone en: η^2 de la señal conductual pura (`sig_part` + `ttp_part`) vs. η^2 del `malscore` de CAPE,

para establecer la comparación justa (sin la circularidad de `type_part`). - **rank_shift bidireccional**: diferencia de posición en la cola entre el ranking v2 y el ranking por malscore, para las muestras de mayor y menor desplazamiento. Cuantifica la re-valoración real del bloque saturado.

Métricas de síntesis (secundarias): - **Fidelidad de payload** (automática): ¿contiene el resumen generado los valores correctos del contrato (urgencia, bucket, familia, TTPs) según el JSON de entrada? - **Rúbrica MESSALA** (humana, sobre submuestra): completitud, precisión, accionabilidad, concisión y ausencia de alucinaciones. La evaluación automática de fidelidad se realizó sobre n=2 muestras Qakbot en el corte de junio de 2026 (plantilla 0,60, phi4-mini 0,40, llama3.2:3b 0,20; §7.6 discute la asimetría estructural de la métrica y un sesgo del propio evaluador en el formato numérico); la rúbrica humana sobre muestra mayor (30-50 fichas) queda como trabajo futuro (§8.5.3).

Latencia operativa: tiempo de procesamiento por informe del pipeline batch (`scripts/build_priority_queue.py`, medido de forma agregada). La latencia p95 del endpoint `POST /api/v2/triage` no se midió en este corte y queda declarada como medición de trabajo futuro (§8.5).

4.4.4 Estudios adicionales

Además de la evaluación principal, se realizan tres estudios complementarios:

Descomposición del η^2 inter-familia: el η^2 total de `severity_v2` incluye la contribución de `type_part`, que es circular por diseño (en el dataset Avast-CTU cada familia tiene un tipo único). Para una comparación justa con el `malscore` de CAPE, se calcula el η^2 de la señal conductual pura (solo `sig_part` y `ttp_part`) y se contrasta con el η^2 del `malscore`. Este análisis responde a RQ1 sin el sesgo de circularidad del componente de tipo.

Análisis de la divergencia bidireccional: se identifican las muestras con mayor desplazamiento positivo (`rank_shift` > 0, promovidas) y negativo (`rank_shift` < 0, rebajadas) respecto al ranking por malscore. Para cada grupo se inspeccionan los factores que explican el desplazamiento (familia, TTPs, presencia en CISA, comportamiento) y se verifica que la reordenación es coherente con el conocimiento experto sobre las familias en cuestión. Este análisis es la evidencia primaria para responder a RQ2.

Evaluación comparativa de la síntesis LLM: se comparan los modos `template` y `llm` del módulo `secops_summary.py` sobre un subconjunto de informes. Para cada modo se mide la fidelidad de payload de forma automática y se aplica la rúbrica MESSALA sobre una submuestra con evaluación humana. Los dos modos de fallo documentados (omisión del valor numérico de urgencia en `llama3.2:3b` y confusión TTPs/IOCs en `phi4-mini`) se analizan como evidencia de las limitaciones de la síntesis LLM en este contexto (responde a RQ3). Evaluación completada sobre n=2 (§7.6); extensión a muestra mayor con rúbrica MESSALA es trabajo futuro.

4.4.5 Amenazas a la validez

Se identifican las siguientes amenazas a la validez del experimento y sus estrategias de mitigación:

Tabla 4.4: Amenazas a la validez y mitigaciones.

Amenaza	Tipo	Mitigación
Circularidad η^2 por `type_part` (familia = tipo en Avast-CTU)	Validez interna	Descomposición explícita: η^2 conductual puro vs. η^2 total; comparación justa con malscore
Dataset único, no estratificado (10 familias de malware)	Validez externa	Declarado explícitamente como limitación; trabajo futuro con n_full y un dataset de benignos [21]
Motor de anomalía sin split train/test estricto	Validez estadística	IsolationForest no supervisado; declarado como limitación en cap08 §8.4

Amenaza	Tipo	Mitigación
FP corrector sin recall evaluado (dataset benignos vacío)	Validez de constructo	Declarado; integración de un dataset de benignos [21] como trabajo futuro
Evaluación de síntesis sobre n=2 fichas	Validez estadística	Resultados presentados como exploración, no como estimación poblacional; MESSALA pendiente
Desfase dataset/producción (Avast-CTU ≠ malware emergente)	Validez ecológica	Documentación de limitación; robustez de `type_part` como trabajo futuro

4.5 Herramientas y entorno

4.5.1 Stack tecnológico

Tabla 4.5: Stack tecnológico del prototipo.

Componente	Herramienta	Versión	Rol
Lenguaje	Python	3.11	Todo el prototipo
API	FastAPI + Uvicorn	0.115 / 0.34	Backend REST y UI
Validación	Pydantic	2.10	Contratos de datos
ML	scikit-learn	1.6.1	Modelos supervisado y anomalía
Datos	pandas + numpy	2.2 / 2.2	Procesamiento tabular
Tracking	MLflow	2.20	Registro de experimentos
Serialización	joblib	1.4	Persistencia de modelos
Tests	pytest	8.3	Suite de tests funcionales
HTTP cliente	httpx	0.28	Tests de contrato API

4.5.2 Entorno de ejecución

El prototipo está diseñado para ejecución local sin dependencias de infraestructura en la nube. Todos los artefactos (dataset, modelos, trazas MLflow, reportes de evaluación) se persisten en el directorio `prototipo/data/`, configurable vía variable de entorno `TFG_DATA_DIR`. La UI se sirve desde el propio proceso FastAPI mediante plantillas Jinja2, eliminando la necesidad de un servidor web separado.

Esta decisión de diseño, la de ser *offline-first*, responde directamente al riesgo de dependencia de servicios externos (Riesgo 4): la demo de defensa puede ejecutarse en un portátil sin conectividad, con modelos pre-entrenados y datos pre-cargados, lo que garantiza la reproducibilidad en el momento de la presentación.

4.5.3 Entorno de sandbox: CAPEv2

La infraestructura de análisis dinámico de malware se despliega en una VM L1 (ubuntu-cape, Ubuntu 22.04.5) con KVM/QEMU sobre el host principal. La decisión metodológica central es usar **el script de instalación oficial** `cape2.sh base cape` como único método de despliegue, descartando la configuración manual. Esto garantiza reproducibilidad del entorno y elimina dependencias de configuración ad hoc que no podrían documentarse de forma exhaustiva.

Decisión de diseño: QEMU del sistema vs. QEMU compilado con parches anti-detección. La documentación de CAPEv2 recomienda `kvm-qemu.sh`, que compila QEMU 9.2.2 desde fuente con parches

anti-sandbox-detection (SeaBIOS ofuscado, ACPI falsificado). Para este TFG (estudio académico, muestras de baja sofisticación como EICAR) se opta por el QEMU del sistema (`apt`) y se documenta explícitamente como limitación: muestras con detección activa de VM pueden evadir el sandbox. El detalle completo de la instalación, los problemas encontrados y su resolución se documenta en el capítulo 6 (Implementación).

Observación de reproducibilidad: Windows Hypervisor Platform (WHP). Durante la replicación del entorno en un host Windows 11 con Hyper-V deshabilitado por BCD (`bcdedit /set hypervisorlaunchtype off`), VirtualBox reportaba aun así `HypervisorPresent=True` dentro de la VM invitada, impidiendo la activación de la virtualización anidada (KVM). La causa fue la *feature* **Windows Hypervisor Platform** (WHP), un hipervisor ligero orientado a WSL2 y Android Subsystem que existe de forma independiente al Hyper-V completo y que no se desactiva con `bcdedit`. La solución es ejecutar antes de la instalación de VirtualBox: `dism /online /disable-feature /featurename:HypervisorPlatform /norestart`. Esta distinción no aparece documentada en los playbooks de instalación de CAPEv2 y constituye una lección de reproducibilidad relevante para cualquier despliegue de virtualización anidada sobre Windows 10/11.

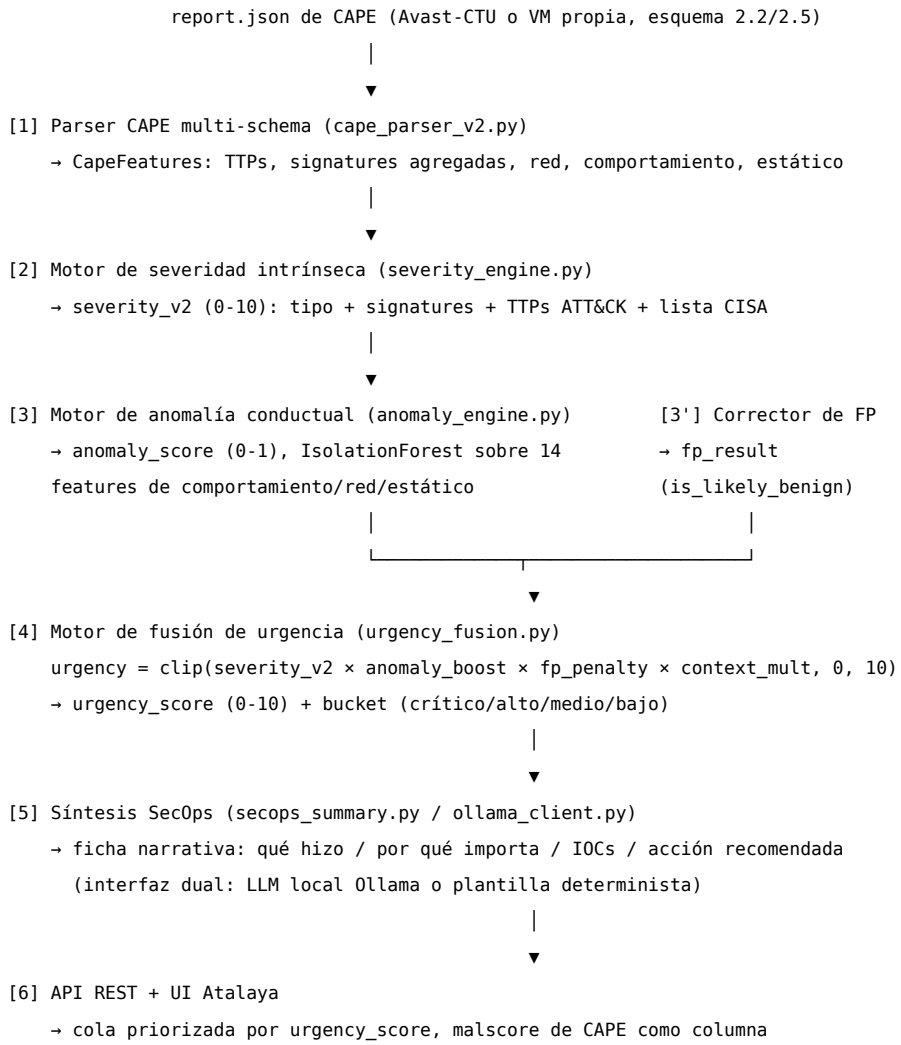
4.5.4 Reproducibilidad

La reproducibilidad completa de cualquier experimento requiere fijar cuatro parámetros: `dataset_version` (semilla y parámetros de generación), `model_version`, `feature_set_version` y `training_window`. Estos cuatro parámetros se registran en cada corrida de MLflow y se incluyen en la respuesta de cada petición de scoring, permitiendo auditar retroactivamente con qué versión del sistema se evaluó cualquier evento.

Capítulo 5. Diseño y arquitectura del sistema

5.1 Visión general

La versión 2 del sistema reorganiza el pipeline en torno a una idea central: **CAPE ya clasifica la muestra y emite un veredicto (`malscore`)**; el sistema no repite ese trabajo, sino que **re-valora ese veredicto con señales independientes** y lo traduce en una urgencia accionable para el analista de SOC. El pipeline se compone de seis elementos con dependencias unidireccionales:



`comparativa, ficha de incidente con desglose y resumen narrativo`

Cada componente expone un contrato de entrada/salida explícito y **ninguno acepta el `malscore`/`cape_score` como argumento de su lógica de decisión**: la urgencia se construye desde abajo (familia, TTPs, comportamiento, contexto), y el `malscore` original de CAPE solo se reintroduce al final, como dato comparativo en la cola y en el prompt de síntesis. Esta restricción, verificada mediante tests introspectivos en cada módulo (`TestNoLeakage`), es la que corrige el leakage circular detectado en el prototipo v1 (ver §5.10 y §7.9.2).

El bloque de enriquecimiento contextual del diseño original se materializa como el `context_multiplier` del motor de fusión (§5.6): un gancho documentado y testeado, pero sin fuente de contexto real conectada en el MVP, conforme a la decisión de dejar la integración multi-fuente fuera de alcance, con la interfaz preparada.

Fundamento de la arquitectura: qué hace útil a una alarma

Esta organización no responde solo a una intuición de ingeniería. Alahmadi et al. [20], a partir de entrevistas con analistas de SOC en activo, derivan cinco propiedades que caracterizan a una alarma útil desde el punto de vista de quien la atiende. Sirven como criterio de diseño y, leídas contra el pipeline anterior, explican por qué el sistema tiene la forma que tiene:

Tabla 5.1: Propiedades de una alarma útil según Alahmadi et al. [20] y cómo las aborda el sistema.

Propiedad [20]	Cómo la aborda el sistema
Fiable (*reliable*): que merezca la atención que reclama	El corrector de falsos positivos y la rebaja de muestras con veredicto alto pero conducta nula (§7.5) atacan justamente las alarmas que no merecen su posición
Explicable (*explainable*): que el analista entienda por qué se ha disparado	La urgencia es una función determinista descompuesta componente a componente, y la ficha expone ese desglose junto al `malscore` original (§5.6, §5.8)
Analítica (*analytical*): que aporte los datos necesarios para decidir	La síntesis SecOps (componente 5) entrega familia, técnicas MITRE, IOCs y acción recomendada sin obligar a leer el JSON de CAPE (§5.7)
Contextual (*contextual*): que incorpore el contexto de la amenaza y del entorno	Es el eje del motor de severidad (componente 2): tipo de amenaza, notoriedad en inteligencia pública y tácticas ATT&CK observadas, información que el `malscore` no usa (§5.4)
Transferible (*transferable*): que su valoración sea trasladable entre entornos	La política vive en constantes nombradas y auditables, no en un modelo entrenado sobre un corpus concreto: se puede revisar y reajustar para otro SOC (§5.4, §7.7.4)

La cuarta propiedad merece subrayarse porque es, literalmente, la tesis de este trabajo. El `malscore` de CAPE es una medida de **evidencia de malicia** producida en aislamiento: no sabe qué tipo de amenaza es, ni si esa familia está bajo seguimiento activo, ni qué tácticas implica lo observado. La saturación documentada en §7.2 es la consecuencia previsible de puntuar sin contexto. Que la literatura identifique el contexto como propiedad exigible a una alarma, y no como un adorno, es el respaldo académico de la decisión central de la arquitectura: **re-valorar el veredicto del sandbox con señales que el sandbox no tiene**.

5.2 Contrato de datos: de EventSchema v1 a `CapeFeatures`

El prototipo conserva `EventSchema v1` como contrato de ingesta general, con Pydantic, `source_type` (`cape | firewall | siem | endpoint | other`), bloques `IOCData`, `NetworkData`, `BehaviorData` y anonimización obligatoria, para no cerrar la puerta a fuentes distintas de CAPE en el futuro (interfaz multi-fuente). Sin embargo, el **pipeline de valoración v2 no opera directamente sobre `EventSchema`**: opera sobre `CapeFeatures`, una estructura intermedia específica de informes CAPE que produce el parser v2 (§5.3) y que alimenta los componentes 2 a 4.

`CapeFeatures` agrupa:

Tabla 5.2: Contrato de datos `CapeFeatures`: bloques y origen en el informe CAPE.

Bloque	Campos principales	Origen en el report CAPE
TTPs	`ttps: set[str]`: códigos `T####` validados, deduplicados	`signatures[].ttp` / `signatures[].ttps` (3 variantes, ver §5.3)
Signatures agregadas	`signature_count`, `sig_max_severity`, `sig_sum_weight`, `sig_mean_severity`, `sig_high_severity_count`, `signature_names` (top 50), `categories`/`families`/`mbcs` (solo CAPE 2.5)	`signatures[]`
Red	`tcp_count`, `udp_count`, `http_count`, `dns_count`, `hosts_count`, `domains_count` (deduplicados)	`network` (defensivo a `None`/lista vacía)
Comportamiento	`file_ops`, `registry_ops`, `mutex_ops`, `resolved_apis_count`, `executed_commands_count`, `process_count`	`behavior.summary`
Estático	`pe_imports_count`, `pe_sections_count`	`static` (PE)
Detección (opcional)	`detections`, `malfamily_tag`	Solo presentes en Avast-CTU; ausentes en informes CAPE 2.5 propios

Todos los campos textuales potencialmente sensibles (hosts, IPs, dominios, hashes de usuario) pasan por `_anon()` de `anonymizer.py` antes de construir `CapeFeatures`, igual que en el pipeline de ingesta general.

`CapeFeatures` **no contiene `family`, `type` ni `label`**: esos campos se incorporan después, desde una fuente externa al report (`public_labels.csv` de Avast-CTU para el dataset de desarrollo), precisamente para que la severidad (§5.4) no pueda derivarse circularmente del propio veredicto de CAPE.

5.3 Componente 1: parser CAPE multi-schema (`cape_parser_v2.py`)

El parser v1 asumía un único formato de informe y derivaba el `label` del `cape_score` (la causa raíz del leakage de v1). El parser v2, `parse_cape_report_v2(report, anonymize=True) -> CapeFeatures`, se diseñó para dos objetivos simultáneos: **eliminar esa derivación y soportar, con la misma función, los informes del dataset Avast-CTU (CAPE 2.2) y los informes generados por la VM de análisis propia cuando esté disponible (CAPE 2.5)**.

Variantes de `ttps` manejadas

La inspección de los datos reales (el análisis de la estructura de Avast-CTU y los informes reales de la VM) reveló tres formatos distintos para el campo de técnicas ATT&CK dentro de `signatures[]`:

Tabla 5.3: Variantes del campo `ttps` manejadas por el parser multi-schema.

Variante	Forma	Origen observado
Diccionario	`{"T1106": "descripción"}`	Avast-CTU (algunos informes 2.2)
Lista con clave singular	`[{"ttp": "T1106", "signature": "..."}]`	Avast-CTU (otros informes 2.2)
Lista con clave plural	`[{"signature": "...", "ttps": ["T1082"], "mbcs": [...]}]`	CAPE 2.5 (VM propia)

`extract_ttps()` normaliza las tres variantes a un único `set[str]` de códigos `T####` validados por expresión regular, descartando cualquier entrada que no encaje (defensa ante "garbage" en el campo). Esta robustez es relevante porque el mismo parser debe servir tanto al desarrollo offline con Avast-CTU como a la validación final con informes reales de CAPE 2.5 (Gate 3, pendiente).

Extracción defensiva del resto de bloques

- `extract_signatures()`: agrega `count`, `max_severity`, `sum_weight`, `mean_severity`, `high_severity_count` (firmas con `severity >= 3`) y los 50 primeros `signature_names`; en CAPE 2.5 añade `categories`, `families` y `mbcs` cuando existen.
- `extract_network()`: tolera `network` ausente, `None` o listas vacías (común en informes CAPE 2.5 sin tráfico capturado); deduplica IPs y dominios antes de contar.
- `extract_behavior()`: tolera `behavior.summary == None`.
- `extract_static()`: extrae conteo de imports y secciones PE si el bloque `static` existe.
- `detections` y `malfamily_tag` son **opcionales y exclusivos de Avast-CTU**: los informes de la VM propia (CAPE 2.5) no los incluyen, y el parser no falla por su ausencia.

Garantía anti-leakage

`parse_cape_report_v2` **no emite `label`, `family` ni `type`** bajo ninguna circunstancia: estos campos se incorporan más tarde desde `public_labels.csv` (fuente externa, independiente del contenido del report). El test `test_avast_no_label_derivation` (clase `TestAvastCTUReal`, `tests/test_cape_parser_v2.py`) verifica explícitamente esta propiedad sobre informes Avast-CTU reales extraídos del dataset.

La suite del parser (26 casos en 9 clases) cubre las tres variantes de TTPs, los casos límite de cada extractor, 3 informes reales de Avast-CTU y 3 informes reales de la VM CAPE 2.5 (`docs/evidencias/cape_reports_reales/`, *gitignored*; los tests se saltan automáticamente si el directorio no existe, manteniendo la suite verde en un clon limpio) y tres casos de informes corruptos o con tipos inesperados que el parser debe degradar sin lanzar excepción.

5.4 Componente 2: motor de severidad intrínseca (`severity_engine.py`)

Este es el motor de severidad del pipeline v2: una función **determinista y auditable**, deliberadamente sin aprendizaje automático, que re-calcula la severidad de una muestra a partir de cuatro señales independientes del `malscore`.

```
compute_severity(family, type_, sig_max_severity, sig_sum_weight,
                 sig_high_severity_count, ttp_codes) -> SeverityResult
```

Fuente de `family` y `type_`: no proceden de `CapeFeatures` ni del campo `detections` del report CAPE (§5.2 excluye explícitamente esos campos de `CapeFeatures`). En el experimento Avast-CTU, ambos vienen de `public_labels.csv` (ground-truth externo al informe); en producción vendrían del pipeline de alertas SIEM/EDR/threat-intel. Nótese que `sig_part` y `ttp_part` sí extraen señales del mismo report CAPE (signatures y TTPs), de modo que la independencia del motor es estricta respecto al **veredicto** (`malscore`), no respecto al report subyacente.

`SeverityResult.score` es la suma de cuatro componentes con topes fijos que suman 10:

Tabla 5.4: Componentes del motor de severidad: rango y cálculo.

Componente	Rango	Cálculo
`type_part`	0 – 3.5	Escala ordinal por tipo de malware (tabla siguiente)
`sig_part`	0 – 3.0	`max_severity` (hasta 1.5) + `sum_weight` normalizado linealmente en [0, 200] (hasta 1.0) + bonus por número de firmas de severidad alta (hasta 0.5)
`ttp_part`	0 – 2.5	Suma de `peso(táctica) × 0.25` por cada TTP distinto presente, con tope 2.5
`cisa_part`	0 – 1.0	Bonus fijo si la familia aparece en la lista CISA Top Malware Strains (AA22-216A)

Escala ordinal de tipo (`type_part`)

Tabla 5.5: Escala ordinal de tipo de amenaza (`type_part`).

Tipo	Valor
`ransomware` / `wiper`	3.5
`rat`	3.0
`banker` / `backdoor`	2.5
`trojan`	2.0
`pws` / `keylogger`	1.5
`coinminer`	1.0
`adware`	0.5
(desconocido)	valor por defecto bajo

Ponderación de TTPs por táctica (`ttp_part`)

Cada código `T####` se mapea a una de 13 tácticas MITRE ATT&CK mediante una tabla curada de más de 60 técnicas; las subtécnicas (p. ej. `T1547.001`) se normalizan a su código base (`T1547`) antes de la búsqueda. Códigos no reconocidos reciben el peso mínimo (0.1) en lugar de ser ignorados, para no penalizar la cobertura del mapeo.

Tabla 5.6: Pesos por táctica MITRE ATT&CK (`ttp_part`).

Táctica	Peso
Impact	1.0
Exfiltration	0.9
Lateral Movement	0.9
Credential Access	0.8
Persistence	0.7
Privilege Escalation	0.7
Command & Control	0.6
Collection	0.5
Initial Access	0.5
Defense Evasion	0.4
Execution	0.3
Discovery	0.2
Reconnaissance	0.1
(no reconocido)	0.1

Lista CISA (`cisa_part`)

Lista de notoriedad propia del sistema, construida a partir de advisories CISA y ampliada con familias de alta relevancia operativa no incluidas en el advisory AA22-216A original (Emotet, njRAT, gh0st, asyncrat, raccoon). De las 10 familias presentes en Avast-CTU, **Emotet, Qakbot, Trickbot, Lokibot, Ursnif y njRAT** reciben el bonus completo (+1.0); **Zeus, HarHar, Swisyn y Adload** no están en la lista y reciben 0.0. La comparación case-insensitive evita falsos negativos por capitalización del nombre de familia.

Garantía anti-leakage

La firma de `compute_severity` **no acepta `malscore`, `cape_score` ni `score`**, y `SeverityResult` no expone ningún atributo equivalente. Dos tests introspectivos (`TestNoLeakage`, `tests/test_severity_engine.py`) verifican ambas propiedades por inspección de la firma y de los atributos del dataclass, no solo por comportamiento.

`severity_from_row(row)` aplica el motor a una fila del dataset ingerido en el componente 1 (donde los TTPs viajan como CSV en `ttp_codes_str`), y es la función que usan los scripts de comparación y de construcción de la cola priorizada (§5.6).

Los resultados de aplicar este motor sobre las 48.976 muestras de desarrollo de Avast-CTU, incluida la comparación de saturación y de separación inter-familia (η^2) frente al `malscore`, se presentan en el capítulo 7.

Los topes de los cuatro componentes (3,5 / 3,0 / 2,5 / 1,0) y los pesos por táctica son una **política razonada, no el resultado de una calibración empírica**: codifican un orden de prioridades SOC defendible (impacto > exfiltración > persistencia > descubrimiento; tipo de malware como señal dominante; notoriedad como desempate acotado al 10 % de la escala) tomado de las fuentes documentadas en §5.4.x. Para acotar el riesgo de esta decisión, un análisis de sensibilidad perturbó cada constante

±25 % y recomputó la cola completa (n=48.976): el orden resultante es robusto (Spearman ≥ 0,9887 en todo el barrido; retención del top-364 por rango ≥ 85,44 %), de modo que las conclusiones no dependen del ajuste fino de estos valores. El conteo exacto de casos por bucket sí varía con la magnitud (es un parámetro de presentación, §7.5).

5.5 Componente 3: anomalía conductual y corrección de falsos positivos

Este bloque cubre los dos motores complementarios del componente 3 (anomalía y corrección de FP): detectar comportamiento atípico que la familia/tipo no captura, y neutralizar posibles falsos positivos del sandbox sobre binarios benignos conocidos. Ambos operan **exclusivamente sobre features de comportamiento, red y estáticas**, sin acceso a `family`, `type`, `malscore` ni a las señales del motor de severidad: son, deliberadamente, "el otro lado del cerebro".

5.5.1 Motor de anomalía (`anomaly_engine.py`)

`AnomalyEngine` encapsula un `IsolationForest` de scikit-learn entrenado sobre 14 features explícitas, listadas en la constante `FEATURE_NAMES`:

```
file_ops, registry_ops, mutex_ops, resolved_apis_count, executed_commands_count,
process_count, tcp_count, udp_count, http_count, dns_count, hosts_count,
domains_count, pe_imports_count, pe_sections_count
```

Estas 14 features son un subconjunto deliberado de `CapeFeatures`: excluyen explícitamente `signature_*`, `ttps`, `family`, `type` y `malscore`, para que la señal de anomalía sea **complementaria** a la del motor de severidad y no una redundancia del mismo.

Hiperparámetros de entrenamiento (`scripts/train_anomaly_model.py`): `contamination=0.10`, `n_estimators=200`, `random_state=42`. El modelo, junto con el `StandardScaler` ajustado y los metadatos del entrenamiento, se persiste en `data/models/anomaly_v2.joblib` (~30 KB), siguiendo el mismo patrón de persistencia que los modelos del prototipo v1.

`anomaly_score(features) -> float` normaliza la salida (no acotada) de `decision_function` mediante una sigmoide:

```
anomaly_score = sigmoid(k × (−decision_function(x)))      (k = 5)
```

de forma que valores cercanos a 1 indican comportamiento atípico y valores cercanos a 0 indican comportamiento típico dentro de la población de entrenamiento. `score_batch()` aplica la misma transformación a un lote completo.

Garantía anti-leakage: `FEATURE_NAMES` no incluye `malscore`, `family` ni `type`; `features_to_vector()` ignora cualquier clave adicional presente en el diccionario de entrada (tolerante a un `CapeFeatures` completo); y la firma de `AnomalyEngine.fit` no acepta `y`, `family` ni `target`, de modo que el ajuste es no supervisado por construcción. Tres tests introspectivos lo verifican.

5.5.2 Corrector de falsos positivos (`fp_correction.py`)

```
is_likely_benign(features) -> FPCheckResult
```

`FPCheckResult` acumula un `benign_score` a partir de hasta cuatro fuentes de evidencia, cada una con un peso parcial:

- coincidencia de `imphash` con un registro de binarios benignos conocidos,
- coincidencia de `sha256`,

- coincidencia del nombre de una signature con una lista de firmas consideradas benignas (p. ej. firmas que solo indican empaquetado o comportamiento administrativo habitual),
- patrones funcionales arbitrarios registrados como funciones (`register_benign_pattern`).

Una muestra se marca como `is_benign=True` cuando `benign_score >= 0.7`. El módulo expone una API de extensión (`register_benign_imphash`, `register_benign_sha256`, `register_benign_signature_name`, `register_benign_pattern(label, fn)`, `reset_benign_lists()`, `list_active_rules()`) pensada para poblarse con el dataset de benignos Cuckoo (11.735 binarios benignos) en una sesión de trabajo futura.

Estado actual: las listas de `imphash`, `sha256` y nombres de firma están **vacías**; solo existe un patrón placeholder (`no_observable_activity_in_sandbox`) que aporta 0.6 puntos por sí solo, insuficiente para cruzar el umbral de 0.7, de modo que el corrector **no marca ninguna muestra como benigna mientras las listas estén vacías**. Sobre las 48.976 muestras de Avast-CTU (100 % malware, ningún benigno verdadero en el dataset de desarrollo), el corrector produce 0/48.976 falsos benignos. Esta observación es la **consecuencia directa de que el hook no puede disparar con listas vacías** (no de que el detector sea preciso): confirma la neutralidad del corrector sobre malware (no introduce ruido), pero no es una garantía de precisión del detector. El recall real solo podrá evaluarse cuando el dataset de benignos Cuckoo se integre.

5.6 Componente 4: motor de fusión de urgencia (`urgency_fusion.py`)

Este es el punto de fusión del pipeline v2, donde convergen las tres señales anteriores en un único número interpretable por el SOC.

```
compute_urgency(severity_v2, anomaly_score, fp_result, context=None) -> UrgencyResult
```

La fórmula es **multiplicativa**, no aditiva, para que cada modulador tenga una interpretación porcentual directa sobre la severidad base:

```
urgency_score = clip(
    severity_v2
    × anomaly_boost(anomaly_score)     ∈ [1.00, 1.20]
    × fp_penalty(fp_result)            ∈ [0.20, 1.00]
    × context_multiplier(context)      ∈ [0.80, 1.30]   (por defecto 1.00)
,  0, 10)
```

- **`anomaly_boost`**: fórmula **lineal** `1,0 + 0,20 × anomaly_score`, rango teórico [1,00, 1,20]. El `anomaly_score` $\in$ `[0,1]` es la salida de una sigmoide invertida (`1/(1+e^{raw×5})`, k=5) sobre el `decision_function` crudo de IsolationForest; mayor valor = más anómalo. En el dataset Avast-CTU las medias por familia oscilan entre 0,32 (Swisyn) y 0,48 (Qakbot), produciendo un rango de boost observado de ≈ [1,06, 1,10] para las medias de familia; los casos individuales más atípicos alcanzan ≈ 1,16. Sin modelo entrenado, el servicio usa `anomaly_score = 0,5` como score neutro (boost = +10 %). La anomalía **no domina** la severidad: una muestra de severidad baja no se convierte en crítica solo por ser anómala, pero sí asciende dentro de su rango.
- **`fp_penalty`**: una muestra fuertemente identificada como benigna conocida puede reducir la urgencia hasta un 80 % (suelo de 0.20, nunca llega a 0). El suelo es deliberado: incluso ante evidencia fuerte de benignidad, el sistema conserva trazabilidad ("potencialmente benigno, urgencia residual 0.4 por comportamiento") y deja la decisión final al analista.
- **`context_multiplier`**: gancho documentado y testeado para criticidad de host (`low` 0.90 / `medium` 1.00 / `high` 1.15 / `critical` 1.30) y privilegios del usuario (`standard` 0.95 / `admin` 1.10 / `root` 1.20), combinables con tope en 1.30. **No hay ninguna fuente de contexto real conectada** en el MVP (queda fuera de alcance): el motor se invoca con `context=None` y el multiplicador queda en 1.0.

Esto es exactamente el bloque de enriquecimiento contextual del diseño original citado en §5.1, materializado como interfaz preparada y no como implementación operativa.

Fórmula efectiva en el MVP offline. Con las listas del corrector de FP vacías (`benign_score = 0 → fp_penalty = 1,0`) y sin fuente de contexto conectada (`context=None → context_multiplier = 1,0`), los dos moduladores de producción están activos por contrato pero neutrales en valor. La fórmula efectiva del MVP es por tanto:

```
urgency_score ≈ severity_v2 × (1 + 0,20 × anomaly_score)
```

La anomalía aporta como máximo un 20 % adicional sobre la severidad base. `fp_penalty` y `context_multiplier` son **ganchos de producción** (implementados, testados y listos para activarse), pero no contribuyen a la urgencia calculada en este corte. El primero se activa cuando se integren muestras benignas reales (dataset de Ilić et al. [21], §5.5.2 y capítulo 8); el segundo, cuando se disponga de una fuente de contexto multi-señal (trabajo futuro, capítulo 8).

Buckets categóricos para la UI

El sistema combina una urgencia numérica interna con un bucket categórico para el analista:

Tabla 5.7: Buckets categóricos de urgencia para la interfaz.

Bucket	Rango de `urgency_score`
crítico	≥ 8.0
alto	6.0 – 8.0
medio	3.0 – 6.0
bajo	< 3.0

`UrgencyResult` incluye, además de `urgency_score` y `bucket`, el desglose de los cinco componentes (severidad base y los tres multiplicadores aplicados), las reglas de FP que hicieron match y notas de auditoría: toda la cadena de cálculo es trazable desde el número final hasta sus entradas.

Garantía anti-leakage

`compute_urgency` no acepta `malscore`, `cape_score` ni `score` como argumento, y `UrgencyResult` no expone ningún atributo equivalente (verificado por dos tests introspectivos, `TestNoLeakage`). El `malscore` original de CAPE **solo aparece como columna informativa al lado de la cola priorizada** (`scripts/build_priority_queue.py`), para que el analista pueda comparar "lo que decía CAPE" con "lo que dice el sistema", nunca como entrada de la fórmula.

Los resultados de aplicar la fusión completa sobre las 48.976 muestras de desarrollo (distribución por bucket, cola por familia y casos de divergencia frente al orden por `malscore`) se presentan en el capítulo 7.

5.7 Componente 5: síntesis SecOps (`secops_summary.py` / `ollama_client.py`)

Este componente traduce los datos técnicos (urgencia, desglose, payload del evento) en una **ficha narrativa accionable** para el analista: es la etapa de síntesis del pipeline (§5.1).

Interfaz dual

```
generate_summary(event_payload, severity, anomaly, urgency, mode="auto",
                 model="llama3.2:3b", ollama_client=None) -> SummaryResult
```

- **`mode="template"`**: generación determinista por plantilla, sin dependencias externas. Garantiza por construcción que la ficha menciona la familia, el valor de urgencia y el bucket.
- **`mode="llm"`**: invoca un modelo local vía Ollama (`ollama_client.py`, `localhost:11434`, `format=json` para forzar JSON válido, timeout por defecto 180 s). Si el LLM omite campos del contrato o emite formatos alternativos (p. ej. `iocs` como cadena separada por comas en lugar de lista), `generate_summary` normaliza la salida y **completa los campos faltantes con la plantilla**, de modo que la ficha entregada al analista nunca está incompleta, independientemente de la calidad del LLM.
- **`mode="auto"`** (por defecto): usa el LLM si `ollama_client.is_available()` responde; si no, degrada a plantilla sin error: *graceful degradation* ante un entorno sin Ollama disponible.

Contrato de salida

```
{
  "que_hizo": "...",
  "por_que_importa": "...",
  "iocs": ["..."],
  "accion_recomendada": "...",
  "narrativa": "...",
  "mode_used": "template | llm",
  "model": "llama3.2:3b | phi4-mini:latest | null",
  "latency_seconds": 0.0,
  "payload_fidelity": 0.0
}
```

Los cinco primeros campos son la ficha en español dirigida al analista; los cuatro últimos son metadatos de auditoría del propio componente.

Hallazgo de ingeniería: el system prompt defensivo

Durante la integración se verificó experimentalmente que, sin un encuadre explícito de la tarea, `llama3.2:3b` **rechaza la petición** ("no puedo ayudar con malware"), incluso cuando la tarea es puramente descriptiva sobre datos ya generados por el sandbox. La solución, `DEFENSIVE_SOC_SYSTEM_PROMPT`, encuadra la tarea como **análisis defensivo dentro de un SOC** y se aplica por defecto a cualquier llamada del cliente Ollama (`test_default_is_defensive_soc` lo verifica). Sin este encuadre, ambos modelos locales probados son inutilizables para esta tarea; con él, generan la ficha correctamente. Este hallazgo se documenta como una decisión de ingeniería citable, no como una curiosidad anecdótica.

Métrica de fidelidad: `payload_fidelity`

Cada ficha generada se evalúa automáticamente con cinco comprobaciones binarias:

1. menciona la familia de malware,
2. menciona el valor numérico de urgencia,
3. menciona el bucket categórico,
4. menciona al menos una técnica TTP,
5. incluye todos los campos requeridos del contrato.

`payload_fidelity` es la proporción de comprobaciones superadas (0.0–1.0). Es una métrica automática y por tanto reproducible en CI, pensada como complemento, no sustituto, de la evaluación humana

con la rúbrica MESSALA, que queda como trabajo pendiente: una evaluación dedicada sobre 30-50 fichas (capítulo 8).

Anti-leakage por contrato

`generate_summary` **no acepta** `malscore` como argumento. El prompt enviado al LLM incluye el `malscore` de CAPE únicamente como **referencia comparativa** ("CAPE asignó un malscore de X; el sistema calcula una urgencia de Y"), pero la urgencia que se le pide describir y justificar al modelo es la calculada por el motor de fusión (severidad + anomalía + corrección de FP), no una re-derivación del `malscore`. Un test introspectivo verifica la firma de la función.

Los resultados de la comparativa real entre plantilla, `phi4-mini:latest` y `llama3.2:3b` (latencia, fidelidad, observaciones cualitativas) se presentan en el capítulo 7.

Matiz sobre la etiqueta de familia mostrada. La síntesis usa como etiqueta de familia, *solo para el texto que se muestra*, `event_payload["family"]` y, si falta, el campo `detections` del propio informe CAPE como respaldo de display. Esto tiene una consecuencia que conviene declarar: la mención de notoriedad de la ficha narrativa se evalúa sobre esa etiqueta mostrada, mientras que el **bonus de notoriedad del motor de severidad** (§5.4) exige una familia verificada de fuente externa. Cuando un caso carece de familia externa y la ficha recurre a `detections`, la mención de notoriedad del texto puede no corresponderse con el bonus realmente aplicado por el motor. En el experimento Avast-CTU no se produce (todas las muestras traen familia de `public_labels.csv`), pero en producción con casos sin etiquetar es un supuesto real; se declara como distinción display-vs-motor y su alineación queda como trabajo de cableado (§8.5).

5.8 Componente 6: API REST y UI Atalaya

Con el cierre de la **Fase F del pivote v2** (2026-06-11), los componentes 1 a 5 se integran en un orquestador único y se exponen mediante una **API y una UI v2 nuevas**, que coexisten con el dashboard v1 sin modificarlo: decisión deliberada para preservar los Gates 4-5 (cerrados, capítulo 7.9) intactos.

Orquestador (`triage_service.py`)

```
triage_from_payload(event_payload, summary_mode="template") -> TriageResult
triage_from_cape_report(report, family=None, type_=None, summary_mode="template") -> TriageResult
```

Encadena los cinco motores existentes **sin modificarlos**: `cape_parser_v2 → severity_engine → (anomaly_engine + fp_correction) → urgency_fusion → secops_summary`. El motor de anomalía se instancia como *singleton* perezoso (`_get_anomaly_engine()`, con caché) y degrada a un `anomaly_score` neutro (0,5) si no hay modelo entrenado disponible, evitando que la ausencia de `anomaly_v2.joblib` rompa el pipeline completo.

Decisión de diseño (por contrato): `DEFAULT_SUMMARY_MODE = "template"`. La plantilla determinista (`payload_fidelity` = 0,60, capítulo 7.6) es el motor de síntesis por defecto tanto en el orquestador como en el endpoint; el modo LLM (`summary_mode="llm"`) queda disponible como parámetro explícito para experimentación, sin activarse por defecto.

Anti-leakage de extremo a extremo: la firma de `triage_from_payload` no acepta `malscore` como argumento. Si el payload de entrada contiene un campo `malscore`, este solo se traslada al resultado como `malscore_cape` (un campo de visualización) y un test funcional verifica explícitamente que **modificar el `malscore` del payload no altera la `urgency_score` calculada**.

API v2 (`routes_triage.py`)

Tabla 5.8: Endpoints de la API v2.

Endpoint	Descripción
`POST /api/v2/triage`	Body `{event_payload, mode?, model?}` → `TriageResult` completo (severidad, anomalía, urgencia, bucket, ficha SecOps)
`POST /api/v2/triage/cape_report`	Body `{report, family?, type?, mode?}`; entrada desde un `report.json` crudo de CAPE
`GET /api/v2/triage/queue?top=N`	Cola priorizada (top-N) leída del último `priority_queue_*.csv` de la Fase D
`GET /api/v2/triage/queue/{rank}`	Fila individual de la cola (agregados por componente; el desglose completo se obtiene vía `POST /api/v2/triage`)

Estos endpoints se registran en `app/main.py` **junto a**, no en sustitución de, los endpoints v1 (`/api/v1/score`, `/api/v1/incidents`, `/api/v1/status/model`, `/health`), cuyo contrato permanece intacto y está protegido por un test de regresión dedicado (`test_routes_triage.py`).

UI Atalaya (`/ui/triage`, `/ui/triage/{rank}`)

- **`GET /ui/triage`** (cola): cuatro tarjetas-resumen por bucket (crítico/alto/medio/ bajo) con código de color, tabla ordenada por `urgency_score` descendente con la columna `malscore` de CAPE al lado para comparación directa, y una columna `Shift` (verde si la muestra fue promovida respecto al orden por `malscore`, rojo si fue rebajada; capítulo 7.5). Si no existe ningún `priority_queue_*.csv`, muestra un *empty state* con la instrucción para generarlo (`build_priority_queue.py`). Ocupando la mitad superior de la página, un panel **«Divergencia real frente a CAPE»** materializa la tesis del sistema de forma bidireccional: dos tablas contrapuestas, *Promovidos* (casos que CAPE infra-puntuaba) y *Rebajados* (casos que sobre-puntuaba), con el `malscore` de CAPE junto al `Shift` de cada fila. El panel se restringe deliberadamente a muestras con `malscore < 10`, es decir, **fuera del bloque saturado**, para que la comparación de posiciones sea 1:1 y legítima: dentro del empate en 10,0 no hay orden que comparar. Es la vista que hace visible, de un vistazo, que la re-valoración no solo sube unos casos, sino que reordena en ambos sentidos.
- **`GET /ui/triage/{rank}`** (ficha): cuatro tarjetas (urgencia, severidad, anomalía, `malscore` original de CAPE), el **desglose explicable** de la fórmula de fusión (`base × anomaly_boost × fp_penalty × context_multiplier = urgency`, visible al analista), una tabla de aporte por componente de severidad (tipo / signatures / TTPs / CISA), *badges* con las tácticas MITRE detectadas, el resumen SecOps en plantilla (los cuatro campos del contrato + narrativa) y un detalle plegable con el `payload_fidelity` de la ficha.
- Ambas plantillas reutilizan `base.html`, cuya navegación enlaza la cola de triaje (`/ui/triage`, también destino de la raíz `/` vía redirección), la página de estado del modelo v1 (`/status/model`, marcada como legacy) y la documentación de la API (`/docs`); el dashboard v1 queda accesible en `/ui/legacy` con aviso de legacy, fuera de la navegación. Extienden `style.css` con clases nuevas, sin modificar las existentes.

Estado de la integración

Las seis fases del pivote v2 (A-F) están **cerradas**: 225 tests al cierre del pivote (199 de las Fases A-E + 26 de la Fase F: 15 de `test_triage_service.py` + 11 de `test_routes_triage.py`), `demo.ps1` 21/21 PASS sin regresión sobre el dashboard v1. El detalle de implementación (estructura de `TriageResult`, plantillas Jinja2, capturas de la cola y la ficha) se documenta en el capítulo 6.

El resto de la infraestructura de la capa de presentación (anonimización obligatoria en ingesta, validación de calidad con `quality.py`, persistencia en SQLite con `storage.py`, `GET /api/v1/status/model` y `GET /health`) se mantiene sin cambios respecto al prototipo v1 y sigue siendo válida para v2.

5.9 Integración con CAPEv2

La integración con la API REST de CAPEv2 (puerto 8000 del servidor de sandbox) no se ve afectada por el pivote: el flujo de envío de muestra, *polling* de estado y obtención del informe completo es el mismo que en v1, y es precisamente la **fuente de los `report.json`** que consume el parser (componente 1, §5.3) cuando se trabaja con informes de CAPE 2.5 reales en lugar de con el dataset Avast-CTU.

```
Archivo sospechoso
      |
      ▼
POST /apiv2/tasks/create/file/          ← submit de la muestra
      |
      ▼  {"error": false, "data": {"task_ids": [N], "message": "..."}}
      |
      ▼ (polling cada T segundos)
GET /apiv2/tasks/view/{id}/             ← estado: pending → running → completed → reported
      |
      ▼ (cuando status == "reported")
GET /apiv2/tasks/get/report/{id}/       ← informe completo
      |
      ▼
cape_parser_v2.parse_cape_report_v2()   → CapeFeatures
```

Como se documentó en la Fase A (§5.3), el parser v2 ya está preparado para los informes CAPE 2.5 que produciría esta integración: fue validado contra 11 informes reales de CAPE 2.5 del paquete `_tmp/CAPE_GOLD_EXPORT_20260417/`. Pero la **réplica 1:1 del entorno CAPE (Gate 3) sigue pendiente por una limitación de hardware (ver §8.5.1)**. Por diseño, **v2 no depende de Gate 3 para su desarrollo y evaluación**, que se realizan íntegramente offline con Avast-CTU; Gate 3 queda como validación final con muestras propias (EICAR + binarios de MalwareBazaar) si se cierra a tiempo, y se documenta como trabajo pendiente en caso contrario (capítulo 8).

5.10 Decisiones de diseño relevantes

El sistema que describe este capítulo no es un clasificador de aprendizaje automático: es un **motor de decisión determinista y auditable** que codifica política SOC explícita (tipología de amenaza, notoriedad según lista propia basada en advisories CISA y técnicas MITRE ATT&CK) en una función con componentes acotados y trazables desde el número final hasta sus entradas. El aprendizaje automático tiene un rol **auxiliar y exploratorio**: el `IsolationForest` detecta comportamiento atípico que la política explícita no captura (§5.5.1) y el LLM local genera la síntesis narrativa en lenguaje natural (§5.7); ninguno de los dos determina la urgencia. La contribución de diseño que articula el trabajo es la **trazabilidad completa** dato → severidad → anomalía → urgencia → ficha, verificada por contrato en cada módulo (`TestNoLeakage`). Las decisiones que siguen se ordenan a partir de este principio.

Motores deterministas y auditables antes que modelos entrenados, donde la explicabilidad lo exige. El motor de severidad (§5.4) es una función explícita con componentes acotados, no un modelo

de caja negra: cada punto de la escala 0-10 se puede desglosar y justificar ante el evaluador, que es exactamente lo que necesita un analista de SOC para confiar en una re-priorización. El aprendizaje automático se reserva para la señal complementaria de anomalía (§5.5.1), donde no se requiere justificar componente a componente sino detectar lo atípico.

Separación estricta entre señales independientes y señal de referencia. Los componentes 2 a 4 forman, en conjunto, una "caja" que nunca recibe `malscore` como entrada (§5.1, §5.4, §5.5, §5.6) y cuyo resultado nunca lo expone como atributo. El `malscore` reaparece exclusivamente como dato comparativo en la cola (§5.6) y en el prompt de síntesis (§5.7). Esta separación, verificada por tests introspectivos en cada módulo, es la corrección estructural del leakage circular de v1: no un parche posterior, sino una propiedad de diseño exigible por contrato. Conviene matizar: `sig_part` y `ttp_part` **re-ponderan las mismas señales** que alimentan el `malscore` de CAPE (signatures y TTPs del informe CAPE), con diferente función de agregación y anti-saturación; la independencia es estricta respecto al **veredicto** (`malscore`), no respecto al report subyacente (η^2(conductual) = 0,477 ≈ η^2(malscore) = 0,510 lo confirma a escala de 48.976 muestras).

Fusión multiplicativa con topes interpretables. La fórmula de urgencia (§5.6) se eligió multiplicativa porque cada modulador admite una lectura porcentual directa ("esta muestra es un 15 % más urgente por su comportamiento anómalo"), frente a una fórmula aditiva donde "+1 punto sobre 10" no tiene una interpretación tan inmediata. Los topes (`anomaly_boost` ≤ 1.20, `fp_penalty` ≥ 0.20, `context_multiplier` $\in$ [0.80, 1.30]) garantizan que ninguna señal secundaria pueda invertir por sí sola el orden que establece la severidad base.

Interfaz dual (LLM/plantilla) desde el primer día. La disponibilidad de un servicio Ollama local no está garantizada en cualquier máquina o turno de un SOC. El template determinista no es solo un *fallback*: actúa también como *ground truth* contra la que se compara la fidelidad de los modelos LLM (§5.7, capítulo 7). El diseño dual es, en sí mismo, una decisión defendible de ingeniería de sistemas (*graceful degradation*).

Offline-first se mantiene. Como en v1, ningún componente del pipeline v2 depende de servicios externos en tiempo de ejecución: Avast-CTU se procesa localmente, los modelos (`anomaly_v2.joblib`) se entrenan y persisten en disco, y el LLM, cuando está disponible, es local (Ollama). La demo de defensa puede ejecutarse sin conectividad.

Estado de los componentes con datos pendientes. El corrector de falsos positivos (§5.5.2) se documenta con sus listas vacías y su dependencia declarada del dataset de benignos de Ilić et al. [21], en lugar de simularse con datos de relleno. La estructura está lista y testeada (0 falsos benignos sobre 48.976 muestras malware, por construcción); su evaluación de recall queda pendiente de la integración de benignos reales, declarada como trabajo futuro en el capítulo 8.

SQLite, FastAPI y Jinja2 se conservan de v1 por las mismas razones que en v1: coste de administración nulo, portabilidad, y migración trivial a un motor con servidor si el alcance creciera más allá del MVP académico.

Capítulo 6. Implementación

6.1 Estructura del repositorio y stack

El prototipo (`01_TFG_COMPLETO/prototipo/`) es una aplicación FastAPI 0.115 con modelos Pydantic 2.10, scikit-learn 1.6 para los componentes de aprendizaje automático, joblib para la persistencia de modelos, MLflow 2.20 para el registro de entrenamientos del clasificador v1, Jinja2 para la UI server-side y pytest 8.3 como framework de pruebas. Versión de Python: 3.11. El cliente LLM (`ollama_client.py`) usa `requests` contra una instancia local de Ollama (`localhost:11434`).

Estructura relevante para el pivote v2:

```
prototipo/
├── app/
│   ├── main.py                    # registra los routers v1 y v2
│   ├── config.py                  # settings (rutas, anonymization_salt)
│   ├── services/
│   │   ├── cape_parser_v2.py      # [1] parser multi-schema CAPE 2.2/2.5
│   │   ├── severity_engine.py     # [2] severidad intrínseca determinista
│   │   ├── anomaly_engine.py      # [3] IsolationForest, 14 features
│   │   ├── fp_correction.py       # [3'] corrección de falsos positivos
│   │   ├── urgency_fusion.py      # [4] fusión multiplicativa -> urgencia
│   │   ├── ollama_client.py       # cliente Ollama (system prompt defensivo)
│   │   ├── secops_summary.py      # [5] síntesis dual template/LLM
│   │   ├── triage_service.py      # [6] orquestador A-E
│   │   ├── anonymizer.py          # SHA256 con salt, reutilizado por [1]
│   │   └── (v1, sin tocar): scoring.py, prioritization.py,
│   │       feature_engineering.py, incident_service.py, quality.py, storage.py
│   ├── api/
│   │   ├── routes_triage.py       # rutas /api/v2/triage*
│   │   ├── routes_ui.py           # + /ui/triage, /ui/triage/{rank}
│   │   └── (v1, sin tocar): routes_events.py, routes_score.py,
│   │       routes_incidents.py, routes_status.py
│   └── templates/
│       ├── triage_queue.html      # cola priorizada v2
│       ├── triage_detail.html     # ficha explicable v2
│       └── base.html              # + enlace "Cola de triaje" en la nav
├── scripts/
│   ├── ingest_avast_ctu.py        # Fase A: ingesta Avast-CTU -> dataset
│   ├── compare_severity.py        # Fase B: severity_v2 vs malscore
│   ├── train_anomaly_model.py     # Fase C: entrena anomaly_v2.joblib
```

```
|   ├── evaluate_anomaly_fp.py       # Fase C: evaluación anomalía + FP
|   ├── build_priority_queue.py      # Fase D: genera priority_queue_*.csv
|   ├── run_secops_summary.py        # Fase E: comparativa template vs LLM
|   └── build_cap07_evaluation.py    # Fase F: consolida todo para cap07
└── tests/
    ├── test_cape_parser_v2.py       # 26 tests
    ├── test_severity_engine.py      # 34 tests
    ├── test_anomaly_engine.py       # 14 tests
    ├── test_fp_correction.py        # 15 tests
    ├── test_urgency_fusion.py       # 26 tests
    ├── test_secops_summary.py       # 16 tests
    ├── test_ollama_client.py        # 15 tests
    ├── test_triage_service.py       # 15 tests
    ├── test_routes_triage.py        # 11 tests
    └── (v1 + transversales, sin tocar): 53 tests preexistentes
```

Nota sobre v1. El clasificador supervisado original (`risk_score = 0,55·sup + 0,25·anom + 0,20·ctx`, descrito en versiones anteriores de este capítulo) sigue presente en el repositorio y sigue sirviendo los endpoints `/api/v1/*` y el dashboard `/`; ningún test de los 53 preexistentes se ha tocado. El pivote v2 (`/api/v2/triage*`, `/ui/triage`) es un pipeline **independiente y paralelo**: no reutiliza `scoring.py` ni `prioritization.py`, y ninguno de sus componentes acepta `malscore`/`cape_score`/`score` como entrada (ver §6.7). v1 se mantiene como contexto histórico del proyecto: el origen del pivote fue precisamente la constatación de que `cape_score` (malscore de CAPE) actuaba como leakage en v1 (decisión de pivote del 31 de mayo de 2026).

6.2 El pipeline de 6 componentes

El pivote v2 se organiza en 6 componentes con contratos de entrada/salida explícitos y una regla transversal: **ningún componente del pipeline v2 acepta `malscore`, `cape_score` ni `score` como argumento de entrada**. El malscore de CAPE solo reaparece al final, como campo `malscore_cape`, para que el analista compare "lo que decía CAPE" contra la urgencia recalculada.

6.2.1 `cape_parser_v2.py`: parser multi-schema (463 líneas)

Convierte un `report.json` de CAPE (esquema 2.2 o 2.5) en un dataclass `CapeFeatures` con ~35 campos, sin derivar ninguna etiqueta de malicia.

Extractores defensivos por sección del informe:

- `extract_ttps(ttps_raw)`: soporta las tres variantes observadas en los datasets reales: `ttps` como dict `{T1106: "descripción"}` (Avast-CTU 2022), lista con clave singular `ttp` (otra variante de CAPE 2.2), y lista con clave plural `ttps` (CAPE 2.5).
- `extract_signatures(sigs_raw)`: cuenta, severidad máxima, suma de pesos, severidad media, número de firmas de severidad alta, nombres (máx. 50), categorías, familias y códigos MBC.
- `extract_network(net_raw)`: conteos de TCP/UDP/HTTP/DNS/ICMP/SMTP/IRC, número de hosts y dominios, IPs y dominios únicos (máx. 50 cada uno, soporta hosts/domains como lista de strings o de dicts).
- `extract_behavior(beh_raw)`: `file_ops` y `registry_ops` como suma de las listas `files`/`read_files`/`write_files`/`delete_files` (y equivalentes de registro), más `mutex_ops`, `resolved_apis_count`, `executed_commands_count`, conteo de servicios creados/iniciados y `process_count`.

- `extract_static(static_raw)`: `pe_imphash`, número de DLLs importadas, número total de imports, número de secciones PE y timestamp.
- `extract_info(info_raw)`: `task_id`, `cape_version`, `started`, `ended`, `duration`, `category`, `machine_name`, `package`.

La función pública `parse_cape_report_v2(report, anonymize=True)`:

- Lanza `ValueError` si `report` no es un dict.
- Normaliza `malscore`: si viene en escala 0-100 (`> 10.0`) lo divide por 10 y lo limita a 10; los negativos se recortan a 0. El resultado se redondea a 3 decimales y se almacena en `CapeFeatures.malscore` **solo como dato de referencia**; ningún componente posterior lo lee.
- Obtiene `sha256` de `target.file.sha256`, con fallback a `target.file.md5` si el primero no es un string.
- Si `anonymize=True` (por defecto): anonimiza `machine_name` y las listas `unique_ips`/`unique_domains` reutilizando `_anon()` de `app/services/anonymizer.py` (SHA256 de `f"{salt}:{valor}"`, prefijo `anon:`, primeros 16 caracteres del digest). Valores ya anonimizados (`anon:...`) no se vuelven a hashear.
- `CapeFeatures.detections` y `malfamily_tag` se extraen del informe **tal cual** (sin derivar ninguna heurística de "malicioso"/"sospechoso" como hacía el `cape_parser.py` de v1 a partir de `info.score`).

Cobertura de tests (`tests/test_cape_parser_v2.py`, 26 casos en 9 clases): `TestExtractTTPs` (5, las tres variantes de esquema + casos vacío/garbage), `TestExtractSignatures` (3), `TestExtractNetwork` (3), `TestExtractBehavior` (2), `TestExtractStatic` (2), `TestAvastCTUReal` (4, parsea 3 informes reales del dataset Avast-CTU, incluyendo `test_avast_no_label_derivation` que verifica que no se deriva ninguna etiqueta), `TestVMCape25Real` (4, parsea **3 informes reales capturados de la VM CAPE 2.5**, fixtures previas al parón de la máquina, ver §6.6), y `TestParserDoesNotCrashOnGarbage` (3, dict vacío, tipo incorrecto, todos los campos con tipos erróneos).

6.2.2 `severity_engine.py`: severidad intrínseca (378 líneas)

Calcula `severity_v2` (0-10) a partir de cuatro componentes deterministas, **sin usar malscore**. Es el reemplazo directo del `cape_score` como medida de severidad: la motivación es que el malscore de CAPE satura (56,28 % de los 48.976 informes de Avast-CTU tienen `malscore = 10.0`, ver cap07), mientras que `severity_v2` distribuye la severidad según la familia, las firmas, las tácticas MITRE y la presencia en listas de amenazas prioritarias.

```
severity_v2 = type_part + sig_part + ttp_part + cisa_part   (capado a [0, 10])
              (0-3.5)      (0-3.0)    (0-2.5)    (0-1.0)
```

`type_part` (0-3.5, `TYPE_MAX`): `TYPE_SEVERITY` asigna un valor por tipo de malware: `ransomware`/`wiper` = 3.5, `rat` = 3.0, `banker`/`backdoor` = 2.5, `trojan` = 2.0, `pws`/`keylogger`/`infostealer` = 1.5, `coinminer`/`cryptominer`/`spyware` = 1.0, `adware` = 0.5, y `unknown`/`""` = 0.5 (mínimo, no cero: un tipo desconocido no es "inocuo"). Comparación insensible a mayúsculas.

`sig_part` (0-3.0, `SIG_MAX`): suma de tres sub-componentes a partir de los agregados de `cape_parser_v2`: - `severity_part` (0-1.5) = `sig_max_severity / 3 * 1.5`. - `weight_part` (0-1.0) = `min(sig_sum_weight / 200, 1.0)`. - `high_part` (0-0.5) = `min(sig_high_severity_count, 5) / 5 * 0.5`.

`ttp_part` (0-2.5, `TTP_MAX`): para cada TTP MITRE ATT&CK distinto detectado, `TTP_TO_TACTIC` lo mapea a su táctica (manejando subtécnicas: `T1055.001` se normaliza a la base `T1055`), y `TACTIC_WEIGHT` asigna un peso por táctica: `impact` = 1.0, `exfiltration` = 0.9, `lateral_movement` = 0.9, `credential_access` = 0.8, `persistence` = 0.7, `privilege_escalation` = 0.7, `command_and_control` = 0.6, `collection` = 0.5, `initial_access` = 0.5, `defense_evasion` = 0.4, `execution` = 0.3, `discovery` = 0.2, `resource_development` = 0.2, `reconnaissance` = 0.1, y `_unknown` = 0.1 para códigos no mapeados. Cada TTP distinto aporta `peso(táctica) * 0.25`, sumando hasta el tope de 2.5 (equivalente a 10 tácticas de `impact` o combinaciones mixtas). `TTP_TO_TACTIC` cubre más de 60 códigos T#### documentados con comentario de qué técnica representan.

`cisa_part` (0 o 1.0, `CISA_MAX`): `CISA_TOP_MALWARE` es la **lista de notoriedad propia del sistema**: 13 familias en minúsculas construida sobre advisories CISA y ampliada con familias de alta relevancia operativa

no cubiertas por el advisory AA22-216A original: `agenttesla`, `azorult`, `formbook`, `lokibot`, `ursnif`, `remcos`, `trickbot`, `qakbot`, `emotet`, `njrat`, `gh0st`, `asyncrat`, `raccoon`. Si la familia de la muestra (comparación insensible a mayúsculas) está en esta lista, suma 1.0; si no, suma 0.0. De las 10 familias del subconjunto Avast-CTU usado en Fases A-F, seis activan el bono (`Emotet`, `Qakbot`, `Trickbot`, `Lokibot`, `Ursnif`, `njRAT`) y cuatro no (`Zeus`, `HarHar`, `Swisyn`, `Adload`).

La función pública `compute_severity(family=None, type_=None, sig_max_severity=0, sig_sum_weight=0, sig_high_severity_count=0, ttp_codes=None)` devuelve un `SeverityResult` (score total redondeado a 4 decimales, capado en `[0, 10]`, más los cuatro sub-componentes, las tácticas detectadas y una propiedad `breakdown` que suma exactamente al total, verificado por `test_breakdown_sums_to_total`). `severity_from_row(row)` es un helper que parsea `ttp_codes_str` (CSV) desde una fila de dataset.

Cobertura de tests (`tests/test_severity_engine.py`, 34 casos en 9 clases): `TestTypeComponent` (4: orden descendente, RAT alto, tipo desconocido mínimo, insensibilidad a mayúsculas), `TestSignaturesComponent` (5: monotonía en severidad y peso, cap en 3.0, bono por conteo alto), `TestTTPsComponent` (7: tácticas de impact > discovery, normalización de subtécnicas, deduplicación, código desconocido mínimo, cap en 2.5), `TestCISAComponent` (3), `TestGlobalLimits` (4: rango [0,10], cap cuando los inputs exceden límites, suelo en 0, breakdown suma al total), `TestNoLeakage` (2: `compute_severity` no acepta `malscore` por firma; `SeverityResult` no expone `malscore`), `TestSeverityFromRow` (3), `TestSeverityOnVMRealReports` (2, **11 informes reales de la VM CAPE 2.5** puntuados dentro de rango) y `TestConstantsSane` (4, valida que las tablas de constantes están bien formadas).

En la comparación empírica (Fase B, n=48.976 Avast-CTU), `severity_v2` explica $\eta^2 = 0{,}833$ de la varianza de severidad por familia frente a $\eta^2 = 0{,}510$ del malscore original (+32,3 puntos porcentuales); ver cap07.

6.2.3 `anomaly_engine.py`: detección de anomalías conductuales (251 líneas)

Modelo no supervisado (`IsolationForest` + `StandardScaler`) sobre 14 features puramente conductuales/de red/estáticas, definidas en `FEATURE_NAMES` con orden fijo (el orden importa para el modelo entrenado):

```
Behavior (6): file_ops, registry_ops, mutex_ops, resolved_apis_count,
              executed_commands_count, process_count
Network  (6): tcp_count, udp_count, http_count, dns_count,
              hosts_count, domains_count
Static   (2): pe_imports_count, pe_sections_count
```

Deliberadamente **no incluye** `malscore`, familia, tipo ni signatures (las signatures las cubre `severity_engine`). `features_to_vector(features)` extrae el vector de 14 posiciones, sustituyendo por `0.0` cualquier valor ausente o no convertible a float, e **ignora cualquier clave extra** que se le pase (incluido `malscore` si estuviera en el dict de entrada).

`AnomalyEngine` (dataclass con `contamination=0.10`, `n_estimators=200`, `random_state=42`) expone: - `fit(feature_rows, metadata=None)`: entrena `IsolationForest(n_estimators, contamination, random_state, n_jobs=-1)` sobre los datos escalados con `StandardScaler`. Solo lee las 14 keys de `FEATURE_NAMES`. - `save(path=None)` / `load(path=None)`: persisten/cargan `{model, scaler, trained_on, feature_names}` en `data/models/anomaly_v2.joblib` vía `joblib`. `load()` lanza `FileNotFoundError` con instrucciones (`scripts/train_anomaly_model.py`) si el fichero no existe. - `score(features) -> AnomalyResult`: `raw = decision_function(x_scaled)` (más alto = más normal); `anomaly_score = 1 / (1 + exp(raw * 5.0))` mapea a `[0, 1]` donde más alto = más anómalo; `is_anomaly = (predict(x) == -1)`. - `score_batch(feature_rows)`: versión vectorizada para evaluación masiva.

Cobertura de tests (`tests/test_anomaly_engine.py`, 14 casos en 5 clases): `TestFeaturesToVector` (4: forma correcta, orden preservado, features ausentes -> 0, valores inválidos -> 0), `TestNoLeakage` (3: `features_to_vector` ignora `malscore` si está presente, `FEATURE_NAMES` no contiene ninguna etiqueta ni score, la

firma de `fit` no acepta labels), `TestFitSaveLoadRoundtrip` (4, con `tmp_path`), `TestBatchScoring` (2) y `TestAnomalyOnVMReports` (1, **los informes reales de la VM CAPE 2.5** puntúan dentro de `[0, 1]`).

En la evaluación Fase C (n=48.976), Qakbot tiene el 40,4 % de sus muestras marcadas como anómalas (`is_anomaly=True`), la familia con mayor proporción del subconjunto, y no se registran falsos benignos (0/48.976 con las reglas por defecto, ver §6.2.4).

6.2.4 `fp_correction.py`: corrección de falsos positivos (208 líneas)

Detecta patrones de binarios **benignos** que CAPE podría sobre-puntuar, para rebajar la urgencia hacia 0 cuando corresponda. Estado actual (Fase C): las listas base están **vacías por diseño**: `_KNOWN_BENIGN_IMPHASHES`, `_KNOWN_BENIGN_SHA256` y `_BENIGN_SIGNATURE_NAMES` son `set()` vacíos para no marcar nada como benigno sin evidencia empírica. La integración con el dataset de benignos de Ilić et al. [21] (11.735 binarios benignos) queda como trabajo futuro explícito.

API de extensión (hooks documentados, cubiertos por tests): - `register_benign_imphash(imphash)`, `register_benign_sha256(sha256)`, `register_benign_signature_name(name)`. - `register_benign_pattern(label, predicate)`: registra una función `dict -> bool` sobre las features; excepciones dentro del predicado se capturan y se ignoran (`test_pattern_exception_swallowed`). - `reset_benign_lists()`: limpia todas las listas (uso en tests).

`is_likely_benign(features)` evalúa, en orden: coincidencia exacta de `pe_imphash` (score 0.95), coincidencia exacta de `sha256` (score 1.0), proporción de `sig_names` presentes en la lista de nombres benignos (score entre 0.4 y 0.9 según el ratio), y patrones registrados (score 0.6 si disparan). El umbral `benign_score >= 0.7` determina `is_likely_benign=True`. Devuelve `FPCheckResult(is_likely_benign, benign_score, matched_rules, notes)`.

Patrón inicial registrado (`_initialize_default_patterns`, `no_observable_activity_in_sandbox`): si `sig_count == 0`, `ttp_count == 0`, `tcp_count == 0` y `file_ops == 0` simultáneamente, el predicado dispara, pero por sí solo (score 0.6) no alcanza el umbral 0.7, documentado explícitamente como señal **débil** (un APT podría evadir detección con esta misma huella).

Regla metodológica por contrato: `is_likely_benign` no usa `malscore` ni familia/tipo para decidir benignidad: usarlos reintroduciría el bucle "CAPE dice malware con score alto y este componente lo rebaja por su propio patrón basado en ese score".

Cobertura de tests (`tests/test_fp_correction.py`, 15 casos en 5 clases): `TestBasicCases` (3), `TestRegistrationHooks` (8: registro de imphash, sha256 y nombre de signature, insensibilidad a mayúsculas en imphash, coincidencia parcial de signature que NO dispara, registro de patrón custom, excepción en patrón capturada y `list_active_rules`), `TestNoLeakage` (2: la firma no acepta `malscore`; `malscore` presente en features no decide nada), `TestReset` (1) y `TestFPOnVMReports` (1, ningún informe real de la VM se marca como falso benigno con las reglas por defecto vacías).

6.2.5 `urgency_fusion.py`: fusión multiplicativa (236 líneas)

Combina `severity_v2` (0-10), `anomaly_score` (0-1) y `fp_result` (de `fp_correction`) en una **urgencia operativa** 0-10 más un bucket categórico, mediante una fórmula multiplicativa explícita y auditable:

```
base    = severity_v2
a_boost = 1.0 + 0.20 * anomaly_score          # 1.00 - 1.20
fp_pen  = max(0.2, 1.0 - 0.8 * benign_score)  # 0.20 - 1.00
ctx_mul = context_multiplier(context)         # 0.80 - 1.30, default 1.0
urgency = clip(base * a_boost * fp_pen * ctx_mul, 0, 10)
```

Buckets (`BUCKET_THRESHOLDS`): `critico` si `urgency >= 8.0`, `alto` si `6.0 <= urgency < 8.0`, `medio` si `3.0 <= urgency < 6.0`, `bajo` en otro caso.

El **multiplicador de contexto** (`context_multiplier`) es un hook documentado y testeado pero **desconectado** en el MVP (la integración multi-fuente queda fuera de alcance). Si `context=None` (el caso siempre en el MVP), devuelve `(1.0, {})`. Si se le pasa un dict con `host_criticality` (`low`=0.90, `medium`=1.00, `high`=1.15, `critical`=1.30) y/o `user_privileges` (`standard`=0.95, `admin`=1.10, `root`=1.20), los multiplica y capa el resultado a `[0.80, 1.30]`.

`compute_urgency(severity_v2, anomaly_score, fp_result=None, context=None)` valida que `severity_v2` ∈ `[0,10]` y `anomaly_score` ∈ `[0,1]` (lanza `ValueError` si no), acepta `fp_result` como `FPCheckResult`, dict equivalente o `None` (= `benign_score=0`), y devuelve un `UrgencyResult` cuya propiedad `breakdown` expone la cadena completa `base_severity x anomaly_boost x fp_penalty x context_multiplier = urgency_score`; es este breakdown el que se muestra al analista en la ficha de `/ui/triage/{rank}` (§6.5).

Regla metodológica por contrato: `compute_urgency` no acepta `malscore` ni lo expone en `UrgencyResult`. El docstring del módulo es explícito sobre la consecuencia: si CAPE marca una muestra con `malscore=10` y el motor v2 calcula `severity_v2=2`, `anomaly_score=0.1` y un benigno fuerte, dando `urgency≈0.4`, esa **divergencia** entre CAPE y el motor v2 es información valiosa para el analista: se muestra al lado en la cola, pero no entra en la fórmula.

Cobertura de tests (`tests/test_urgency_fusion.py`, 26 casos en 8 clases): `TestMonotonia` (3: severidad, anomalía y FP monótonos en la dirección esperada), `TestCaps` (6: severidad 0 -> urgencia 0, inputs máximos capados en 10, boost de anomalía máximo, suelo de penalización FP, `ValueError` por `severity_v2` fuera de `[0,10]`, `ValueError` por `anomaly_score` fuera de `[0,1]`), `TestBuckets` (3), `TestContextMultiplier` (7: `None`/vacío -> neutro, keys inválidas ignoradas, criticidad de host, privilegios de usuario, combinación capada, combinación baja capada por abajo), `TestNoLeakage` (2), `TestFPInputForms` (3: `None`, dict, tipo inválido -> `TypeError`), `TestBreakdown` (1) y `TestEndToEndOnVMReports` (1, el pipeline severidad->anomalía->FP->urgencia corre sin error sobre los informes reales de la VM CAPE 2.5).

En la cola priorizada generada sobre n=48.976 (Fase D), la distribución de buckets es `{crítico: 364, alto: 22.562, medio: 22.201, bajo: 3.849}` (0,7 % crítico). Los casos con mayor divergencia frente al ranking por malscore son njRAT (+37.231 puestos, promovido) y Swisyn (−21.791 puestos, rebajado); ver cap07.

6.2.6 `secops_summary.py` y `ollama_client.py`: síntesis dual (414 + 192 líneas)

`ollama_client.py` es un cliente HTTP mínimo contra Ollama (`localhost:11434`, timeout por defecto 180 s para `/api/chat` y 5 s para la comprobación de disponibilidad en `/api/tags`). Expone `is_available()`, `list_models()`, `chat(model, user_prompt, format_json=True, temperature=0.2, num_ctx=4096)` y `chat_json(...)` (parsea la respuesta a dict, devuelve `{}` si el parseo falla).

El módulo incorpora un hallazgo metodológico verificado el 2026-06-01: un prompt directo del tipo "resume malware Emotet" provoca que `llama3.2:3b` **rechace** la petición por guardarrail de seguridad. La solución aplicada es `DEFENSIVE_SOC_SYSTEM_PROMPT`, un mensaje `system` que encuadra la tarea como **análisis defensivo de un SOC** sobre informes ya generados por un sandbox: trabajo legítimo de ciberseguridad, no creación de malware. Con este `system`, tanto `llama3.2:3b` como `phi4-mini` generan la ficha de triaje en español correctamente. `chat()` siempre antepone este `system` (o el que se pase explícitamente al construir el cliente) al mensaje `user`, y `format=true` en la petición fuerza a Ollama a devolver JSON válido.

Cobertura de tests (`tests/test_ollama_client.py`, 15 casos en 4 clases): `TestIsAvailable` (4: 200, error de red, timeout, código no-200), `TestListModels` (2), `TestChat` (5: payload correcto, `format` deshabilitado si se pide, respuesta normalizada, `OllamaUnavailable` en error de red, `RuntimeError` si la respuesta trae `error`), `TestChatJson` (2: JSON válido e inválido) y `TestSystemPrompt` (2: el system prompt por defecto es el defensivo, y puede sobreescribirse).

`secops_summary.py` genera la ficha narrativa para el analista a partir de `severity`, `anomaly_score`, `urgency` y el payload del parser. Contrato único de salida (`SummaryResult`) en ambos modos: `que_hizo`, `por_que_importa`,

iocs (lista), accion_recomendada, narrativa, más metadatos mode_used, model, latency_seconds y payload_fidelity.

- **Modo `template`** (_template_summary): NLG determinista sin LLM. que_hizo describe familia, tipo, número de firmas y TTPs, y el comportamiento agregado (file_ops, registry_ops, tcp_count, process_count). por_que_importa cita urgency_score, bucket, severity_v2 y anomaly_score, y añade una mención a CISA Top Malware si la familia está en {emotet, qakbot, trickbot, lokibot, ursnif, njrat}. accion_recomendada se toma de ACTION_BY_BUCKET: critico -> "Aislar el host inmediatamente y escalar a SOC nivel 2 (S1)"; alto -> "Aislar el host, recolectar evidencia y abrir ticket SOC"; medio -> "Revisar manualmente en cola normal, recolectar IOCs"; bajo -> "Monitorizar; reanalizar si reaparece patrón".
- **Modo `llm`**: construye un prompt user con los hechos del caso (_build_user_prompt, incluye el malscore **solo como referencia explícita "NO usado en urgencia"**) y pide un JSON con los 5 campos. _normalize_llm_output aplana variantes (p. ej. iocs como CSV en vez de lista) y trunca longitudes. Si al LLM le faltan campos, se completan desde la plantilla y se recalcula la fidelidad.
- **Modo `auto`**: si OllamaClient.is_available() es True, usa llm; si no, degrada a template sin error.
- En cualquier modo, _check_payload_fidelity calcula 5 flags post-hoc: mentions_family, mentions_urgency_value, mentions_bucket, mentions_at_least_one_ttp, has_all_required_fields.

Regla metodológica por contrato: generate_summary no acepta malscore como argumento independiente; si el payload lo contiene, el prompt al LLM lo menciona únicamente como dato comparativo, nunca como input que cambie la urgencia.

Cobertura de tests (tests/test_secops_summary.py, 16 casos en 7 clases): TestTemplateMode (4), TestAutoGraceful (1, cae a template si Ollama no responde), TestLLMModeMocked (4, incluye normalización de IOCs como string y fallback a template si Ollama no está disponible), TestNoLeakage (2), TestAntiRefusalSystemPrompt (2, verifica que el encuadre defensivo está presente en el prompt y que el cliente por defecto lo usa), TestInputValidation (1, urgency debe ser UrgencyResult) y TestLLMRealOptional (2, marcados para ejecutarse contra un Ollama real si está disponible).

En la comparativa real (Fase E, n=2 muestras Qakbot, run_secops_summary.py): la fidelidad de la plantilla es 0,60, frente a 0,40 de phi4-mini (35,78 s por ficha) y 0,20 de llama3.2:3b (31,98 s por ficha), ambas latencias en CPU Ryzen 5 5600. Esta diferencia motiva usar la plantilla como modo por defecto (§6.3).

6.3 Orquestación: `triage_service.py`

app/services/triage_service.py (266 líneas) cablea los componentes 1 a 5 en dos funciones públicas.

DEFAULT_SUMMARY_MODE: Literal["template", "llm", "auto"] = "template": esta constante materializa la **decisión de diseño** del 3 de junio de 2026: el resumen por defecto del MVP usa la plantilla determinista, no el LLM, porque su fidelidad (0,60) supera a la de ambos modelos locales evaluados (0,40 y 0,20, §6.2.6). El modo LLM queda disponible como parámetro explícito (summary_mode="llm"/"auto") para experimentación, sin alterar el comportamiento por defecto.

TriageResult (dataclass) agrupa el resultado íntegro de un caso: sha256, family, type, parser_version, malscore_cape (float, **solo referencia**), y los cinco resultados de componente (severity: SeverityResult, anomaly: AnomalyResult, fp: FPCheckResult, urgency: UrgencyResult, summary: SummaryResult), más summary_mode_used, pipeline_version="v2" y notes. as_dict() serializa cada componente delegando en su propio .as_dict(), produciendo un único JSON anidado y trazable de extremo a extremo.

Carga perezosa del modelo de anomalía: _get_anomaly_engine() mantiene un singleton _engine_cache a nivel de módulo. La primera llamada construye AnomalyEngine() y llama a load(); si el fichero data/models/anomaly_v2.joblib no existe, captura FileNotFoundError y devuelve None; el llamante entonces usa

un resultado neutro (`anomaly_score=0.5`, `is_anomaly=False`, con una nota explicando la ausencia de modelo) en vez de fallar. `reset_anomaly_engine_cache()` permite a los tests forzar la recarga.

`triage_from_payload(event_payload, summary_mode=DEFAULT_SUMMARY_MODE, summary_model=DEFAULT_MODEL, ollama_client=None, family_override=None, type_override=None)` ejecuta la secuencia completa:

1. Resuelve `family`/`type` desde `family_override`/`type_override` o, si no se pasan, desde `event_payload["family"]`/`["detections"]` y `["type"]`/`["classification_type"]`. Inyecta ambos en una copia del payload (no muta el original).
2. **Severidad (B)**: `compute_severity(family, type_, sig_max_severity, sig_sum_weight, sig_high_severity_count, ttp_codes)`. `ttp_codes` se parsea desde CSV si viene como string.
3. **Anomalía (C1)**: si hay motor de anomalía cargado, construye el vector de las 14 `ANOMALY_FEATURE_NAMES` desde el payload y llama a `engine.score(...)`; si no, usa el fallback neutro descrito arriba.
4. **Corrección de FP (C2)**: `is_likely_benign({pe_imphash, sha256, sig_count, ttp_count, tcp_count, file_ops, sig_names})`.
5. **Fusión (D)**: `compute_urgency(severity_v2=severity.score, anomaly_score=anomaly.anomaly_score, fp_result=fp, context=None)`, con `context=None` explícitamente, porque no hay fuente de contexto conectada.
6. **Síntesis (E)**: `generate_summary(payload, severity, anomaly.anomaly_score, urgency, mode=summary_mode, model=summary_model, ollama_client=ollama_client)`.
7. Construye y devuelve `TriageResult`, con `notes` documentando explícitamente que `malscore_cape` no entra en `urgency`.

`triage_from_cape_report(report, summary_mode=..., summary_model=..., ollama_client=None, family=None, type_=None, anonymize=True)` es el punto de entrada desde un `report.json` crudo: llama a `parse_cape_report_v2(report, anonymize=anonymize)`, convierte el `CapeFeatures` a dict con `.as_dict()` y delega en `triage_from_payload`.

Cobertura de tests (`tests/test_triage_service.py`, 15 casos en 6 clases): `TestTriagePipelineTemplate` (6: resultado completo, severidad calculada, urgencia en rango, resumen en modo template, modo por defecto = template, `as_dict()` serializable), `TestNoLeakage` (2: la firma no acepta `malscore`; **cambiar `malscore` en el payload de entrada no altera `urgency`**, test funcional y no solo de firma), `TestCoherence` (2: la severidad y urgencia del triaje coinciden con llamar directamente a `compute_severity`/`compute_urgency`), `TestAnomalyFallback` (1, score neutro 0.5 cuando no hay modelo entrenado, con `monkeypatch`+`tmp_path`), `TestRobustness` (3: payload mínimo no rompe, familia desconocida sin bono CISA, override de familia/tipo) y `TestLLMModeMocked` (1, modo LLM con cliente inyectado).

6.4 API: rutas v2

`app/api/routes_triage.py` (191 líneas) define `router = APIRouter( prefix="/api/v2", tags=["triage-v2"])`, registrado en `app/main.py` junto a los routers v1 (`events_router`, `score_router`, `incidents_router`, `status_router`) y `ui_router`, **sin modificar ninguno de los existentes**:

```
app.include_router(events_router)
app.include_router(score_router)
app.include_router(incidents_router)
app.include_router(status_router)
app.include_router(triage_router)   # ← nuevo, pivote v2
app.include_router(ui_router)
```

Modelos Pydantic de petición/respuesta: `TriageRequest` (`event_payload: dict`, `mode: Literal["template","llm","auto"] = DEFAULT_SUMMARY_MODE`, `model: str | None`), `TriageCapeReportRequest` (igual más `report: dict`, `family`/`type` opcionales y `anonymize: bool = True`), `TriageQueueItem` (una fila de la cola: `rank`, `sha256_prefix`, `family`, `type`, `bucket`,

urgency_score, severity_v2, anomaly_score, fp_benign_score, malscore_cape, rank_shift) y TriageQueueResponse (total, items, source_csv, note).

Tabla 6.1: Rutas de la API v2 en la implementación.

Ruta	Método	Descripción
/api/v2/triage	POST	triage_from_payload sobre un payload ya parseado. ValueError -> 422.
/api/v2/triage/cape_report	POST	triage_from_cape_report sobre un report.json crudo (CAPE 2.2 o 2.5).
/api/v2/triage/queue	GET (?top=20)	Lee el priority_queue_*.csv más reciente de reports/. 404 con instrucción (build_priority_queue.py) si no existe.
/api/v2/triage/queue/{rank}	GET	Fila concreta de la cola por rank. 404 si no hay CSV o el rank no existe.

_find_latest_queue_csv() ordena los priority_queue_*.csv de settings.reports_dir por mtime descendente y devuelve el más reciente (o None). _read_queue(top) los lee con csv.DictReader, limitando a top filas.

Las rutas v1 permanecen intactas; tests/test_routes_triage.py incluye una clase dedicada (TestV1ContractIntact) que ejecuta /api/v1/status/model y /health para detectar regresiones.

6.5 UI Atalaya (/ui/triage y /ui/triage/{rank})

app/api/routes_ui.py añade dos rutas HTML al ui_router existente, sin modificar dashboard, incident_detail ni model_status_page (Gates 4-5 intactos). Ambas reutilizan _find_latest_queue_csv() / _read_queue_csv() (misma fuente que la API v2) y un helper _bucket_class(bucket) que mapea cada bucket a una clase CSS (bucket-critico, bucket-alto, bucket-medio, bucket-bajo).

GET /ui/triage?top=50 -> triage_queue.html: lee el CSV más reciente, enriquece cada fila con valores redondeados para la plantilla (urgency_score_f, malscore_cape_f, severity_v2_f, anomaly_score_f, fp_benign_score_f), calcula bucket_class, y deriva el signo del rank_shift en rank_shift_direction (promovido/rebajado/sin cambio) y rank_shift_class (shift-up/shift-down/shift-zero) para colorear la tabla. Agrega también bucket_counts ({critico, alto, medio, bajo}) para las 4 tarjetas de resumen. Si no hay CSV, la plantilla muestra un estado vacío con la instrucción de ejecutar build_priority_queue.py. Sobre la tabla, la plantilla construye el panel **«Divergencia real frente a CAPE»** con dos sub-tablas, *Promovidos* (mayores rank_shift positivos) y *Rebajados* (mayores negativos), filtrando las filas a malscore_cape < 10 para comparar posiciones 1:1 fuera del bloque saturado; cada fila muestra el malscore de CAPE junto al Shift. Es la vista que hace explícita la re-valoración bidireccional del sistema.

La cola priorizada y el panel de divergencia se muestran en las Figuras D.1-D.3 (Anexo D).

GET /ui/triage/{rank} -> triage_detail.html: localiza la fila de la cola con ese rank (404 si no existe o no hay CSV); intenta completarla con la fila correspondiente del último avast_ctu_*_dataset.csv (buscando por prefijo de sha256, primeros 16 caracteres); combina ambas fuentes en un único payload, convirtiendo ttp_codes_str (CSV) a lista y coaccionando a int/float los campos numéricos clave (conteos de

firmas, red, comportamiento). Con ese payload reconstruido, **vuelve a ejecutar el pipeline completo** vía `triage_from_payload(payload, summary_mode="template", family_override=..., type_override=...)`, con la plantilla como modo por defecto también en la UI. La plantilla recibe el `TriageResult` serializado, el `breakdown` de la urgencia (`base_severity x anomaly_boost x fp_penalty x context_multiplier = urgency_score`) y `bucket_class`.

La ficha explicable, con el desglose de la fórmula y el aporte por componente, se muestra en las Figuras D.1-D.3 (Anexo D).

Decisión documentada: recomputar en vivo (en vez de persistir el resultado completo) es deliberado: el CSV de la cola solo trae agregados, y recomponer desde la fila + dataset es determinista y rápido (< 1 s por ficha), evitando duplicar el almacenamiento del resultado completo del pipeline.

`base.html` reorganiza la navegación en torno al pipeline v2: enlaza la cola de triaje (`/ui/triage`, también destino de la raíz `/` vía redirección), la página de estado del modelo v1 (`/status/model`, marcada como legacy) y la documentación de la API (`/docs`); el dashboard v1 queda accesible en `/ui/legacy` con aviso de legacy, fuera de la navegación. `style.css` se extiende con clases nuevas (`bucket-*`, `shift-*`, desglose, badges de resumen) **al final del fichero**, sin tocar las clases existentes: preserva el contrato de `test_ui_contract.py` (Gate 5 UI).

Cobertura de tests (parte de `tests/test_routes_triage.py`, `TestUIv2`, 2 casos): la cola renderiza con estado vacío cuando no hay CSV (`monkeypatch`+`tmp_path`), y la ficha de detalle devuelve 404 con el detalle correcto cuando no hay cola.

6.6 Pruebas

6.6.1 Estrategia

Cada componente del pipeline v2 tiene su fichero de tests dedicado, con tres tipos de clases recurrentes:

- **Clases de comportamiento**: monotonía, casos límite, validación de rangos, formatos de entrada alternativos.
- **`TestNoLeakage`**: presentes en `severity_engine`, `anomaly_engine`, `fp_correction`, `urgency_fusion`, `secops_summary` y `triage_service`; verifican por introspección de firma y/o funcionalmente que ningún componente acepta ni usa `malscore`/`cape_score`/`score` (ver §6.7).
- **Clases sobre informes reales** (`TestAvastCTUReal`, `TestVMCape25Real`, `TestSeverityOnVMRealReports`, `TestAnomalyOnVMReports`, `TestEndToEndOnVMReports`): ejecutan el pipeline sobre fixtures de `report.json` reales: 3 informes Avast-CTU (esquema 2.2), 3 informes capturados de la VM CAPE 2.5 para el parser, y 11 informes de la VM CAPE 2.5 para validar severidad/anomalía/urgencia de extremo a extremo. Estas fixtures se capturaron en sesiones previas de la VM `ubuntu-cape` y son independientes de la disponibilidad actual de la máquina (§6.6.3).

6.6.2 Conteo de tests por fase

Tabla 6.2: Conteo de tests por fase del pivote v2.

Fase	Módulo(s)	Tests nuevos	Total acumulado
(baseline, pre-pivote)	v1 + transversales	—	53
A	`cape_parser_v2.py`	26	79
B	`severity_engine.py`	34	113
C	`anomaly_engine.py` (14) + `fp_correction.py` (15)	29	142

Fase	Módulo(s)	Tests nuevos	Total acumulado
D	`urgency_fusion.py`	26	168
E	`secops_summary.py` (16) + `ollama_client.py` (15)	31	199
F	`triage_service.py` (15) + `routes_triage.py` (11)	26	**225**

El conteo coincide con la cifra que reportaba el cap06 anterior para el baseline pre-pivote (53 tests, Gate 5 no-VM 2026-05-02): 53 + 172 (suma de los tests A-F) = 225. **Nota**: los tests de las clases `TestSeverityOnVMRealReports`, `TestAnomalyOnVMReports`, `TestEndToEndOnVMReports` y equivalentes requieren las 11 fixtures de `docs/evidencias/cape_reports_reales/` (informes reales de la VM CAPE 2.5, gitignored); en un clone limpio sin esas fixtures, estos tests se saltan automáticamente con `pytest.skip`. La cifra 225 es el total de tests *definidos* **al cierre del pivote v2**; el conjunto *ejecutable* sin fixtures gitignored es 225 menos los tests que verifican esos reportes. La suite ha seguido creciendo con las correcciones posteriores al cierre: a fecha de redacción de esta memoria, `pytest --collect-only` recolecta **241** tests a 18 de julio de 2026 (228 pasan y 13 se omiten automáticamente por depender de las fixtures gitignored citadas arriba).

6.6.3 Demo end-to-end

`scripts/demo.ps1` ejercita el flujo end-to-end del prototipo **v1** (`/api/v1/events/ingest` → `/api/v1/score` → `/api/v1/incidents` → `/api/v1/status/model`) mediante 21 comprobaciones. Resultado de la última ejecución (2026-06-08): **21/21 checks PASS**. Su función tras el pivote es servir de **prueba de no regresión del contrato v1**, que el desarrollo de v2 no debía romper (de ahí que se ejecutara de nuevo al cerrar el MVP v2). El flujo v2 **no está cubierto por esta demo**: su verificación end-to-end automatizada son los tests de `test_triage_service.py` (15) y `test_routes_triage.py` (11), y su comprobación manual, la UI de §6.5.

`scripts/build_cap07_evaluation.py` consolida los reportes gitignored de las 5 fases (`eda_avast_ctu_*.json`, `severity_comparison_*.json`, `anomaly_fp_eval_*.json`, `priority_queue_*.json`, `secops_summary_comparison_*.json`) en `reports/cap07_evaluation_<ts>.{json,md}` y marca explícitamente como "AUSENTE" cualquier fase cuyo reporte no encuentre, con la instrucción del script que lo regenera; el reporte ejecutado el 2026-06-11 (`cap07_evaluation_20260611_175627`) es la base directa del capítulo 7.

6.7 Garantías de ingeniería

6.7.1 Anti-leakage de extremo a extremo (verificado en código)

La decisión de diseño central del pivote v2 es que **ningún componente del pipeline use `malscore`/`cape_score`/`score` para decidir severidad, anomalía, corrección de FP o urgencia**. Esto no es una afirmación narrativa: está verificado por tests introspectivos y funcionales en cada módulo:

- `severity_engine`: `compute_severity` no tiene parámetro `malscore`; `SeverityResult` no lo expone (`TestNoLeakage`, 2 tests).
- `anomaly_engine`: `FEATURE_NAMES` (14 elementos) no incluye ninguna variante de score/label; `features_to_vector` ignora `malscore` si está en el dict de entrada; la firma de `fit` no acepta labels (`TestNoLeakage`, 3 tests).
- `fp_correction`: `is_likely_benign` no usa `malscore` para decidir, ni aunque esté presente en `features` (`TestNoLeakage`, 2 tests).
- `urgency_fusion`: `compute_urgency` no acepta `malscore`; `UrgencyResult` no lo expone (`TestNoLeakage`, 2 tests).

- `secops_summary`: `generate_summary` no acepta `malscore` como argumento independiente; la plantilla no cita el malscore como si fuera la urgencia (`TestNoLeakage`, 2 tests).
- `triage_service`: la firma de `triage_from_payload`/ `triage_from_cape_report` no acepta `malscore`; y, el test más fuerte, **`test_malscore_in_payload_does_not_affect_urgency`** construye dos payloads idénticos salvo el valor de `malscore` y verifica que `urgency_score` no cambia (`TestNoLeakage`, 2 tests).

En total, 13 tests `TestNoLeakage` distribuidos en 6 módulos. El único punto del sistema donde `malscore` reaparece es el campo `malscore_cape` de `TriageResult`, explícitamente documentado como "solo display" en el docstring de `triage_service.py` y usado en la UI únicamente para mostrar la divergencia entre lo que decía CAPE y la urgencia recalculada.

6.7.2 Degradación elegante del LLM

El componente 5 (`secops_summary` y `ollama_client`) está diseñado para que la ausencia de Ollama nunca rompa el pipeline: `mode="auto"` comprueba `is_available()` antes de intentar el LLM y cae a `template` si no responde; `mode="llm"` explícito captura `OllamaUnavailable` y degrada igualmente a `template` (`TestAutoGraceful`, `TestLLMModeMocked.test_llm_falls_back_to_template_on_unavailable`). Combinado con la plantilla por defecto (`DEFAULT_SUMMARY_MODE="template"`), el pipeline completo, incluida la UI, funciona sin ninguna dependencia de red ni de un servicio externo instalado.

6.7.3 Ingesta sin leakage: la etiqueta viene del CSV, no del informe

`scripts/ingest_avast_ctu.py` (Fase A) construye el dataset de evaluación asociando cada `report.json` con su `family`/`type` de ground truth desde `public_labels.csv` del dataset Avast-CTU, y **no** desde ninguna heurística sobre el propio informe (a diferencia del `cape_parser.py` de v1, que derivaba `label` de `info.score` con los umbrales `≥7.0`/`≥3.5`, documentado como bug de leakage en el diseño v2). `test_avast_no_label_derivation` (`test_cape_parser_v2.py`) verifica que `parse_cape_report_v2` no emite ningún campo de etiqueta/clasificación de malicia a partir del contenido del informe. La familia/tipo se inyecta después, explícitamente, vía `family_override`/`type_override` en `triage_from_payload`, visible en la firma misma de la función (§6.3).

6.7.4 Relación con Gate 3 y la máquina CAPE

A fecha de cierre de este capítulo (2026-06-14), la máquina donde corría la VM `ubuntu-cape` (CAPE 2.5) está parada por un problema de procesador aún sin diagnosticar. Esto **no bloquea** lo descrito en este capítulo: el pivote v2 es offline-first por diseño y se ha desarrollado, testeado y cerrado (MVP cerrado en junio de 2026, 225 tests, demo 21/21) íntegramente contra el dataset Avast-CTU (n=500, descargado y procesado en Fase A) y contra los `report.json` reales capturados de la VM en sesiones previas (3 fixtures para el parser, 11 para severidad/anomalía/urgencia, §6.6.1). La validación con CAPE 2.5 en vivo sobre la máquina de la Universidad queda como trabajo futuro (capítulo 8), no como una pieza pendiente de este capítulo.

Capítulo 7. Evaluación experimental

Nota de alcance (leer antes de interpretar los resultados): Este capítulo evalúa el **pipeline v2** descrito en el capítulo 5: parser CAPE multi-schema → motor de severidad intrínseca → motor de anomalía conductual + corrector de falsos positivos → motor de fusión de urgencia → síntesis SecOps. El sistema **no reclasifica malware** (CAPE ya lo hace); re-valora el veredicto de CAPE (`malscore`) con señales independientes de él, y el objetivo de la evaluación es comprobar si esa re-valoración es **más discriminativa y más accionable** que el `malscore` bruto.

Los resultados de esta versión proceden de los **48.976 informes reales de CAPEv2** del dataset público Avast-CTU (dataset completo `full`, 13,97 GB), con etiquetas de familia y tipo tomadas de `public_labels.csv`, una fuente **externa** al contenido del informe y base de la garantía anti-leakage descrita en el capítulo 5. La re-evaluación con el dataset completo se realizó el 2026-06-27 (versión de ingesta `20260626_225446`, 48.976/48.976 informes parseados sin error); todos los hallazgos estrella de §7.2–§7.5 se confirman a esta escala sin cambio de signo ni de conclusión. La validación final con informes CAPE 2.5 generados por la VM propia del proyecto (Gate 3) **sigue pendiente** (ver §7.8) y no condiciona los resultados de este capítulo, porque el diseño v2 no depende de ella.

Por transparencia metodológica, §7.9 resume la evaluación del prototipo v1 (clasificador supervisado sobre dataset sintético, F1 = 1,0 / FP = 0,0) y el motivo por el que **se descarta como resultado principal del TFG**: una auditoría funcional (2026-05-31) detectó leakage circular entre la etiqueta y el `cape_score`, y separabilidad artificial del generador sintético. Esa auditoría es la que originó el pivote a v2 documentado en este capítulo.

7.1 Configuración experimental

7.1.1 Dataset

Tabla 7.1: Configuración experimental.

Parámetro	Valor
Fuente	Avast-CTU Public CAPE Dataset (github.com/avast/avast-ctu-cape-dataset, arXiv:2209.03188)
Dataset utilizado	`full` (13,97 GB, 48.976 informes), dataset completo
Muestras evaluadas (n)	**48.976**

Parámetro	Valor
Familias	10: Emotet, Qakbot, Trickbot, Lokibot, njRAT, Zeus, Ursnif, Adload, Swisyn, HarHar
Tipos	6: banker, trojan, pws, coinminer, rat, keylogger
Etiquetas	`public_labels.csv` (sha256 → familia, tipo, fecha), fuente externa al informe
Versión de ingesta	`20260626_225446`
Ingesta	`scripts/ingest_avast_ctu.py --source full` (48.976/48.976 informes, 0 errores, 0 sin label)

El número de muestras por familia en n = 48.976 es el siguiente (usado como base de las tablas §7.2–§7.5):

Tabla 7.2: Distribución del dataset de evaluación por familia.

Familia	n
Emotet	14.429
Swisyn	12.591
Lokibot	4.191
Qakbot	4.895
njRAT	3.372
Trickbot	4.202
Zeus	2.594
Ursnif	1.343
Adload	704
HarHar	655
Total	**48.976**

La distribución no está balanceada por diseño: refleja la composición real del dataset Avast-CTU, no un muestreo estratificado. Emotet (14.429) y Swisyn (12.591) dominan el promedio global por su peso muestral, con una presencia mucho más representativa que en el subconjunto de 500 utilizado en fases previas de desarrollo.

7.1.2 Componentes evaluados

Tabla 7.3: Componentes evaluados y su naturaleza.

Componente	Implementación	Naturaleza
Parser	`cape_parser_v2.py`	Determinista, multi-schema (CAPE 2.2/2.5)
Severidad intrínseca	`severity_engine.py`	Función determinista, 4 componentes acotados (cap. 5.4)

Componente	Implementación	Naturaleza
Anomalía conductual	`anomaly_engine.py`	`IsolationForest` (`contamination=0.10`, `n_estimators=200`, `random_state=42`), 14 features
Corrector de FP	`fp_correction.py`	Reglas extensibles, listas vacías en este corte
Fusión de urgencia	`urgency_fusion.py`	Función determinista multiplicativa (cap. 5.6)
Síntesis SecOps	`secops_summary.py` / `ollama_client.py`	Plantilla determinista + LLM local (Ollama: `phi4-mini:latest`, `llama3.2:3b`)

A diferencia del prototipo v1, el motor de severidad y el motor de fusión **no se entrenan**: son funciones deterministas evaluadas directamente sobre las 48.976 muestras. El único componente entrenado es el motor de anomalía, ajustado sobre las mismas 48.976 muestras (n_train = n_eval = 48.976); esta ausencia de partición train/test para el componente de anomalía se discute como amenaza a la validez en §7.7.

7.1.3 Preguntas e hipótesis evaluadas

Este capítulo aporta evidencia para las preguntas de investigación e hipótesis del capítulo 3:

Tabla 7.4: Preguntas de investigación e hipótesis, y sección que las responde.

Sección	RQ / H	Resumen
§7.2 – §7.3	RQ1, H1	¿La re-valoración contextual discrimina mejor que el `malscore` bruto?
§7.4	H2	¿La anomalía conductual aporta información que familia/tipo no capturan?
§7.5	RQ1, RQ2, H1, H2	Cola priorizada final: distribución por bucket y divergencias (`rank_shift`) frente al `malscore`
§7.6	RQ3, H3	¿La síntesis SecOps es fiel al payload técnico (plantilla vs LLM)?

7.2 Tabla estrella 1: saturación del `malscore` frente a `severity_v2`

El hallazgo de partida de la Fase A (EDA sobre el dataset ingerido, `scripts/eda_avast_ctu.py`) es que el `malscore` de CAPE **no discrimina urgencia entre familias** en este dataset: satura en su valor máximo para una mayoría de muestras.

Tabla 7.5: Saturación del `malscore` frente a `severity_v2`.

Métrica	`malscore` (CAPE)	`severity_v2`
% de muestras en el valor máximo (10,0)	**56,28 %** (de 48.976)	**0,0 %**
Familias con mediana = 10,0	**7/10**	**0/10**

Las 7 familias cuya mediana de `malscore` es 10,0 son Qakbot, njRAT, Emotet, Trickbot, Ursnif, Zeus y Lokibot, las mismas en el dataset completo que en el subconjunto de 500. Es decir, **el `malscore` por sí solo trataría a 7 de las 10 familias del dataset como igualmente urgentes (máxima urgencia)**, lo que en una cola priorizada equivaldría a una cola plana sin orden útil dentro de ese 70 % de familias.

La siguiente tabla compara, por familia, la media del `malscore` original frente a la media de `severity_v2` calculada por el motor del capítulo 5 (fuente: `scripts/compare_severity.py`):

Tabla 7.6: Saturación por familia: `malscore` medio frente a `severity_v2` medio.

Familia	n	`malscore` medio	`severity_v2` medio
Qakbot	4.895	9,80	6,44
njRAT	3.372	7,38	6,04
Emotet	14.429	9,41	5,77
Ursnif	1.343	8,67	5,66
Trickbot	4.202	8,45	5,58
Lokibot	4.191	8,79	5,01
Zeus	2.594	9,25	4,95
Adload	704	7,05	3,82
Swisyn	12.591	4,20	3,45
HarHar	655	2,67	1,89

Dos lecturas se desprenden de esta tabla:

1. **Anti-saturación**: `severity_v2` reordena dentro del rango 1,9–6,4 lo que el `malscore` aplana en el rango 7,1–9,8 para 8 de las 10 familias. La diferencia entre Qakbot (6,44) y Zeus (4,95), de 1,5 puntos sobre 10, es invisible para el `malscore` (9,80 vs 9,25).
2. **El orden relativo no se invierte arbitrariamente**: las familias que el `malscore` ya situaba más bajo (Swisyn, HarHar) siguen siendo las de menor `severity_v2`. La re-valoración **redistribuye dentro del rango alto**, no contradice sistemáticamente a CAPE.
3. **El `malscore` no es ciego, pero tiene un punto ciego concreto**: sí separa el tramo bajo (HarHar 2,7; Swisyn 4,2; Adload 7,1) de las siete familias restantes. Lo que no ofrece es **resolución dentro de ese tramo alto** (las 7 familias con mediana 10,0, el 70 % de las muestras), que es precisamente donde un SOC necesita priorizar: entre "todo es crítico" y "esto es más crítico que aquello".

7.3 Tabla estrella 2: separación inter-familia y descomposición del η^2 (H1)

La primera versión de esta sección comparaba un único η^2 agregado de `severity_v2` (0,833 sobre el dataset completo) frente al de `malscore` (0,510) y leía la diferencia (+32,3 pp) como evidencia de que

`severity_v2` "discrimina mejor" entre familias. Una revisión metodológica posterior (12 de junio de 2026) identificó que esa lectura es **circular** y la reformula aquí con una **descomposición por componente**, que es la versión que se mantiene como resultado de este capítulo.

Por qué el η² agregado es circular

En el dataset Avast-CTU, **cada familia tiene exactamente un tipo** (`type_ordinal`), por lo que agrupar por familia y agrupar por tipo son, a efectos de varianza, casi el mismo particionado. `severity_v2` incluye `type_part` y `cisa_part` (ambos derivados directa o indirectamente de la familia/tipo) como dos de sus cuatro componentes (capítulo 5.4). El camino `familia → tipo → severity_v2 → agrupar por familia` cierra un círculo: parte de lo que el η² agregado atribuye a "mejor discriminación" es, en realidad, la etiqueta reapareciendo dentro de su propia métrica de evaluación. **Cuantificando la circularidad**: en el dataset Avast-CTU, `type_part` contribuye de media **2,29 puntos** y `cisa_part` **0,66 puntos**, sumando **2,95 de los 5,05 puntos** de `severity_v2` media (≈ **58 %**); ambos se derivan del campo `type`/`family` de `public_labels.csv`. El 42 % restante (`sig_part` + `ttp_part`) es señal puramente conductual y, como muestra la descomposición siguiente, queda ligeramente por debajo del `malscore` de CAPE (η² = 0,477 vs 0,510).

Descomposición del η² inter-familia (n = 48.976)

Para aislar qué parte de la separación es señal conductual legítima y qué parte es la etiqueta reapareciendo, se descompone `severity_v2` en sus componentes y se recalcula η² inter-familia para cada uno (fuente: `scripts/eta_decomposition.py`, script versionado y reproducible; resultado completo archivado en `docs/evidencias/eta_decomposition/eta_decomposition_20260717_001821_avast.{json,md}`):

Tabla 7.7: Descomposición del η² inter-familia por señal.

Señal	η² inter-familia
Solo componente `type_part`	**0,9916** (≈ 1:1 familia↔tipo; levemente <1,000 con la distribución completa de 48.976 muestras)
`severity_v2` completa	**0,833**
`severity_v2` sin `type_part`	**0,758**
Solo señal conductual (`sig_part` + `ttp_part`)	**0,477**
`malscore` (referencia CAPE)	**0,510**

Lectura de los resultados

1. **El η²(`type_part`) = 0,9916 (≈ 1,000) no es un hallazgo, es una propiedad del dataset**: si cada familia tiene esencialmente un único tipo, agrupar por familia y observar `type_part` es casi observar la propia etiqueta. A escala completa (48.976 muestras), el `type_part` baja levemente de 1,000 a 0,9916 porque la correspondencia familia↔tipo deja de ser perfecta 1:1 en la población completa. Gran parte del salto de 0,510 a 0,833 viene de aquí y de `cisa_part` (también ligado a familia/tipo), no de que el motor "entienda" mejor el malware.
2. **`severity_v2` separa familias PORQUE codifica explícitamente tipo y notoriedad (CISA): es una propiedad de diseño verificada, no un descubrimiento estadístico.** Asignar mayor severidad a un `banker` con presencia en CISA que a un `adware` sin ella es una política de priorización legítima y deliberada (capítulo 5.4); lo que esta sección corrige es la *interpretación* del η² agregado como prueba de capacidad predictiva, no el diseño del motor.
3. **En señal puramente conductual (signatures + TTPs MITRE, sin tipo ni CISA), `severity_v2` (0,477) y `malscore` (0,510) son comparables**. De hecho, la señal conductual queda ligeramente por debajo del `malscore` (gap −0,033, frente a −0,043 en el subconjunto de 500). Esto confirma

a escala completa que la ventaja de `severity_v2` sobre `malscore` para H1 no procede de una mejor lectura del *comportamiento* observado en sandbox, sino de incorporar contexto externo (tipo, CISA) que el `malscore` de CAPE no usa.

4. **Qué se invalida y qué no**: lo inválido es el *test* (el η^2 agregado como medida de "discriminación conductual"), no el *sistema*. El motor de severidad sigue siendo coherente, determinista, explicable y libre de leakage respecto al `malscore` (capítulo 5, `TestNoLeakage`); simplemente no debe leerse como evidencia de que "entiende mejor" el comportamiento de la muestra que el sandbox.
5. **El η^2 inter-tipo tiene la misma circularidad de forma aún más directa**: `type_part` es, por definición, una función de la variable de agrupación, por lo que no se reporta como evidencia independiente en esta versión; queda subsumido en la descomposición anterior.

La aportación del sistema: reordenar el bloque saturado

La señal conductual de `severity_v2` ($\eta^2 = 0{,}477$) no discrimina familias mejor que el `malscore` de CAPE (0,510): la tabla anterior lo confirma, y se asume sin reservas. La aportación del sistema **no es discriminativa sino estructural**: el `malscore` satura al 56,28 % de las muestras en su valor máximo (§7.2), produciendo una cola en la que el analista debe tratar el 56,28 % de las 48.976 muestras, de 10 familias distintas, como igualmente urgentes. `severity_v2` descomprime ese bloque con una **política explícita y trazable** (tipo + CISA + TTPs MITRE), generando un rango 1,7–6,4 donde cada posición tiene una justificación auditable componente a componente. El valor del sistema es el **reordenamiento trazable del bloque saturado de CAPE**, no una capacidad de discriminación conductual superior a la del sandbox. Esta interpretación, y no el η^2 agregado, es la que se sostiene para H1 en el resto de este capítulo.

Disclaimer metodológico: Spearman vs `type_ordinal`

Por completitud se incluye también la correlación de Spearman de cada score con el `type_ordinal` usado como `type_part` (capítulo 5):

Tabla 7.8: Correlación de Spearman frente a `type_ordinal`.

Score	ρ de Spearman vs `type_ordinal`	p-valor
`malscore`	0,465	< 1e-25
`severity_v2`	0,728	< 1e-83

Esta tabla es el mismo síntoma que la descomposición anterior, vista desde otro ángulo: `type_part` es uno de los cuatro componentes que suman `severity_v2`, así que una correlación alta con `type_ordinal` es en parte tautológica. Se reporta aquí únicamente por transparencia, para dejar constancia de su circularidad; la evidencia que se sostiene para H1 es la descomposición de la tabla anterior, no esta correlación ni el η^2 agregado.

7.4 Tabla estrella 3: anomalía conductual por familia y caso Swisyn (H2)

El motor de anomalía (`anomaly_engine.py`, IsolationForest sobre 14 features de comportamiento/red/estático, capítulo 5.5.1) se evaluó sobre las mismas 48.976 muestras (fuente: `scripts/evaluate_anomaly_fp.py`). La media de `anomaly_score` (0–1, sigmoide del `decision_function`) por familia es:

Tabla 7.9: Anomalía conductual por familia.

Familia	`anomaly_score` medio	% muestras flagged
Qakbot	**0,479**	**40,4 %**
Ursnif	0,450	34,9 %
njRAT	0,449	21,4 %
Trickbot	0,424	20,3 %
Zeus	0,412	15,2 %
Lokibot	0,403	5,1 %
Emotet	0,376	0,8 %
Adload	0,358	0,4 %
HarHar	0,346	0,0 %
Swisyn	0,321	1,2 %

Nota metodológica sobre el " % flagged": el total de muestras marcadas como anómalas en las 48.976 es 4.898 (10 %, coincide exactamente con el hiperparámetro `contamination=0,10` del `IsolationForest`, capítulo 5.5.1). Esa cifra global, por tanto, **no es un hallazgo**: es una consecuencia directa de cómo está configurado el modelo, no algo que el modelo "descubra". Lo informativo no es *cuántas* muestras se marcan, sino *cuáles*: la distribución por familia de la tabla anterior (Qakbot 40,4 % frente a Swisyn con media 0,321, la más baja) sí es información del modelo, porque refleja qué familias concentran el comportamiento marcado dentro del 10 % fijo.

Hallazgo principal: Qakbot tiene el 40,4 % de sus muestras marcadas como anómalas por comportamiento, consistente con un volumen de actividad (`file_ops`, `registry_ops`, `tcp_count`) sistemáticamente alto que distingue a esta familia del resto, en línea con su posición también más alta en `severity_v2` (§7.2).

Caso de uso del componente de anomalía: una muestra Swisyn (prueba de existencia)

Swisyn es, en promedio, la familia **menos anómala** (0,321, la más baja de las 10) y una de las de menor `severity_v2` (3,45, §7.2). Con 12.591 muestras, es la familia más grande del dataset; su media de anomalía es la más baja, y sin embargo **la muestra individual más anómala de las 48.976 pertenece a Swisyn** (posición #1 en el top-10 de muestras más anómalas; sha256 `8baed1a69a3e22c8…`, truncado por anonimización), con valores extremos en sus features de comportamiento y red:

Tabla 7.10: Caso Swisyn: features de la muestra atípica.

Feature	Valor de la muestra	Contexto
`file_ops`	446	muy por encima de la media de Swisyn
`registry_ops`	15.339	muy por encima de la media de Swisyn
`tcp_count`	3.244	muy por encima de la media de Swisyn

El motor de severidad (§7.2), basado en familia + tipo + TTPs, asignaría a esta muestra un `severity_v2` similar al resto de Swisyn (bajo). El motor de anomalía la identifica como atípica **dentro de su propia familia** por su comportamiento extremo. Este es, precisamente, el caso de uso para el que se diseñó el motor de anomalía (§5.5.1): detectar lo que la familia/tipo no captura. El efecto conjunto de esta señal sobre la urgencia final de esta muestra se observa en la cola priorizada (§7.5).

Alcance de este caso: se presenta como **prueba de existencia (n = 1)**, no como un patrón de la familia Swisyn, cuya media de anomalía es, de hecho, la más baja de las 10 familias (0,321 sobre 12.591 muestras). Lo que demuestra es que el mecanismo *puede* capturar un atípico intra-familia cuando

existe, incluso dentro de la familia con el perfil conductual más homogéneo del dataset; no que Swisyn como familia tienda a comportamiento anómalo, ni que el componente detecte sistemáticamente casos así. Una validación de la tasa real de detección de este tipo de casos requeriría atípicos conocidos de forma verificada, fuera del alcance de este corte.

Corrector de falsos positivos

Sobre las mismas 48.976 muestras (100 % malware confirmado por `public_labels.csv`), el corrector de FP (`fp_correction.py`, capítulo 5.5.2) marcó **0/48.976 muestras como benignas**, con sus listas de `imphash`/`sha256`/nombres de firma vacías y un único patrón placeholder que no llega al umbral de decisión (`benign_score ≥ 0,7`) por sí solo. Esto refleja la **neutralidad del hook con listas vacías**: el corrector no puede disparar y no produce falsos negativos sobre malware, pero no equivale a una garantía de precisión del detector. El recall real del corrector de FP solo podrá medirse cuando se integren muestras benignas (dataset de Ilić et al. [21], trabajo futuro, capítulo 8).

7.5 Tabla estrella 4: cola priorizada, distribución por bucket y `rank_shift`

La cola priorizada final (`scripts/build_priority_queue.py`, `reports/priority_queue_20260626_225446_avast.{csv,json,md}`) aplica la fusión completa (severidad × anomalía × corrección FP × contexto neutro, capítulo 5.6) a las 48.976 muestras y las ordena por `urgency_score` descendente, mostrando el `malscore` original de CAPE como columna comparativa.

Distribución por bucket

Tabla 7.11: Distribución de la cola priorizada por bucket.

Bucket	n	%
crítico (≥ 8,0)	364	0,7 %
alto (6,0–8,0)	22.562	46,1 %
medio (3,0–6,0)	22.201	45,3 %
bajo (< 3,0)	3.849	7,9 %

A modo de contraste: si se aplicaran los mismos cortes de bucket directamente sobre el `malscore`, el 56,28 % de las muestras (§7.2) caería en "crítico", produciendo una cola prácticamente plana e inservible para priorización. Con `urgency_score`, solo el 0,7 % (364 de 48.976) alcanza el bucket crítico, y la distribución entre alto/medio/bajo es razonablemente equilibrada: **la cola es accionable**.

El número exacto de casos críticos (364) depende del umbral de presentación (urgency ≥ 8,0) y de la magnitud de las constantes; el análisis de sensibilidad muestra que lo robusto es el ORDEN de la cola (Spearman ≥ 0,9887 bajo perturbaciones ±25 %), mientras que la línea de corte es una política de presentación configurable por el SOC (¿dónde se pone la raya?), no un hallazgo empírico.

Estadísticos globales

Tabla 7.12: Estadísticos globales: `urgency_score` frente a `malscore`.

	`urgency_score`	`malscore` (referencia)
Media	5,45	7,68
Mediana	5,91	10,0

	urgency_score	malscore (referencia)
Percentil 10	3,49	—
Percentil 90	6,70	—
% en el valor máximo	0,0 % (bucket crítico = 0,7 %)	56,28 %

Cola por familia

Tabla 7.13: Cola priorizada por familia.

Familia	n	urgency_score medio	malscore medio	severity_v2 medio
Qakbot	4.895	**7,06**	9,80	6,44
njRAT	3.372	6,59	7,38	6,04
Emotet	14.429	6,20	9,41	5,77
Ursnif	1.343	6,16	8,67	5,66
Trickbot	4.202	6,06	8,45	5,58
Lokibot	4.191	5,42	8,79	5,01
Zeus	2.594	5,36	9,25	4,95
Adload	704	4,09	7,05	3,82
Swisyn	12.591	3,67	4,20	3,45
HarHar	655	2,02	2,67	1,89

El orden por urgency_score sitúa a Qakbot y njRAT en cabeza (familias CISA con tipo banker/rat y comportamiento conductual elevado, §7.4) y a HarHar/Swisyn al final, sin pertenencia a CISA y sin TTPs de impacto. Es coherente con la combinación de severidad y anomalía de §7.2–§7.4.

Divergencias frente al orden por malscore (rank_shift)

El campo rank_shift mide cuántas posiciones cambia una muestra entre el orden por malscore y el orden por urgency_score. Las magnitudes escalan con el tamaño de la cola (con n=500 eran del orden de ±cientos; con n=48.976 son del orden de ±decenas de miles): lo que se compara entre escalas es la **estructura y la dirección**, no el valor absoluto. El cálculo usa **rango promedio** (rank(method="average")) para desempatar las muestras que comparten malscore = 10,0 (el 56,28 % del total) y así evitar la inflación artificial de caídas dentro del bloque saturado (build_priority_queue.py, donde rank_by_urgency_avg y rank_by_malscore_avg se calculan con rank(method="average")).

A escala n=48.976 (fuente: reports/cap07_evaluation_20260626_225610.{json,md}):

- **Top-5 promociones: todas njRAT**, con desplazamientos +37.231 / +35.696 / +35.302 / +34.846 / +34.766 posiciones (malscore de partida 3,0–4,2, fuera del bloque saturado). **njRAT domina la divergencia positiva**: su familia está en la lista de notoriedad (base CISA) y su tipo (rat) puntúa alto en la escala ordinal, y su comportamiento es conductualmente elevado, por lo que el motor les asigna una urgencia que los sitúa muy por delante de las decenas de miles de muestras Emotet/Trickbot con malscore=10,0.
- **Top-5 demociones extremas: Swisyn + Adload** con malscore=10,0 y conducta mínima (−21.791 / −21.698 / −21.674 / −21.636 / −21.613 posiciones). El patrón es el mismo que a escala 500: malscore=10,0 pero file_ops=0, tcp_count=0 y tipo trojan sin presencia en CISA → el motor las reclasifica en los buckets medio/bajo.

shift_mean por familia (fuente: scripts/build_cap07_evaluation.py):

Tabla 7.14: Divergencias `rank_shift` medias por familia.

Familia	`shift_mean` (n=48.976)	Dirección
njRAT	**+13.739**	promovida (la más)
Qakbot	+6.422	promovida
Ursnif	+4.024	promovida
Trickbot	+3.287	promovida
Emotet	+734	≈ neutra/promovida
Swisyn	**−2.568**	rebajada
Adload	−5.691	rebajada
HarHar	−5.862	rebajada
Lokibot	−8.924	rebajada
Zeus	**−11.561**	rebajada (la más, por media)

El **orden de familias rebajadas por media es idéntico** al observado a escala 500 (Zeus < Lokibot < HarHar < Adload < Swisyn), lo que confirma que la estructura bidireccional no es un artefacto del tamaño de muestra: escala con el dataset completo sin cambiar de signo en ninguna familia.

Correlación global y robustez

La correlación de Spearman entre `urgency_score` y `malscore_cape` sobre las 48.976 muestras es **ρ = 0,807** ($p \approx 0$): la cola coincide en lo grueso con el orden de CAPE (no es un reordenamiento caótico) y diverge en los márgenes, que es precisamente donde está la propuesta de valor del sistema (los casos njRAT/Swisyn anteriores). El Spearman sube de 0,792 (n=500) a 0,807 (n=48.976), lo que indica que el orden no se degrada con la distribución natural del dataset completo.

Un análisis de sensibilidad perturbó ±25 % cada constante del motor (incluido el coeficiente de anomalía) y recomputó la cola completa (n=48.976): el orden se mantiene con Spearman ≥ 0,9887 en todo el barrido y la retención del top-364 por rango nunca baja del 85,44 %. El ranking **no está ajustado a mano** para producir los casos njRAT/Swisyn. Como contrapartida, la perturbación del coeficiente de anomalía apenas mueve el orden global (Spearman 0,9997), lo que indica que, *para la ordenación global*, la anomalía aporta poco; su valor está en el flag a nivel de caso individual (§7.4, caso Swisyn), no en desplazar masas de muestras.

Qué demuestran (y qué no) estos casos

El 66 % de la severidad media de njRAT (4,0 de 6,08, §7.2/§7.3) procede de `type_part` + `cisa_part`; con señal puramente conductual (§7.3), njRAT caería de su posición media a posiciones intermedias. La promoción de njRAT **es el resultado de aplicar una política explícita**: el tipo `rat` y la presencia en CISA pesan más que un `malscore` moderado. No es un patrón descubierto por el modelo a partir del comportamiento observado.

En conjunto, estos casos son una **demostración de diseño con trazabilidad completa**: un sistema con una política explícita y auditable (tipo + notoriedad + comportamiento) produce un orden materialmente distinto al `malscore`, y cada cambio es trazable hasta sus features de entrada (capítulo 5). **No son una validación de exactitud**: no existe un *ground truth* de "urgencia verdadera" contra el que comprobar si +37.231 (njRAT) o −21.791 (Swisyn) son los desplazamientos "correctos"; esa validación queda para Gate 3 / validación con analistas SOC (§7.8, capítulo 8).

Validación cualitativa de la cola

Las secciones anteriores miden la cola de forma agregada: distribuciones, η^2, correlaciones y desplazamientos medios. Ninguna de ellas responde a la pregunta que un analista haría primero: *¿tiene sentido*

caso a caso? Una cola puede exhibir estadísticos impecables y colocar disparates en cabeza. Esta subsección documenta la inspección manual que se realizó para responder a esa pregunta, y es la primera validación cualitativa del sistema.

Método. Se inspeccionaron **64 casos** de la cola priorizada, correspondientes a **61 muestras únicas**, seleccionados para cubrir tanto los extremos como el cuerpo de la distribución: las promociones y demociones de mayor magnitud (los casos de §7.5 sobre los que descansa el argumento del sistema), muestras representativas de las diez familias, y casos situados en las fronteras entre buckets. Para cada caso se contrastó la posición asignada contra la descomposición completa del *score*: las cuatro componentes de `severity_v2` (`type_part`, `sig_part`, `ttp_part`, `cisa_part`), el `anomaly_score`, el multiplicador de fusión resultante y el `malscore` de CAPE como referencia. La inspección se apoyó en una reconstrucción independiente del motor a partir del código, que recomputó severidad y urgencia sobre las 48.976 filas y reprodujo el CSV de la cola al 100 % (cero discrepancias): la inspección juzga, por tanto, la **política** codificada en el motor, no un posible defecto de implementación.

Cada caso se clasificó en una de tres categorías:

- **Cuadra**: la posición en la cola y su justificación componente a componente son coherentes con el juicio que un analista SOC emitiría ante la misma evidencia. No exige que la posición sea óptima (eso no es verificable), sino que sea defendible y trazable.
- **Cuadra con salvedad**: la posición es defendible, pero su justificación descansa en una decisión de política discutible o insuficientemente documentada. El resultado no es erróneo; el criterio que lo produce es cuestionable.
- **Absurda**: ninguna lectura razonable de la evidencia justifica la posición asignada: la posición contradice los datos del informe.

Resultado: 57 cuadran / 7 con salvedad / 0 absurdas. El dato más relevante no es el 57, sino el **cero**: en 64 inspecciones no apareció ninguna posición que contradijera la evidencia del informe. Esto es lo máximo que la inspección puede afirmar, y conviene ser preciso sobre su alcance: acredita que la política del motor **no produce sinsentidos** y que su salida es auditable caso a caso; no acredita que el orden resultante sea el mejor posible.

Muestra ilustrativa. La tabla recoge cuatro casos ya documentados en esta memoria, a modo de ejemplo de cada lectura; **no es la inspección completa**, sino una selección representativa de los patrones encontrados:

Tabla 7.15: Validación cualitativa: muestra ilustrativa de casos inspeccionados.

Caso (documentado en §7.5)	Señal de partida	Desplazamiento	Lectura
njRAT, mayor promoción	`malscore` 3,0–4,2 (fuera del bloque saturado)	**+37.231** posiciones	**Cuadra**: familia en la lista de notoriedad, tipo `rat` alto en la escala y conducta elevada; el motor la saca de la mitad de la cola.
Swisyn, mayor democión	`malscore` 10,0 con `file_ops` = 0 y `tcp_count` = 0	**−21.791** posiciones	**Cuadra**: máxima puntuación del sandbox sin ninguna conducta observada; el motor la reclasifica al bloque medio/bajo.

Caso (documentado en §7.5)	Señal de partida	Desplazamiento	Lectura
HarHar (media de familia)	sobre-puntuada por el sandbox	`shift_mean` **−5.862**	**Cuadra**: la familia entera baja de forma consistente, sin cambio de signo entre escalas.
Zeus (media de familia)	*banker* real; una muestra con 30 firmas y 8 TTPs	`shift_mean` **−11.561** (la mayor bajada media)	**Salvedad**: el resultado es defendible, pero Zeus queda fuera de la lista de notoriedad mientras njRAT entra, y la pertenencia a esa lista no tiene un criterio escrito con precisión.

Los dos patrones de salvedad. Las siete salvedades no se reparten al azar: se concentran en dos decisiones de política que la inspección dejó al descubierto.

1. **Membresía de la lista de notoriedad sin criterio escrito** (el caso Zeus de la tabla). La lista de 13 familias parte del advisory CISA pero incorpora ampliaciones propias; njRAT está dentro y Zeus fuera sin que exista una regla documentada que explique la frontera. El efecto está acotado (el bonus vale como máximo 1,0 sobre 10, el 10 % de la escala) y el análisis de sensibilidad confirma que perturbarlo apenas mueve el orden global, pero el criterio de pertenencia es una decisión de autor que debe declararse como tal.
2. **Suelo de etiqueta.** Una muestra sin firmas y sin actividad observada no cae al fondo de la cola: conserva la puntuación que le otorgan su tipo y su notoriedad, y queda en el bloque medio. Es una política razonable ("desconocido no es inofensivo", coherente con la lectura adversarial de §8.4.7), pero no estaba declarada, y explica posiciones que a primera vista sorprenden.

Ambos patrones son decisiones de política, no defectos de cálculo, y ambos quedan declarados: el primero como salvedad del bonus de notoriedad (§5.4), el segundo como suelo de la escala (§5.4, conectado con la dependencia del campo `type` de §8.4.2).

La salvedad, y por qué es la conclusión correcta. Esta inspección no es un experimento ciego: la realizó el autor del sistema, con protocolo escrito y sobre una cola reproducible, pero juez y parte. Y, sobre todo, **no existe un *ground truth* de "urgencia verdadera"**: nadie dispone de la lista de qué debió atenderse primero, de modo que no hay contra qué medir la exactitud del orden. Lo que esta validación sostiene, y es exactamente lo que el sistema promete, es que el orden es **accionable, explicable y coherente**, no demostrablemente **óptimo**. El paso metodológico siguiente está identificado: una evaluación con analistas externos sobre 30-50 fichas mediante rúbrica MESSALA (§8.5.3), que sustituiría la plausibilidad por un juicio independiente.

Esta subsección es la contrapartida cualitativa del argumento cuantitativo del capítulo: el `malscore` satura al 56,28 % de las muestras en 10,0 y, bucketizado con los mismos umbrales, declararía críticos 29.809 casos (60,9 %) frente a los 364 (0,74 %) del sistema: un factor 82 veces menor de carga crítica (§7.2, §7.5). La inspección caso a caso es lo que permite afirmar que esa reducción **no se logra descartando arbitrariamente**, sino aplicando una política que, revisada una a una, no produce sinsentidos. Ambas mitades responden juntas a la objeción de fondo (*¿por qué recalcular un* `malscore` *que el sandbox ya entrega?*): porque ese `malscore` mide evidencia de malicia, no urgencia de atención, y las dos divergen precisamente dentro del bloque saturado, que es donde el analista necesita ayuda para decidir.

Nota de reproducibilidad: los ficheros `reports/priority_queue_20260626_225446_avast.{csv,json,md}` y `reports/cap07_evaluation_20260626_225610.{json,md}` son *gitignored* y no se conservan en el repositorio. Los `shift_mean` por familia y el Spearman ρ=0,807 proceden del informe que genera `scripts/build_cap07_evaluation.py`. La descomposición η² (tabla de §7.3) es reproducible con `scripts/eta_decomposition.py` (script versionado en el repositorio); su resultado completo se conserva en `docs/evidencias/eta_decomposition/eta_decomposition_20260717_001821_avast.{json,md}`. Para regenerar la cola: `python scripts/build_priority_queue.py`; para el consolidado: `python scripts/build_cap07_evaluation.py`. La inspección caso a caso de la validación cualitativa es reproducible sobre ese mismo `priority_queue_*.csv`, que contiene por fila, entre otras columnas, `rank`, `sha256`, `family`, `type`, `severity_v2`, `anomaly_score`, `fp_benign_score`, `urgency_score`, `bucket`, `malscore_cape` de referencia, `rank_divergence` y `rank_shift`; el desglose en las cuatro componentes de la severidad no se persiste en el CSV y se obtiene reejecutando `severity_engine.compute_severity` sobre la fila, que es como se generó en la inspección.

7.6 Tabla estrella 5: fidelidad de la síntesis SecOps (plantilla vs LLM)

La síntesis SecOps (`secops_summary.py`, capítulo 5.7) se evaluó con `scripts/run_secops_summary.py` sobre el **top-2 de la cola priorizada** (las dos muestras Qakbot de mayor `urgency_score`), comparando los tres modos disponibles: plantilla determinista, `phi4-mini:latest` y `llama3.2:3b` (ambos vía Ollama local, `localhost:11434`, CPU Ryzen 5 5600). Fuente: `reports/secops_summary_comparison_20260603_202803.{json,md}`.

Tabla 7.16: Fidelidad de la síntesis SecOps: plantilla frente a LLM.

Modo	n	latencia media	latencia máx.	`payload_fidelity` media
plantilla	2	— (instantánea)	—	**0,60**
`phi4-mini:latest`	2	35,78 s	42,52 s	0,40
`llama3.2:3b`	2	31,98 s	36,29 s	0,20

`payload_fidelity` es la proporción de 5 comprobaciones automáticas superadas (menciona la familia, menciona el valor numérico de urgencia, menciona el bucket, menciona al menos una TTP, incluye todos los campos del contrato; capítulo 5.7). Estas cifras corresponden al corte del 3 de junio de 2026 (`secops_summary_comparison_20260603_202803`) y no se han vuelto a medir; se reportan con fecha para no presentarlas como resultado vigente.

Sesgo del evaluador automático: la comprobación «menciona el valor numérico de urgencia» busca el número formateado con separador decimal de punto: el código construye `f"{urgency_score:.1f}"`, p. ej. `"6.4"`, y comprueba su presencia literal en el texto (`secops_summary.py:298-299`). Un modelo que redacte «6,4», separador decimal correcto en español, falla ese `check` aunque el valor sea exacto (queda un segundo intento por el nombre del bucket, no por el número). Parte de la diferencia de fidelidad entre la plantilla y los LLMs de esta tabla es, por tanto, atribuible al propio evaluador, anglocéntrico en el formato numérico, y no a una carencia del modelo. Es un defecto corregible del medidor, declarado aquí para no sobreatribuir el gap a los LLMs.

Sesgo estructural de la métrica: esta comparación no enfrenta a tres generadores en igualdad de condiciones. La plantilla **inserta directamente** los valores de familia, `urgency_score` y bucket en el texto (son interpolación de variables, no generación), por lo que las tres primeras de las 5 comprobaciones las supera **por construcción**, con independencia de cuán informativo sea el resultado. Los LLMs, en cambio, deben *redactar* esos mismos datos a partir del payload, lo que es una tarea estrictamente más difícil. La tabla, por tanto, no compara "tres generadores de texto": compara un sistema que copia

campos de un contrato con dos que narran un contrato. Esto no invalida la medición (usar la plantilla por defecto, más abajo, sigue siendo una decisión razonable), pero sí invalida leer la tabla como "la plantilla es un generador mejor que los LLMs".

Lectura de los resultados:

- La **plantilla** alcanza 0,60 (3/5) **por construcción**: siempre menciona urgencia y bucket exactos porque los inserta literalmente, pero no narra TTPs de forma libre. Su ventaja en esta métrica es la ventaja de diseño descrita arriba, no una propiedad emergente de "redactar mejor".
- **`phi4-mini`** alcanza 0,40 (2/5): menciona familia y bucket, pero no reproduce el valor numérico exacto de urgencia. Se detectó además un **modo de fallo** concreto: en una respuesta, el modelo colocó códigos `T####` (TTPs MITRE) en el campo `iocs` (que debería contener dominios/IPs), aislado por el normalizador de `iocs` del propio componente sin degradar el resto de la ficha.
- **`llama3.2:3b`** alcanza 0,20 (1/5): menciona la familia pero no urgencia ni bucket exactos. Otro **modo de fallo**: el modelo narra en términos generales sin anclar los valores numéricos del contrato. Es un ~10 % más rápido que `phi4-mini` en esta CPU.
- En cuanto a naturalidad del español generado, ambos LLMs producen prosa fluida; `llama3.2:3b` tiende a abstracciones generales, `phi4-mini` es más técnico y conciso.

Limitación de cableado de la sección de IOCs. En el corte batch evaluado, la lista de IOCs de red que recibe la síntesis está vacía. El parser sí extrae `unique_ips`/`unique_domains` anonimizados en `CapeFeatures` (§6.1), pero la pérdida ocurre **aguas arriba, en la ingesta**: el dataset tabular que produce `ingest_avast_ctu.py` solo persiste los conteos agregados (`hosts_count`, `domains_count`) y descarta las listas de indicadores. La cola (`build_priority_queue.py`) se construye sobre ese dataset, de modo que nunca llegan a ella las listas que podría propagar; la ficha de detalle (`triage_detail.html`) muestra «(sin IOCs registrados)» y el modelo recibe un campo `iocs` vacío. Esto tiene dos consecuencias: (1) la sección de IOCs de la ficha web **no exhibe indicadores de red en este corte** (limitación de cableado en el ingestor, corregible sin tocar los motores), y (2) parte de la confusión de campos de `phi4-mini` (colocar TTPs en `iocs`) es plausiblemente un artefacto de ese hueco: un campo vacío invita a rellenarlo con lo que el modelo tenga a mano, no solo un fallo del modelo. La corrección (persistir las listas anonimizadas en la cola) queda declarada como trabajo de cableado (§8.5).

Interpretación para H3: con n = 2 y la asimetría estructural descrita arriba, estos resultados **no se interpretan como "la plantilla es mejor generador que los LLMs"** (esa comparación sería injusta por diseño). Lo que el experimento documenta de forma útil son los **modos de fallo concretos de LLMs locales pequeños (3-4B) en CPU** al redactar un contrato técnico: omisión de valores numéricos exactos (`llama3.2:3b`) y confusión de campos del esquema (`phi4-mini`, TTPs en `iocs`). Esa información sí es trasladable a trabajo futuro (post-procesado/validación de la salida del LLM antes de mostrarla). La rúbrica humana MESSALA sobre una muestra mayor (30-50 fichas, con un diseño que no favorezca estructuralmente a ningún modo) queda como trabajo pendiente (capítulo 8) para una evaluación robusta de H3.

Con esta evidencia, el diseño (Fase F, capítulo 5.8) fija la plantilla determinista como modo por defecto del MVP (`DEFAULT_SUMMARY_MODE = "template"`) tanto en el orquestador `triage_service.py` como en `/api/v2/triage` y la UI Atalaya, dejando el modo LLM (`mode="llm"`) como parámetro explícito para experimentación. Esta decisión **no depende de que la plantilla "redacte mejor"**: depende de que es determinista, instantánea (no requiere Ollama) y garantiza por construcción que los campos críticos del contrato (familia, urgencia, bucket) aparecen siempre: propiedades operativas relevantes para un MVP, independientes del sesgo de la métrica de fidelidad discutido arriba.

7.7 Amenazas a la validez

7.7.1 Validez interna

Sin partición train/test para el motor de anomalía. El `IsolationForest` se ajustó y evaluó sobre las mismas 48.976 muestras (`anomaly_v2.joblib`, capítulo 5.5.1). Los porcentajes de "muestras flagged" de §7.4 describen el comportamiento del modelo sobre sus propios datos de entrenamiento, no su capacidad de generalización a muestras nuevas. Para los motores de severidad y fusión (deterministas, sin entrenamiento) esta amenaza no aplica: son funciones evaluadas, no modelos ajustados.

`payload_fidelity` sobre n = 2. El experimento de §7.6 cubre únicamente las dos muestras de mayor urgencia (ambas Qakbot). No se puede generalizar la comparación plantilla/LLM a otras familias o buckets de urgencia sin una muestra mayor.

7.7.2 Validez externa

Brecha de esquema CAPE 2.2 / 2.5. El dataset Avast-CTU usa informes generados con versiones más antiguas de CAPE (2.2), mientras que la VM de análisis propia generaría informes 2.5. El parser v2 (capítulo 5.3) maneja ambas variantes y fue validado contra 11 informes reales 2.5, pero los 48.976 informes evaluados en este capítulo son todos 2.2. Campos exclusivos de uno u otro esquema (`detections`/`malfamily_tag` solo en Avast; `mbcs`/`categories` más completos en 2.5) pueden tener cobertura distinta en datos propios.

Ausencia de tráfico benigno. Las 48.976 muestras son 100 % malware confirmado. El corrector de FP (§7.4) no ha sido evaluado contra ningún binario benigno real; su "0 falsos benignos" es una garantía de no introducir ruido sobre malware, no una medida de su utilidad cuando existan binarios benignos en la cola.

Hardware único para latencias de LLM. Las latencias de §7.6 (31-43 s por ficha) se midieron en una única máquina (Ryzen 5 5600, sin GPU). En hardware con GPU o con modelos más pequeños/cuantizados, las latencias, y posiblemente la fidelidad, variarían.

7.7.3 Constructo

Circularidad `type` → familia en la métrica de η^2 inter-familia (identificada en la revisión metodológica del 12 de junio de 2026). En Avast-CTU cada familia tiene esencialmente un tipo (`type_ordinal`), y `severity_v2` incluye `type_part` y `cisa_part` (ambos derivados de familia/tipo) como dos de sus cuatro componentes. El η^2 agregado inter-familia (`severity_v2` = 0,833 vs `malscore` = 0,510, §7.3) mide en buena parte la presencia de la propia etiqueta dentro de la métrica, no una capacidad de discriminación conductual independiente. §7.3 presenta la descomposición que aísla este efecto (η^2 del componente `type_part` = 0,9916; señal conductual pura = 0,477, por debajo del `malscore` = 0,510). La descomposición es reproducible con `scripts/eta_decomposition.py` (versionado). Esta amenaza se considera **mitigada por reformulación**: la sección ya no presenta el η^2 agregado como evidencia de "mejor discriminación conductual", sino que separa explícitamente la contribución de la política (tipo + CISA) de la señal conductual.

η^2 como métrica de discriminación, no de exactitud. Incluso la descomposición de §7.3 mide cuánta varianza del score es explicada por la familia/tipo, pero no implica que `severity_v2` sea "correcto" en un sentido absoluto: una familia podría tener una severidad real distinta de la que el motor le asigna, y η^2 no lo detectaría si el motor es internamente consistente. La validación de que la escala 0-10 de `severity_v2` corresponde a percepciones reales de un analista SOC requeriría una evaluación humana (rúbrica MESSALA, pendiente, igual que para H3).

`payload_fidelity` es una métrica débil por diseño, y además estructuralmente sesgada a favor de la plantilla. Las 5 comprobaciones son automáticas (basadas en coincidencia de patrones/regex sobre el

texto generado) y miden presencia de elementos, no corrección semántica: una ficha podría mencionar el bucket correcto y aun así describir mal la acción recomendada. Además, como se detalla en §7.6, la plantilla inserta los campos del contrato por construcción, por lo que supera 3 de las 5 comprobaciones por diseño y no por "redactar mejor": la tabla de §7.6 documenta modos de fallo de LLMs pequeños, no una comparación de generadores en igualdad de condiciones. Es un proxy útil para CI, no un sustituto de evaluación humana.

Ausencia de *ground truth* de "urgencia verdadera". El `rank_shift` de §7.5 (njRAT top +37.231, Swisyn top −21.791) demuestra que el sistema reordena de forma trazable respecto al `malscore`, pero no existe ninguna fuente, ni en este dataset ni en `public_labels.csv`, que indique cuál sería el orden "correcto" de urgencia para las 48.976 muestras. Por tanto §7.5 se interpreta como demostración de una política explícita y auditable, no como validación de que esa política produce el orden óptimo; esa validación requiere Gate 3 / juicio de analistas SOC (§7.8).

7.7.4 Dos defectos de calibración conocidos en el motor de severidad

La revisión del motor identificó dos defectos reales, no corregidos en este corte y declarados aquí como limitación:

1. **Rampa de pesos infrautilizada.** El subcomponente de firmas normaliza la suma de pesos con una rampa lineal `min(sum_weight/200, 1)`. El percentil 95 real del dataset es ≈ 34 y ninguna de las 48.976 muestras alcanza 200: el subcomponente opera en el rango 0–0,17 de su escala 0–1. El denominador se fijó por política, sin contrastar con la distribución empírica.
2. **TTPs deprecadas mapeadas a peso mínimo.** El 12,2 % de las TTPs del dataset usan identificadores ATT&CK anteriores a la consolidación de la matriz (T1081, T1089, T1063, T1012, T1031, propios de CAPE 2.2) que el mapa técnica→táctica no contempla y caen a `_unknown` (peso 0,1), infraponderando señal real de credential_access, defense_evasion, discovery y persistence. El mapa sí contempla otros dos identificadores deprecados (T1060, T1045), lo que confirma que se trata de una omisión de curación y no de una decisión de diseño.

El análisis de sensibilidad (§7.5/anexo) acota su impacto: corrigiendo ambos defectos, el orden de la cola se mantiene (Spearman ≥ 0,9887; retención del top-364 por rango ≥ 85,44 %). Las conclusiones del capítulo no dependen de ellos, pero el motor los hereda tal cual y su corrección queda para la siguiente iteración (§8.5).

7.8 Validación pendiente: informes CAPE 2.5 reales (Gate 3)

El diseño v2 establece que **el desarrollo y la evaluación principal de v2 no dependen de Gate 3**: se realizan íntegramente offline con Avast-CTU, como se ha presentado en este capítulo. Gate 3 (réplica del entorno CAPEv2, capítulo 5.9) aporta una validación adicional con muestras propias (EICAR + binarios de MalwareBazaar) si se completa antes del cierre del TFG.

A fecha de este capítulo, Gate 3 **sigue pendiente por una limitación de hardware (ver §8.5.1)**. El parser v2 ya está validado contra 11 informes CAPE 2.5 reales del paquete `_tmp/CAPE_GOLD_EXPORT_20260417/` (ver §7.7.2), lo que reduce el riesgo de incompatibilidad de esquema cuando Gate 3 se desbloquee.

Si Gate 3 se cierra antes de la entrega: este capítulo se ampliará con una sección que repita las tablas estrella §7.2–§7.6 sobre el lote de informes CAPE 2.5 reales, señalando explícitamente las diferencias con los resultados de Avast-CTU (n mucho menor, ausencia de etiqueta de familia externa para EICAR/MalwareBazaar). Si no se cierra, esta sección queda documentada como **trabajo futuro** en el capítulo 8, con el protocolo de transferencia ya escrito (`tools/transfer/`).

7.9 Evaluación del prototipo v1 (histórico, contexto del pivote)

Esta sección resume, por transparencia metodológica, la evaluación del prototipo **anterior al pivote**, que fue la evaluación vigente del TFG hasta el 31 de mayo de 2026.

7.9.1 Resultados obtenidos (v1)

El prototipo v1 era un **clasificador supervisado** (`RandomForestClassifier` + `IsolationForest`, fusión aditiva `risk_score = 0,55·sup + 0,25·anom + 0,20·ctx`) evaluado sobre un dataset **sintético** generado por `seed_demo_data.py`. Los resultados reportados (`reports/evaluation_20260502_095250.json`, n_test = 18) eran:

Tabla 7.17: Resultados del prototipo v1 (histórico).

Métrica	Valor
F1	1,000
FP-rate	0,000
ROC-AUC	1,000

Estos valores cumplían formalmente los criterios de aceptación originales del TFG (F1 ≥ 0,80, FP-rate ≤ 10 %), y el propio sistema los acompañaba de una bandera de auto-diagnóstico: `analysis.suspicion_flags.is_suspiciously_perfect_small_dataset: true`.

7.9.2 Por qué se descarta como resultado principal

La auditoría funcional del 31 de mayo de 2026 identificó tres defectos de raíz:

1. **Leakage circular**: el `label` del dataset se derivaba del `cape_score` (`cape_parser.py`), que era a su vez una *feature* del modelo: el modelo aprendía, en parte, a predecir una transformación de su propia entrada.
2. **Separabilidad artificial del generador sintético**: `seed_demo_data.py` producía datos perfectamente separables independientemente del nivel de dificultad configurado (confirmado posteriormente con n = 500 y n = 1.000 y `difficulty=hard`, F1 = 1,0 y FP = 0,0 en las 4 configuraciones probadas).
3. **Explicabilidad por magnitud**: el `top_factors` se calculaba como `valor × importancia` sin normalizar, lo que mezclaba la escala de la *feature* con su peso en el modelo y producía explicaciones poco interpretables.

El primer defecto invalida F1 = 1,0 como evidencia de capacidad predictiva: un clasificador que en parte predice una función de su propia entrada no está resolviendo el problema que pretende resolver. El segundo defecto invalida la robustez de ese F1 incluso si el leakage no existiera. Ante esta evidencia, se decidió el pivote a v2: en lugar de re-clasificar malware (tarea que CAPE ya resuelve, y que el dataset disponible no permite evaluar sin leakage), el sistema **re-valora el veredicto de CAPE con señales externas**, que es el problema evaluado en §7.1–§7.6 de este capítulo.

7.9.3 Qué se conserva de la evaluación v1

El ejercicio de ablación de pesos de v1 (grid search sobre combinaciones `α·sup + β·anom + γ·ctx`) dejó un hallazgo metodológico que sigue siendo relevante para la fusión de v2 (capítulo 5.6), aunque la fórmula haya cambiado de aditiva a multiplicativa: la configuración `anom_only` (solo `IsolationForest`, sin componente supervisado/determinista) obtenía F1 = 0,413, muy por debajo del resto, y `ctx_only` (solo heurísticas contextuales) producía FP-rate = 0,143, muy por encima del umbral. Ambos resultados respaldan la decisión de diseño, mantenida en v2, de que **ninguna señal individual basta por sí sola**:

la severidad intrínseca, la anomalía conductual y la corrección de FP deben combinarse (capítulo 5.6) para producir una urgencia útil.

La infraestructura de v1 que se reutiliza en v2 (FastAPI, `anonymizer.py`, `quality.py`, `storage.py`, estructura de tests) se describe en el capítulo 5 (§5.8, §5.10) y no se repite aquí.

Capítulo 8. Conclusiones y trabajo futuro

8.1 Resumen de aportaciones

Este trabajo ha diseñado, implementado y evaluado un **asistente de triaje post-sandbox** para centros de operaciones de seguridad (SOC), centrado en el problema de priorización: dado un informe de análisis dinámico de CAPEv2, ¿con qué urgencia debe atenderse ese hallazgo y cómo resumirlo en una ficha accionable para el analista sin que este tenga que leer el JSON técnico?

La contribución principal no es un clasificador de aprendizaje automático, sino un **motor de decisión determinista y auditable** que re-valora el veredicto del sandbox (`malscore`) con señales independientes de él. El núcleo codifica política SOC explícita (tipología de amenaza, notoriedad en listas CISA e impacto de técnicas MITRE ATT&CK) en una función con componentes acotados y trazables desde el número final hasta sus entradas. El aprendizaje automático actúa como señal **auxiliar y exploratoria**: la anomalía conductual (IsolationForest) detecta lo que la política no cubre, y el LLM local genera la síntesis narrativa, pero ninguno de los dos determina la urgencia.

El sistema se organiza en seis componentes con contratos verificables:

El **parser CAPE multi-schema** (`cape_parser_v2.py`) unifica la ingesta de informes CAPE 2.2 (Avast-CTU) y 2.5 (VM propia), con anonimización obligatoria y garantía por contrato de que ningún campo (familia, tipo ni etiqueta) se deriva del `malscore`. Validado sobre 3 informes Avast-CTU reales y 3 informes CAPE 2.5 reales.

El **motor de severidad intrínseca** (`severity_engine.py`) es una función determinista con cuatro componentes acotados (tipo de amenaza 0–3,5; peso de firmas CAPE 0–3,0; técnicas MITRE ATT&CK 0–2,5; bonus CISA AA22-216A 0–1,0) que produce una puntuación 0-10 sin ningún acceso al `malscore`. Cada punto de la escala es auditable componente a componente.

En el **motor de anomalía conductual** (`anomaly_engine.py`) y el **corrector de falsos positivos** (`fp_correction.py`), el IsolationForest detecta comportamiento atípico intra-familia sobre 14 features de comportamiento/red/estático, excluidas explícitamente de familia y `malscore`. El corrector expone una interfaz extensible para reducir la urgencia de binarios benignos conocidos; sus listas están vacías en este corte, listas para poblarse con un dataset de benignos (§8.5.2).

El **motor de fusión de urgencia** (`urgency_fusion.py`) combina las tres señales anteriores en una puntuación única 0-10 con fórmula multiplicativa auditable y cuatro buckets categóricos (crítico / alto / medio / bajo). En el MVP offline, la fórmula efectiva es `urgency ≈ severity_v2 × (1 + 0,20 × anomaly_score)`, dado que `fp_penalty` y `context_multiplier` son ganchos de producción neutros en este corte (listas de FP vacías; sin fuente de contexto multiseñal conectada).

El **componente de síntesis SecOps** (`secops_summary.py` y `ollama_client.py`) genera una ficha narrativa en español {qué hizo / por qué importa / IOCs / acción recomendada} con interfaz dual: plantilla determinista (sin dependencias externas, fidelidad automática 0,60, modo por defecto) y LLM local

vía Ollama (phi4-mini o llama3.2:3b, con `DEFENSIVE_SOC_SYSTEM_PROMPT` para evitar rechazos del modelo en tareas de análisis de malware). Degradación automática al modo plantilla si Ollama no está disponible.

La **API REST y la UI Atalaya** (`routes_triage.py`, Jinja2) exponen cuatro endpoints v2 y una cola priorizada por `urgency_score` con el `malscore` original de CAPE como columna comparativa, ficha detallada con desglose de la fórmula de fusión e indicador de `rank_shift`. Coexisten con el dashboard v1 sin modificarlo.

El **anti-leakage por contrato** es una garantía estructural, no solo empírica: ningún componente del pipeline v2 acepta `malscore`/`cape_score`/`score` como argumento; verificado por 13 tests introspectivos `TestNoLeakage` distribuidos en los 6 módulos del pipeline.

El MVP está cerrado: 225 tests automatizados (53 baseline v1 + 172 v2 específicos, Fases A-F del pivote), `demo.ps1` 21/21 PASS sin regresión sobre v1, con el MVP cerrado en junio de 2026.

8.2 Cumplimiento de objetivos específicos

Tabla 8.1: Cumplimiento de los objetivos específicos.

OE	Objetivo	Estado	Evidencia / Bloqueo
OE1	Parser sin leakage; dataset ingestado con etiquetas de fuente externa	Cumplido	`test_avast_no_label_derivation` verde; `public_labels.csv` como fuente externa; 48.976 muestras Avast-CTU ingestadas (versión `20260626_225446`)
OE2	Motor de severidad determinista y auditable	Cumplido	`test_severity_engine.py` (34 tests, `TestNoLeakage` verde); η^2(severity_v2) = 0,833 > η^2(malscore) = 0,510; descomposición en §7.3
OE3	Motor de anomalía (IsolationForest, 14 features) + corrector de FP	Cumplido	`anomaly_v2.joblib` entrenado y persistido; 0/48.976 falsos benignos sobre dataset 100 % malware (garantía de neutralidad: no introduce ruido; la precisión real no es medible sin benignos, §8.4.4); `TestNoLeakage` verde en ambos módulos
OE4	Motor de fusión multiplicativo 0-10 con buckets categóricos	Cumplido	`test_urgency_fusion.py` (26 tests, monotonicidad, caps, `TestNoLeakage`); distribución: 0,7 % crítico / 46,1 % alto / 45,3 % medio / 7,9 % bajo (cumple el ≤ 5 % en crítico)
OE5	Síntesis SecOps con interfaz dual y degradación automática	Cumplido	31 tests (`test_secops_summary.py` + `test_ollama_client.py`); fidelidad plantilla 0,60 ≥ umbral (corte jun. 2026; §7.6 matiza el sesgo del evaluador); modo auto degradado testeado (`TestAutoGraceful`); `TestAntiRefusalSystemPrompt` verde
OE6	Evaluación cuantitativa frente al `malscore` bruto	Cumplido	Tablas η^2 en §7.3 (`scripts/eta_decomposition.py`, reproducible), `rank_shift` en §7.5 (njRAT top +37.231, Swisyn top −21.791; Spearman ρ = 0,807), fidelidad en §7.6; n=48.976, distribución no plana verificada

OE	Objetivo	Estado	Evidencia / Bloqueo
OE7	API REST + UI Atalaya con trazabilidad completa	Cumplido	4 endpoints v2 + UI (`/ui/triage`, `/ui/triage/{rank}`); desglose de fusión en ficha; `demo.ps1` 21/21 PASS; 225 tests al cierre del MVP v2

8.3 Respuesta a las preguntas de investigación

RQ1. ¿Produce la re-valoración contextual una priorización más discriminativa que el `malscore` bruto?

Respuesta con matiz necesario: la re-valoración produce una priorización **estructuralmente más útil**, aunque la señal conductual de `severity_v2` ($\eta^2 = 0{,}477$) queda ligeramente por debajo del `malscore` (0,510) como discriminador inter-familia (§7.3, descomposición reproducible con `scripts/eta_decomposition.py`). Este resultado es deliberado y se interpreta correctamente: la ventaja del sistema no es conductual sino estructural. El `malscore` de CAPE satura al 56,28 % de las 48.976 muestras en su valor máximo (§7.2), produciendo una cola plana donde el 56,28 % de alertas de 7 familias distintas reciben la misma urgencia. El motor v2 descomprime ese bloque con una política explícita (tipo + CISA + TTPs MITRE), generando un rango 1,7–6,4 donde cada posición es justificable componente a componente.

H1 se reformula: el motor v2 distribuye más que el `malscore` porque incorpora contexto externo (tipo, CISA) que el sandbox no usa, no porque "entienda mejor" el comportamiento observado. Esa ventaja es legítima y está documentada, pero debe leerse como ingeniería de política SOC, no como superioridad predictiva conductual.

RQ2. ¿Reduce el sistema el rango de los falsos positivos conocidos?

Respuesta con alcance acotado: el motor de fusión reordena muestras de bajo impacto real (Swisyn, HarHar) hacia posiciones más bajas y muestras de alto impacto (njRAT) hacia posiciones más altas respecto al orden por `malscore` (§7.5). Las divergencias son trazables: njRAT sube porque su familia está en la lista de notoriedad (base CISA), su tipo (`rat`) puntúa alto en la escala ordinal y su comportamiento es atípico; Swisyn baja porque su familia no está en la lista de notoriedad, su tipo (`trojan`) puntúa medio y su comportamiento es mínimo.

H2 se confirma en los casos documentados. El componente específico de corrección de FP (`fp_correction.py`) tiene recall desconocido en este corte: con sus listas vacías produce `benign_score = 0` en las 48.976 muestras (100 % malware confirmado). La evaluación de su recall real requiere binarios benignos reales (trabajo futuro, §8.5).

RQ3. ¿Es el resumen generado completo, preciso y accionable?

Respuesta empírica (n = 2, muestras Qakbot de mayor urgencia): la plantilla determinista alcanza `payload_fidelity` = 0,60 (3/5 comprobaciones, por construcción: inserta familia, urgencia y bucket directamente). Los LLMs locales de 3-4B parámetros en CPU producen texto fluido pero omiten valores numéricos exactos (`llama3.2:3b`, 0,20) o confunden campos del esquema TTPs/IOCs (`phi4-mini`, 0,40). Los modos de fallo documentados en §7.6 son trasladables a trabajo futuro (post-procesado de salida LLM, modelos más grandes o con aceleración GPU).

H3 se confirma para el umbral de plantilla ≥ 0,60. La evaluación robusta de accionabilidad (rúbrica MESSALA sobre 30-50 fichas de familias y buckets variados) queda como trabajo futuro (§8.5).

8.4 Limitaciones

Las limitaciones siguientes no invalidan los resultados presentados, sino que definen su alcance real y deben comunicarse para que puedan interpretarse correctamente.

8.4.1 Dataset único: una sola fuente (Avast-CTU)

Todas las cifras de evaluación proceden del dataset completo Avast-CTU (48.976 informes, 10 familias), lo que elimina la limitación de muestra pequeña presente en versiones anteriores de este capítulo. Sin embargo, el dataset sigue siendo **una sola fuente de un único laboratorio de sandbox**: la distribución de familias refleja la composición de Avast-CTU (dominada por Emotet 14.429 y Swisyn 12.591), no necesariamente la de un SOC real. Los resultados de §7.2–§7.5 son válidos para este dataset; su generalización a otras familias emergentes, otros entornos de sandbox o distribuciones de amenaza distintas requeriría fuentes adicionales (p. ej. informes CAPE propios de Gate 3, u otras colecciones públicas de sandbox).

8.4.2 Dependencia del campo `type` como entrada

La componente `type_part`, responsable de hasta 3,5 de los 10 puntos de `severity_v2`, depende del campo `type` (tipo de malware: banker, rat, trojan, etc.) proveniente de `public_labels.csv` en Avast-CTU o de un parámetro externo al report en el flujo en vivo. Si CAPE 2.5 no etiqueta el tipo de la muestra, o lo etiqueta erróneamente, el motor asigna el valor mínimo (desconocido = 0,5 sobre 3,5) y subestimará amenazas que deberían puntuar alto. Esta dependencia es una amenaza real en producción ante familias emergentes o especímenes polimórficos; su alcance concreto solo puede evaluarse cuando CAPE 2.5 esté operativo con datos propios.

8.4.3 Motor de anomalía sin partición train/test

El IsolationForest se ajustó y evaluó sobre las mismas 48.976 muestras. Los porcentajes de muestras flagged (§7.4) describen el comportamiento del modelo sobre sus propios datos de entrenamiento, no su capacidad de generalización a muestras nuevas. El total de 4.898 muestras flagged (10 %) coincide exactamente con el hiperparámetro `contamination=0,10`: no es un descubrimiento estadístico sino una consecuencia de configuración. Lo informativo es qué familias concentran esas 4.898 detecciones, no cuántas.

8.4.4 Corrector de FP sin recall evaluado

Las listas de `fp_correction.py` están vacías en este corte. La medición de precisión (0/48.976 falsos benignos sobre dataset 100 % malware) es correcta, pero el recall (la capacidad de detectar binarios benignos reales sobre-puntuados por el sandbox) no puede medirse hasta que se integren datos benignos.

8.4.5 Evaluación de síntesis sobre n = 2 y con sesgo estructural

El experimento de §7.6 cubre 2 muestras Qakbot. La plantilla supera 3 de las 5 comprobaciones por diseño (inserta directamente los campos del contrato), no por "redactar mejor": la tabla de `payload_fidelity` documenta modos de fallo de LLMs pequeños en CPU, no una comparativa en igualdad de condiciones.

8.4.6 Validación con CAPE 2.5 en vivo no realizada

El sistema fue diseñado y evaluado offline (Avast-CTU). La máquina de análisis dinámico CAPE 2.5 (una máquina local del proyecto) estuvo inoperativa durante el período de desarrollo por un problema de procesador (indefinido a fecha de cierre del MVP, 2026-06-08). La validación E2E con informes CAPE 2.5 generados en vivo queda como trabajo futuro declarado (§8.5).

8.4.7 Lectura adversarial de las muestras rebajadas

Las muestras que el sistema rebaja frente al `malscore` comparten firma con un evasor de sandbox: `malscore` alto y comportamiento observado nulo. Desde el informe del sandbox no es posible distinguir «no observado» de «inofensivo», de modo que una muestra que evada activamente el análisis dinámico podría recibir menor urgencia. Tres factores acotan el riesgo: el sistema rebaja pero no descarta (la muestra permanece en la cola con su ficha), la anomalía intra-familia empuja en sentido contrario ante comportamiento atípicamente bajo, y la defensa anti-evasión corresponde al propio sandbox aguas arriba. Aun así, la lectura adversarial se declara aquí como limitación de constructo: un veredicto de urgencia basado en comportamiento observado hereda la ceguera del sandbox ante la evasión.

8.5 Trabajo futuro

8.5.1 Validación con CAPE 2.5 en vivo (trabajo futuro declarado)

La máquina de análisis CAPE local del proyecto está parada por un problema de procesador de carácter indefinido. La validación E2E en vivo (enviar una muestra a CAPE, obtener el `report.json` y procesarlo con el pipeline v2) queda documentada como **trabajo futuro**, no como elemento pendiente del presente TFG.

Lo que ya está implementado y validado: el parser v2 fue comprobado contra 11 informes CAPE 2.5 reales obtenidos en la VM de empresa en abril (`_tmp/CAPE_GOLD_EXPORT_20260417/`), con tests específicos en `TestVMCape25Real` (4 tests, parser) y en los motores de severidad, anomalía y fusión (11 informes, tests `TestSeverityOnVMRealReports`, `TestAnomalyOnVMReports`, `TestEndToEndOnVMReports`). Cuando CAPE esté operativo, el riesgo de incompatibilidad de esquema es bajo; el trabajo pendiente es de integración y conectividad, no de diseño del parser.

Protocolo de activación documentado: `$env:TFG_CAPE_ENABLED=1`, URL del endpoint CAPE y ejecución de `scripts/preflight_cape_phase2.py --check-connectivity`.

8.5.2 Integración de un dataset de benignos en el corrector de FP

El módulo `fp_correction.py` expone la interfaz completa para registrar evidencia benigna (imphash, sha256, firma, patrón funcional). El dataset de Ilić et al. [21], con 22.200 informes de Cuckoo de los cuales **11.735 son benignos**, es el candidato identificado como fuente de esa evidencia: incorporarlo permitiría evaluar el *recall* real del componente y ajustar empíricamente el umbral `benign_score ≥ 0,70`, hoy imposible sobre un dataset 100 % malware (§8.4.4).

La integración exige una comprobación previa que no se ha realizado: el dataset publica **informes JSON de Cuckoo, no binarios**, de modo que **no está verificado que exponga los campos `imphash` y `sha256`** sobre los que operan las listas actuales. Si no los expone, la vía sería registrar evidencia por nombre de firma o por patrón funcional (ganchos que la interfaz ya soporta), o normalizar los informes a `CapeFeatures` mediante un adaptador de esquema Cuckoo→CAPE. Determinar cuál de las dos vías aplica es el primer paso de esta línea.

8.5.3 Evaluación cualitativa de la síntesis con rúbrica MESSALA

La comparativa de §7.6 evalúa fidelidad automática (5 comprobaciones) sobre n = 2 y con sesgo estructural hacia la plantilla. Una evaluación de accionabilidad real (¿puede el analista actuar a partir de la ficha sin consultar el JSON de CAPE?) requiere la rúbrica MESSALA (completitud, precisión, accionabilidad, concisión, ausencia de alucinaciones) aplicada sobre 30-50 fichas de familias y buckets variados.

8.5.4 Validación con el dataset completo Avast-CTU: completada (2026-06-27)

La re-evaluación con el dataset completo Avast-CTU (n = 48.976, `full`, 13,97 GB) se realizó el 2026-06-27. Todos los hallazgos de §7.2–§7.5 se confirman sin cambio de signo. La evaluación del motor de anomalía sobre una partición train/test separada (es decir, ajustar el IsolationForest sobre un subconjunto y evaluar sobre otro) sigue siendo un trabajo futuro pendiente: en el corte actual el modelo se entrena y evalúa sobre las mismas 48.976 muestras (§8.4.3).

8.5.5 Robustez del campo `type` ante familias no etiquetadas

La dependencia de `type` como entrada (§8.4.2) puede mitigarse con: (a) evaluar si el campo `malfamily_tag` de los informes CAPE 2.5 proporciona suficiente información para inferir el tipo sin fuente externa, y (b) construir un clasificador de tipo ligero, entrenado sobre features de comportamiento/red de `CapeFeatures`, como fallback cuando `type` no esté disponible. Este clasificador sería un componente separado y previo al motor de severidad, sin circularidad con el `malscore`.

8.5.6 Consolidación pre-entrega del repositorio

Antes de la entrega final, revisar el repositorio para eliminar material auxiliar de trabajo, rutas no reproducibles fuera del entorno del autor y documentos de coordinación de proceso. Esta tarea no depende técnicamente de ningún desarrollo pendiente.

8.5.7 Cableado de IOCs y alineación display-vs-motor en la síntesis

Dos limitaciones declaradas en esta memoria no son defectos de diseño ni de los motores, sino de **cableado** entre componentes, y se corrigen sin tocar la lógica de decisión:

(a) Recuperar los IOCs de red para la ficha. El parser extrae `unique_ips` y `unique_domains` anonimizados en `CapeFeatures`, pero la ingesta (`ingest_avast_ctu.py`) no los vuelca al dataset tabular (solo guarda los conteos `hosts_count`/`domains_count`), de modo que ni la cola ni la ficha llegan a verlos y la sección de IOCs aparece vacía (§7.6). No basta con añadir columnas al constructor de la cola: la fuente de la que lee no contiene las listas. Hay dos vías, ambas sin tocar los motores ni las cifras del capítulo 7: re-generar el dataset persistiendo esas dos listas, o, dado que el dataset conserva el `sha256` completo y el informe original está disponible, re-parsear el informe del caso bajo demanda al abrir la ficha, sin regenerar la cola.

(b) Alinear la etiqueta de familia mostrada con la del motor. La síntesis usa `family or detections` como etiqueta de display, mientras que el bonus de notoriedad del motor de severidad exige familia verificada de fuente externa; cuando ambas difieren (caso que no se da en Avast-CTU, pero sí en producción con muestras sin etiquetar), el texto de la ficha puede mencionar una notoriedad que el motor no aplicó (§5.7). La corrección consiste en que la ficha distinga explícitamente la etiqueta observada por el sandbox de la familia verificada, y en que la mención de notoriedad se derive solo de la segunda.

Ninguna de las dos altera los motores de severidad, anomalía, corrección de FP o fusión, ni el orden de la cola: **las cifras del capítulo 7 no se ven afectadas**. Ambas son requisito para que la ficha explicable exhiba los IOCs del caso en una demostración en vivo.

8.6 Impactos y responsabilidad

Un sistema que decide qué mira primero un analista de seguridad no es una herramienta neutra. Esta sección recoge la reflexión sobre sus impactos sociales y ambientales y sobre la responsabilidad ética y profesional asociada al trabajo.

8.6.1 Impacto social

El impacto directo del sistema es sobre el trabajo de una persona: el analista de SOC. La motivación del trabajo (§1.1) es precisamente un problema laboral además de técnico: una cola saturada convierte a un profesional cualificado en un descartador de ruido. El objetivo es devolverle criterio, no quitárselo.

Esa distinción es una decisión de diseño, no una declaración de intenciones. Atalaya **reordena y explica; no decide ni actúa**: no bloquea, no aísla equipos, no cierra alertas. Entrega una cola ordenada y, para cada caso, la descomposición completa de por qué ocupa esa posición, con el `malscore` original de CAPE visible al lado para que el analista pueda discrepar. El juicio final sigue siendo humano, y el sistema está construido para ser cuestionado: ese es el sentido de la trazabilidad componente a componente (§5.4, §6.5).

Aun así, hay dos riesgos que conviene nombrar:

- **Sesgo de automatización.** Un analista que confía en exceso en el orden propuesto puede dejar de mirar la parte baja de la cola. El sistema mitiga esto siendo explicable (una justificación visible invita a la discrepancia mucho más que un número opaco) y rebajando en lugar de descartar: ninguna muestra sale de la cola (§8.4.7). Pero la mitigación es parcial: si el orden es sistemáticamente erróneo en una zona, la explicación acompaña al error con la misma convicción. Por eso el sistema se presenta como asistente de triaje y no como filtro.
- **Deshabilitación del criterio experto.** Un sistema de apoyo mal entendido puede empobrecer la formación del analista junior, que aprende priorizando. Que la ficha exponga el razonamiento (tipo de amenaza, técnicas MITRE observadas, notoriedad, comportamiento atípico) la convierte en material formativo en lugar de en una caja negra que dicta.

Existe además un impacto positivo indirecto: un triaje más certero reduce el tiempo hasta la detección de incidentes reales, y ese tiempo tiene consecuencias sobre terceros, usuarios cuyos datos están en juego, que no participan en la decisión ni la conocen.

8.6.2 Impacto ambiental

No se ha medido el consumo energético del sistema, de modo que esta reflexión es cualitativa y se declara como tal; cualquier cifra concreta sería inventada.

El coste computacional del trabajo se reparte de forma muy desigual y merece separarse:

- **El sandbox es la parte cara, y es previa al sistema.** Ejecutar cada muestra en una máquina virtual instrumentada durante minutos es, con diferencia, lo más costoso de la cadena. Ese coste lo asume CAPE y existe con o sin este trabajo: Atalaya se sitúa **aguas abajo** y no añade ejecuciones de sandbox. De hecho, opera sobre informes ya generados (dataset Avast-CTU), por lo que toda la evaluación de esta memoria, sobre 48.976 muestras, no requirió detonar ni una sola muestra.
- **El re-valorador es barato.** Los motores de severidad y fusión son aritmética determinista, y el IsolationForest se entrena una vez sobre un conjunto tabular modesto. El pipeline completo se ejecuta en hardware de consumo, sin GPU ni infraestructura dedicada, y está diseñado para correr offline en un portátil (§4.5.2, §7.6).

- **La síntesis con LLM es la parte más costosa del sistema propio, y es opcional y está desactivada por defecto.** La decisión de usar la plantilla determinista como modo por defecto se tomó por robustez y fidelidad (§7.6), pero tiene una consecuencia ambiental favorable: el caso normal no invoca ningún modelo de lenguaje. Cuando se activa, el modelo es **local** (Ollama, 3-4 B de parámetros en CPU): no se envía ni una petición a una API en la nube, lo que evita el coste de inferencia remota en centros de datos y, de paso, la fuga de datos del incidente a un tercero.

Conviene un matiz: elegir modelos pequeños en CPU no fue una decisión ambiental sino una restricción de hardware. El beneficio es real, pero no es mérito de diseño.

8.6.3 Responsabilidad ética y profesional

Manipulación de malware. El trabajo se apoya en análisis de *malware* real, pero conviene precisar su naturaleza: el dataset Avast-CTU [19] son **informes JSON de comportamiento, no binarios**. En el desarrollo y la evaluación de esta memoria no se ha ejecutado, descargado ni redistribuido ninguna muestra ejecutable. El único punto en que el proyecto manipula binarios es el entorno de sandbox propio, aislado en una máquina virtual y sin salida a producción.

Protección de datos. Aunque los informes proceden de un dataset público, contienen artefactos de red (direcciones IP, dominios, nombres de máquina) que se someten a anonimización (SHA-256 con sal) **antes de su persistencia**: el parser la aplica por defecto (`anonymize=True`) y la única forma de desactivarla es una opción explícita reservada a depuración (§4.3.3, §5.3). La política del proyecto prohíbe almacenar información de identificación personal. Esta decisión no viene exigida por el dataset: se toma porque el sistema está pensado para ejecutarse sobre alertas reales de una organización, donde esos campos sí identifican a personas y equipos concretos.

Uso de las fuentes. El dataset se emplea conforme a su publicación oficial y se cita en su referencia primaria; las listas de notoriedad derivan de un *advisory* público de CISA [N2] y la taxonomía de técnicas, de MITRE ATT&CK [N1]. Ninguna fuente se reproduce más allá de lo necesario para el trabajo.

Honestidad metodológica como responsabilidad profesional. En seguridad, un sistema que se vende por encima de lo que puede sostener es un riesgo operativo: alguien confiará en él. Este trabajo declara explícitamente lo que no puede demostrar, aunque debilite el relato:

- la señal conductual del motor **no supera** al `malscore` como discriminador inter-familia (η^2 0,477 frente a 0,510, §7.3), y se dice en lugar de reportar solo el η^2 agregado favorable;
- las constantes del motor son **política razonada, no calibración empírica** (§5.4), con dos defectos de calibración conocidos y declarados (§7.7.4);
- **no existe *ground truth* de urgencia verdadera**, de modo que el sistema defiende un orden accionable, explicable y coherente, no demostrablemente óptimo; y la validación cualitativa que lo respalda la realizó el propio autor, juez y parte (§7.5);
- el corrector de falsos positivos no tiene *recall* medido (§8.4.4), y la evaluación de la síntesis se hizo sobre n = 2 con un sesgo estructural que se documenta contra el propio resultado favorable (§7.6).

Declarar estos límites es parte del trabajo de ingeniería, no un apéndice de él.

8.6.4 Relación con los Objetivos de Desarrollo Sostenible

La contribución de un TFG a los ODS es necesariamente indirecta y modesta, y conviene no sobreactuarla. Dos son razonablemente defendibles:

- **ODS 9 (industria, innovación e infraestructura)**, en su meta de infraestructuras fiables y resilientes: el trabajo contribuye a la capacidad de detectar y atender antes los incidentes que afectan

a infraestructura digital, y lo hace con herramientas abiertas y reproducibles, sin dependencia de plataformas propietarias.
- **ODS 16 (paz, justicia e instituciones sólidas)**, en la medida en que la ciberdelincuencia (fraude, extorsión mediante *ransomware*, robo de credenciales) afecta a la confianza en las instituciones y a sus servicios; mejorar la respuesta de un SOC actúa sobre esa cadena.

No se reclama contribución a ningún otro objetivo: hacerlo sería retórica.

8.7 Cierre

El sistema entregado para la defensa de este TFG es un asistente de triaje post-sandbox **funcionalmente completo, reproducible y verificado**: los seis componentes del pipeline v2 están implementados, testados (241 tests automatizados a 18 de julio de 2026: 228 pasan y 13 se omiten por depender de fixtures no versionadas; `demo.ps1` 21/21 PASS) y coexisten con el prototipo v1 sin modificarlo.

La contribución central (la garantía anti-leakage por contrato y la trazabilidad dato → urgencia → ficha) está verificada a nivel de diseño (ningún componente acepta `malscore` como argumento) y a nivel de tests (13 `TestNoLeakage` en 6 módulos). El reordenamiento del bloque saturado de CAPE (56,28 % de 48.976 muestras con `malscore` = 10,0) en una cola priorizada con distribución no plana (0,7 % crítico, 46,1 % alto, 45,3 % medio, 7,9 % bajo) y con cada posición justificable componente a componente es la propuesta de valor operativa del sistema.

Las limitaciones están documentadas: señal conductual por debajo del `malscore` (η^2 conductual 0,477 vs 0,510 en la descomposición de §7.3, reproducible con `scripts/eta_decomposition.py`), dependencia del campo `type` como entrada, motor de anomalía sin partición train/test separada, componente de FP sin recall evaluado, evaluación de síntesis sobre n = 2 con sesgo estructural, y validación E2E con CAPE 2.5 en vivo no realizada. Estas limitaciones no invalidan el trabajo; definen su alcance real y la agenda de trabajo futuro de §8.5.

El núcleo del sistema, un motor de decisión determinista que codifica política SOC explícita con trazabilidad auditable, es precisamente lo que hace al prototipo **defendible** en un contexto académico: cada posición en la cola tiene una justificación componente a componente, el `malscore` original de CAPE queda visible al lado para comparación, y el analista puede auditar por qué el sistema diverge de CAPE en los casos de interés (njRAT promovido, Swisyn rebajado) sin acceder al código ni al JSON técnico.

Bibliografía

Las referencias siguen el formato APA 7.ª edición.

Referencias principales (ordenadas por categoría)

Análisis dinámico de malware / Sandbox

[08] Gazdík, J. (2023). *Implementace řešení pro bezpečnostní analýzu cracků počítačových her* [Implementation of a Solution for Security Analysis of Computer Game Cracks] [Bakalářská práce]. VŠB – Technická univerzita Ostrava, Fakultad de Ingeniería Eléctrica e Informática. Supervisor: Ing. Jan Plucar, Ph.D.

[09] Bouhnine, A. (2024). *Malware Obfuscation and Evasion: Developing Custom Malware and Extending AVET Framework for CAPEv2 Sandbox Evasion* [Master thesis]. Cyber Defence Lab / Cylab. Supervisores: Prof. Wim Mees y Prof. Georgi Nikolov. https://cylab.be/publications/67/download/2024-malware-obfuscation-and-evasion.pdf

[19] Bosanský, B., Kouba, D., Manhal, O., Sick, T., Lisý, V., Kroustek, J., & Somol, P. (2022). *Avast-CTU Public CAPE Dataset* (arXiv:2209.03188). arXiv. https://arxiv.org/abs/2209.03188 · https://github.com/avast/avast-ctu-cape-dataset

Fatiga de alertas y operación de SOC

[20] Alahmadi, B. A., Axon, L., & Martinovic, I. (2022). 99 % false positives: A qualitative study of SOC analysts' perspectives on security alarms. En *31st USENIX Security Symposium (USENIX Security 22)* (pp. 2783–2800). USENIX Association. ISBN 978-1-939133-31-1. https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi *(Estudio cualitativo por entrevistas. El «99 %» del título es una cita textual de un participante (§6, p. 2792), no una tasa medida; los propios autores precisan que responde mayoritariamente a disparadores benignos y no mide el rendimiento de la tecnología de detección. Fuente además de las cinco propiedades de una alarma útil (fiable, explicable, analítica, contextual y transferible) empleadas en §3.3 y §5.1.)*

Detección de anomalías en logs

[07] Landauer, M., Onder, S., Skopik, F., & Wurzenberger, M. (2023). Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications*, *12*, 100470. https://doi.org/10.1016/j.mlwa.2023.100470

[10] Guo, H., Yuan, S., & Wu, X. (2021). LogBERT: Log anomaly detection via BERT. *arXiv preprint arXiv:2103.04475*. https://arxiv.org/abs/2103.04475

[11] Guan, W., Cao, J., Qian, S., Gao, J., & Ouyang, C. (2025). LogLLM: Log-based anomaly detection using large language models. *arXiv preprint arXiv:2411.08561*. https://arxiv.org/abs/2411.08561

[12] Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. En *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)* (pp. 1285–1298). ACM. https://doi.org/10.1145/3133956.3134015

Datasets de análisis dinámico

[21] Ilić, S., Gnjatović, M., Tot, I., Jovanović, B., Maček, N., & Gavrilović Božović, M. (2024). Going beyond API calls in dynamic malware analysis: A novel dataset. *Electronics*, *13*(17), 3553. https://doi.org/10.3390/electronics13173553 *(22.200 informes de Cuckoo: 11.735 benignos y 10.465 de malware. La licencia CC BY 4.0 está confirmada para el artículo; la del dataset no se ha verificado. Los informes son JSON de Cuckoo, no binarios.)*

Priorización de amenazas y síntesis SOC

[14] Rashid, S. M. Z. U., Alam, M. M., Montasir, I., & Haq, A. (2025). Risk-based MITRE TTP scoring for proactive cyber threat prioritization and response. En *Proceedings of the 2025 International Conference on Software, Cybersecurity and Automation (ICSCA '25)*. ACM. https://doi.org/10.1145/3731806.3731846 *(Texto completo tras paywall; citado por el planteamiento del abstract. Propone AIM ante la ausencia de un marco base para ponderar TTPs ATT&CK ante el analista SOC.)*

[15] Hassan, W. U., Bates, A., & Marino, D. (2020). Tactical provenance analysis for endpoint detection and response systems. En *Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P 2020)*. IEEE. https://ieeexplore.ieee.org/document/9152771 *(IEEE Xplore: 9152771. Sistema RapSheet, precedente seminal de valoración del riesgo por secuencia y táctica de alertas en EDR, frente a veredicto de score aislado.)*

[16] Okada, H., Oba, T., & Yanai, N. (2026). LLMs, you can evaluate it! Design of multi-perspective report evaluation for security operation centers. *arXiv preprint arXiv:2601.03013*. https://arxiv.org/abs/2601.03013 *(Preprint arXiv; sin peer-review confirmado a la fecha de redacción de este TFG. Fuente primaria de la rúbrica MESSALA, evaluación multi-perspectiva de resúmenes SOC.)*

TFGs nacionales relacionados

[01] Entrena Rodríguez, J. (2023). *Evaluación de técnicas de Machine Learning y métodos de selección de atributos para detección de intrusos en el ámbito de IoT* [Trabajo Fin de Grado]. Universidad de Zaragoza, Escuela de Ingeniería y Arquitectura. Director: José García Moros. https://zaguan.unizar.es/record/146776/files/TAZ-TFG-2023-1930.pdf

[02] Huarte Albás, Á. (2019). *Análisis y detección de anomalías en tráfico de red con Machine Learning* [Trabajo Fin de Grado]. Universidad de Zaragoza, Escuela Universitaria Politécnica de Teruel. Directores: Raquel Lacuesta Gilaberte, Santiago Hernández Ramos. https://zaguan.unizar.es/record/156416/files/TAZ-TFG-2019-2948.pdf

[03] Varea Palacios, J. (2024). *Detección de malware utilizando técnicas de machine learning* [Trabajo Fin de Grado]. Universidad de Zaragoza, Escuela de Ingeniería y Arquitectura. Director: Pedro Javier Álvarez Pérez-Aradrós; Codirector: Razvan Radacu. https://zaguan.unizar.es/record/134118/files/TAZ-TFG-2024-079.pdf

[04] Calvera Tonin, A. (2022). *Minería de datos para detección de intrusiones en redes de comunicaciones* [Trabajo Fin de Grado]. Universidad de Zaragoza, EINA. Directores: Sergio Ilarri Artigas, Fernando Tricas García. Proyecto NEAT-AMBIENCE (PID2020-113037RB-I00 / AEI). https://zaguan.unizar.es/record/125557/files/TAZ-TFG-2022-2390.pdf

[05] Rodríguez García, M. (2022). *Evaluación de técnicas de machine learning sobre diferentes bases de datos para detección de intrusos* [Trabajo Fin de Grado]. Universidad de Zaragoza, Escuela de Ingeniería y Arquitectura. Director: José García Moros. https://zaguan.unizar.es/record/126182/files/TAZ-TFG-2022-3261.pdf

[06] Mateos López, R. (2018). *Análisis, diseño e implementación de una solución de monitorización de Aula Informática basada en ELK* [Trabajo Fin de Grado]. Universidad Carlos III de Madrid, Grado en Ingeniería Informática. Tutor: Alejandro Calderón Mateos. Licencia CC BY-NC-ND. https://e-archivo.uc3m.es/bitstream/handle/10016/29055/TFG_Rafael_Mateos_Lopez.pdf

Trabajos internacionales

[13] Alsudani, M. A. M. (2025). *A TOGAF-Aligned Architectural Framework for the Integration and Optimization of AI-Driven Intrusion Detection Systems in Enterprise Environments* [Diplomarbeit, Diplom-Ingenieur]. Technische Universität Wien, Fakultät für Informatik. Betreuer: Prof. Dr. Henderik A. Proper.

[17] Bortoli, T. (2017). *A Framework for Network Intrusion Detection on Open Platform Communications Unified Architecture* [Master-Thesis]. Technische Universität Darmstadt, Fachbereich Sicherheit Informatik / Fraunhofer SIT. URN: urn:nbn:de:tuda-tuprints-68029. http://tuprints.ulb.tu-darmstadt.de/68029 *Nota: el fichero de referencia lleva sufijo "2024" pero el documento es de 2017.*

Referencias técnicas y normativas

Frameworks y estándares

[N1] MITRE ATT&CK. (2023). *MITRE ATT&CK Enterprise Matrix* (v14). https://attack.mitre.org/

[N2] CISA. (2022, August 4). *Alert AA22-216A: 2021 Top Malware Strains*. Cybersecurity and Infrastructure Security Agency. https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-216a

Herramientas y software empleado

[T1] CAPEv2. (2024). *CAPEv2: Malware Configuration And Payload Extraction* [Software]. https://capev2.readthedocs.io/

[T2] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. https://scikit-learn.org/

[T3] FastAPI. (2024). *FastAPI framework* [Software]. https://fastapi.tiangolo.com/

[T4] MLflow. (2024). *MLflow: An open source platform for the machine learning lifecycle* [Software]. https://mlflow.org/

[T5] Ollama. (2024). *Ollama: Get up and running with large language models* [Software]. https://ollama.com/

[T6] Microsoft Research. (2025). *Phi-4-mini* [Modelo de lenguaje]. Microsoft. https://huggingface.co/microsoft/Phi-4-mini-instruct

[T7] Meta AI. (2024). *Llama 3.2: Open foundation and fine-tuned chat models* [Modelo de lenguaje]. Meta. https://www.llama.com/

Datasets del proyecto (D1 evaluado en cap. 7; D2-D3 en estado del arte cap. 2)

ID	Nombre	Referencia
D1	Avast-CTU Public CAPE Dataset	Bosanský et al. 2022 (arXiv:2209.03188) [19]
D2	HDFS	Xu et al., 2009 (via LogBERT [10])
D3	BGL	Oliner & Stearns, 2007 (via LogBERT [10])
D4	Sintético TFG v1	Generado con `seed_demo_data.py` (pre-pivote v2, histórico)
D5	CAPE real (pendiente)	Generado por CAPEv2 en VM de análisis propia
D6	Ilić et al. (2024), *Going beyond API calls in dynamic malware analysis*, 22.200 informes de Cuckoo (11.735 benignos + 10.465 de malware)	Ilić et al. 2024 [21] · dataset de benignos para trabajo futuro (§8.5.2)

Nota: la numeración [01]-[19] refleja el orden de incorporación de las fuentes y no es estrictamente consecutiva. Las referencias técnicas y normativas usan prefijos [N1]-[N2], las herramientas [T1]-[T7] y los datasets [D1]-[D6].

Anexos

Material complementario de referencia. Ninguno de estos anexos introduce resultados nuevos: son instrucciones de reproducción, la estructura del código y el catálogo de interfaces descritos en los capítulos 5 y 6, consolidados aquí para consulta directa.

Anexo A. Manual de reproducción del pipeline (sin CAPE/VM)

Todo el pipeline v2 se ejecuta sin infraestructura de sandbox: opera sobre los informes reales ya generados del dataset Avast-CTU. Los comandos se lanzan desde `prototipo/`.

A.1 Entorno

```
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt
cp .env.example .env
```

A.2 Obtención del dataset (requisito previo)

La ingesta necesita **dos ficheros** del repositorio oficial de Avast-CTU [19], alojados en Google Drive (la descarga exige sesión de Google y está sujeta a cuotas del servicio). Ambos deben quedar en `prototipo/data/external/avast_ctu/`:

Fichero	Contenido	Tamaño
`public_full_reports.zip`	Los 48.976 informes JSON de CAPEv2 (versión *full*)	13,97 GB
`public_labels.csv`	Etiquetas de familia y tipo, **independientes del `malscore`**	—

`public_labels.csv` no es opcional: es la fuente externa de la etiqueta, el mecanismo que evita el *leakage* circular (§4.3.1), y sin él la ingesta aborta con `FileNotFoundError`. La versión *reduced* del zip no reproduce las cifras del capítulo 7, que corresponden a la *full*.

A.3 Pipeline de triaje v2 (fases A–F)

El orden reproduce las seis fases del pivote descritas en §6.1 y §6.2.

```
python scripts/ingest_avast_ctu.py --source full --limit 0   # Fase A: los 48.976 informes (0 = todos)
python scripts/compare_severity.py        # Fase B: severity_v2 frente a malscore (η², Spearman)
python scripts/train_anomaly_model.py     # Fase C: entrena data/models/anomaly_v2.joblib
python scripts/evaluate_anomaly_fp.py     # Fase C: evalúa anomalía + corrector de FP
python scripts/build_priority_queue.py    # Fase D: genera la cola en reports/priority_queue_*.csv|json
```

```
python scripts/run_secops_summary.py      # Fase E: síntesis plantilla vs LLM (requiere Ollama para el modo LLM)
python scripts/build_cap07_evaluation.py  # Fase F: consolida las 5 fases en un informe en reports/
```

> **Nota: `--limit 0` es obligatorio para reproducir el capítulo 7.** El valor por defecto de `--limit` es **200**: ejecutar `ingest_avast_ctu.py` sin ese argumento no falla ni avisa: ingesta 200 informes y produce un dataset pequeño cuyas cifras **no se corresponden** con las de esta memoria. `--source full` es ya el valor por defecto, pero se escribe explícito.

A.4 Servidor y UI

```
uvicorn app.main:app --host 127.0.0.1 --port 8090 --reload
# UI de triaje:  http://127.0.0.1:8090/ui/triage
# Ficha:         http://127.0.0.1:8090/ui/triage/{rank}
```

A.5 Pruebas

```
pytest -q                   # suite completa
```

A.6 Demo end-to-end del prototipo v1

`scripts/demo.ps1` ejercita el flujo del prototipo **v1** (`/api/v1/events/ingest` → `/api/v1/score` → `/api/v1/incidents` → `/api/v1/status/model`) con 21 comprobaciones, y **no cubre el pipeline v2** del capítulo 7 (§6.6.3). Depende de su propio pipeline (dataset sintético y modelos v1), que no generan los pasos A.3; si falta, el script aborta en sus *pre-checks*. Para ejecutarla:

```
python scripts/seed_demo_data.py --samples 120 --difficulty realistic --reset-storage
python scripts/build_dataset.py && python scripts/train_models.py && python scripts/evaluate.py
scripts/demo.ps1                 # 21 checks sobre el contrato v1
```

Se conserva como prueba de no regresión del contrato v1, que el pivote v2 no debía romper, no como vía de reproducción de los resultados v2, que es la de A.3.

> **Nota de reproducibilidad.** El pipeline es determinista (`random_state = 42`, división temporal sin barajado). Una versión distinta de scikit-learn puede desplazar ligeramente los *scores* del IsolationForest; las cifras del capítulo 7 corresponden al corte del dataset *full* (n = 48.976).

Anexo B. Estructura del repositorio del prototipo

Árbol relevante para el pivote v2 (detalle en §6.1). Los módulos v1 se conservan intactos y sirven los endpoints `/api/v1/*` como contexto histórico del proyecto.

```
prototipo/
├── app/
│   ├── main.py                # registra los routers v1 y v2
│   ├── config.py              # settings (rutas, anonymization_salt)
│   ├── services/
│   │   ├── cape_parser_v2.py    # [1] parser multi-schema CAPE 2.2/2.5
│   │   ├── severity_engine.py   # [2] severidad intrínseca determinista
│   │   ├── anomaly_engine.py    # [3] IsolationForest, 14 features
│   │   ├── fp_correction.py     # [3'] corrección de falsos positivos
│   │   ├── urgency_fusion.py    # [4] fusión multiplicativa -> urgencia
│   │   ├── ollama_client.py     # cliente Ollama (system prompt defensivo)
│   │   ├── secops_summary.py    # [5] síntesis dual template/LLM
│   │   ├── triage_service.py    # [6] orquestador A-E
```

```
|   |   └── anonymizer.py        # SHA-256 con sal, reutilizado por [1]
|   ├── api/
|   |   ├── routes_triage.py     # rutas /api/v2/triage*
|   |   └── routes_ui.py         # + /ui/triage, /ui/triage/{rank}
|   └── templates/               # triage_queue.html, triage_detail.html, base.html
├── scripts/                     # ingest_avast_ctu, compare_severity, train_anomaly_model,
|                                # evaluate_anomaly_fp, build_priority_queue, run_secops_summary,
|                                # build_cap07_evaluation, demo.ps1
└── tests/                       # suite del pivote v2 + 53 tests v1/transversales preexistentes
```

Anexo C. Catálogo de endpoints de la API v2

Rutas v2 (API y UI) (§6.4-§6.5). Las rutas v1 permanecen intactas y una clase de test dedicada (`TestV1ContractIntact`) verifica que no hay regresiones.

Ruta	Método	Descripción
`/api/v2/triage`	POST	Triaje sobre un payload ya parseado (`triage_from_payload`). `ValueError` → 422.
`/api/v2/triage/cape_report`	POST	Triaje sobre un `report.json` crudo de CAPE (2.2 o 2.5); anonimización por defecto.
`/api/v2/triage/queue`	GET (`?top=20`)	Lee el `priority_queue_*.csv` más reciente de `reports/`. 404 con instrucción si no existe.
`/api/v2/triage/queue/{rank}`	GET	Fila concreta de la cola por `rank`. 404 si no hay CSV o el `rank` no existe.
`/ui/triage`	GET	Interfaz HTML de la cola priorizada.
`/ui/triage/{rank}`	GET	Ficha explicable de un caso concreto.

Anexo D. Capturas de la interfaz Atalaya

Las capturas siguientes ilustran el flujo de triaje end-to-end sobre la cola real de 48.976 casos. El procedimiento para regenerarlas figura en el Anexo A.

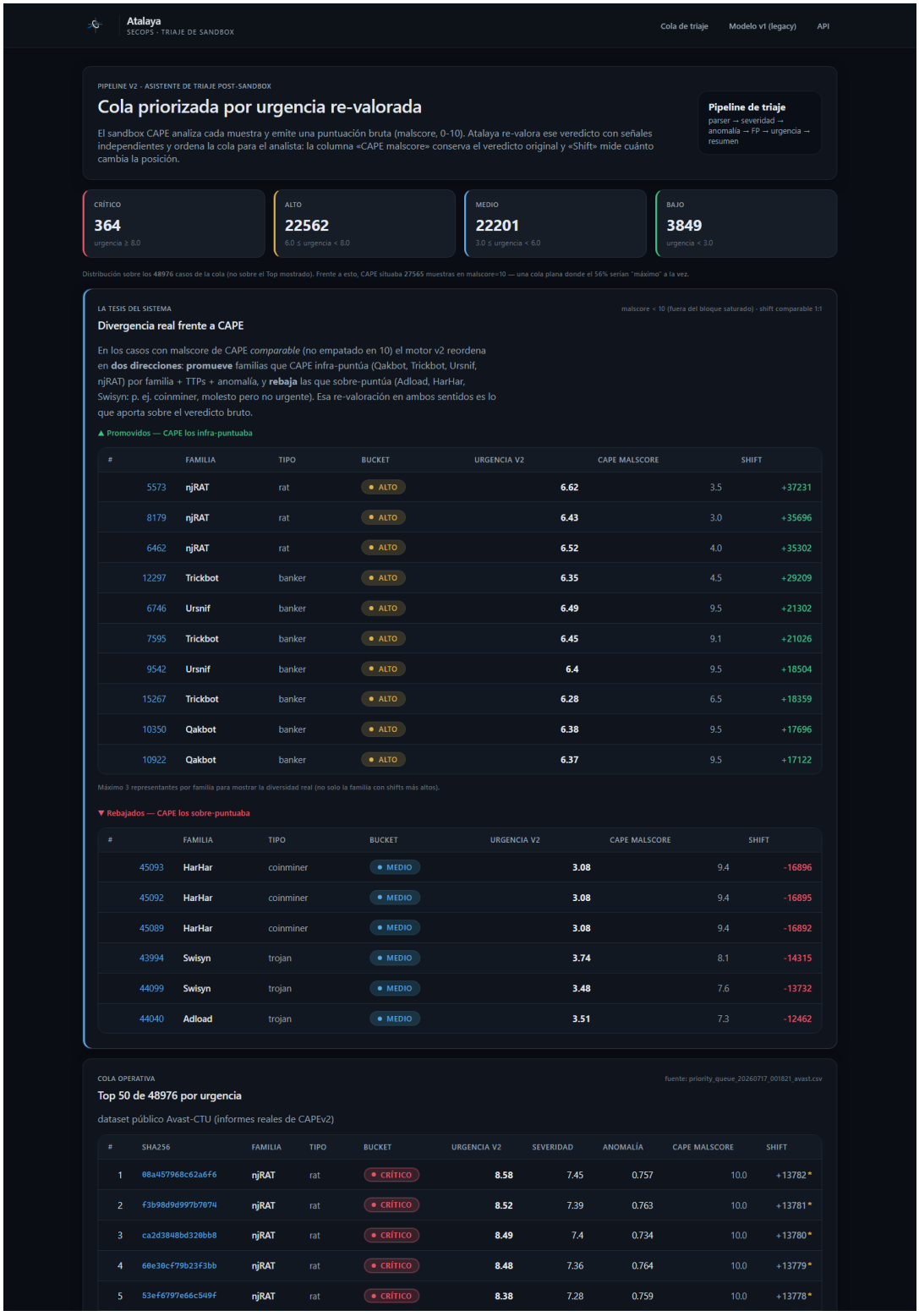


Figura D.1: Cola priorizada (`/ui/triage`). Arriba, la distribución por bucket sobre los 48.976 casos (364 críticos, 22.562 altos, 22.201 medios y 3.849 bajos) frente a las 27.565 muestras que CAPE situaba empatadas en `malscore` = 10. Debajo, el panel «Divergencia real frente a CAPE», restringido a `malscore` < 10 para que la comparación de posiciones sea 1:1: promueve familias que el sandbox infra-puntúa (njRAT, +37.231 posiciones) y rebaja las que sobre-puntúa (HarHar *coinminer*, −16.896).

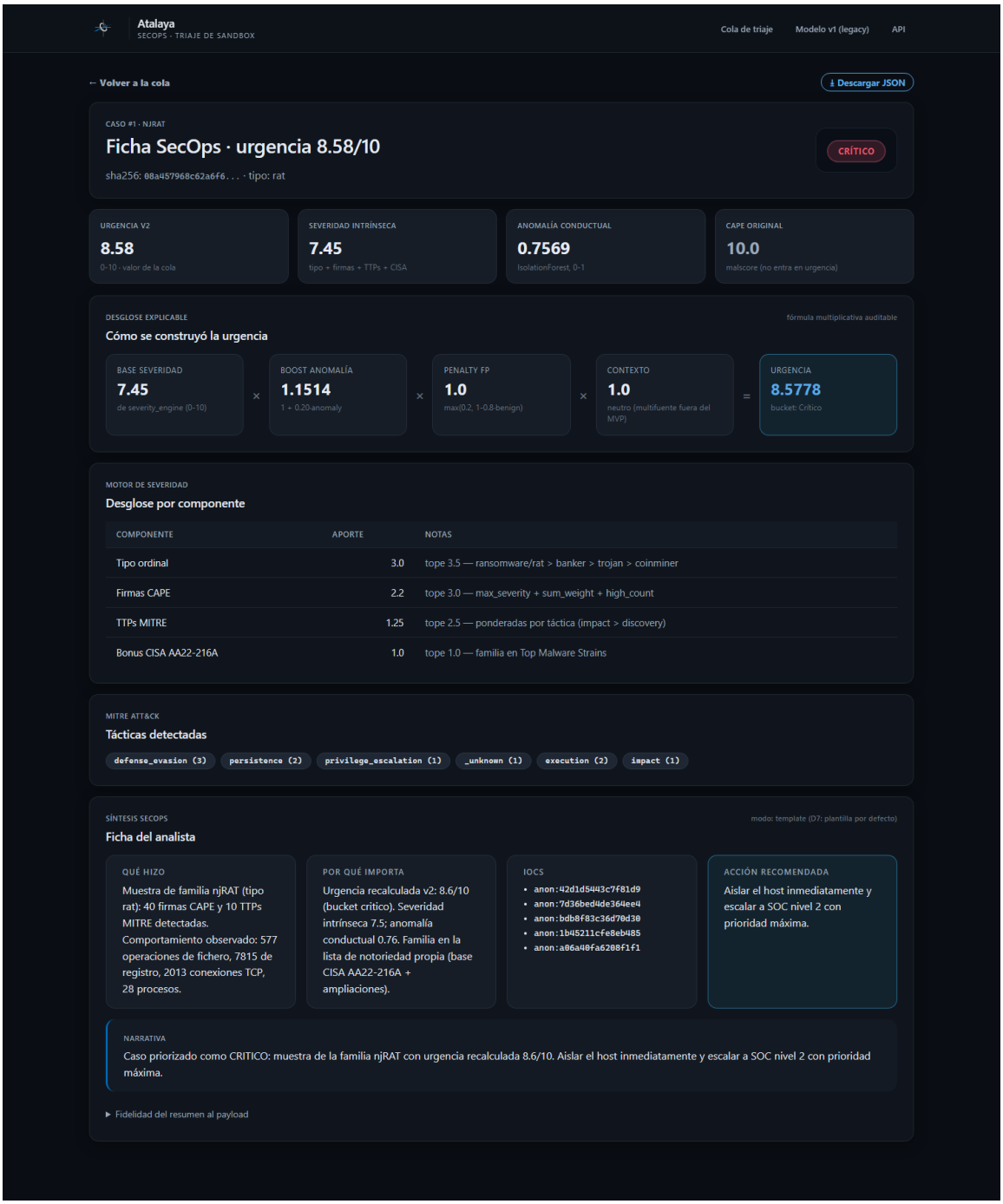


Figura D.2: Ficha explicable de un caso (`/ui/triage/1`, njRAT). La urgencia 8,58 se descompone en la fórmula multiplicativa auditable (severidad 7,45 × *boost* de anomalía 1,1514 × penalización de FP 1,0 × contexto 1,0) y en el aporte de cada componente del motor de severidad (tipo 3,0; firmas CAPE 2,2; TTPs MITRE 1,25; notoriedad 1,0). Abajo, la ficha SecOps en modo plantilla con los IOCs de red anonimizados y la acción recomendada.

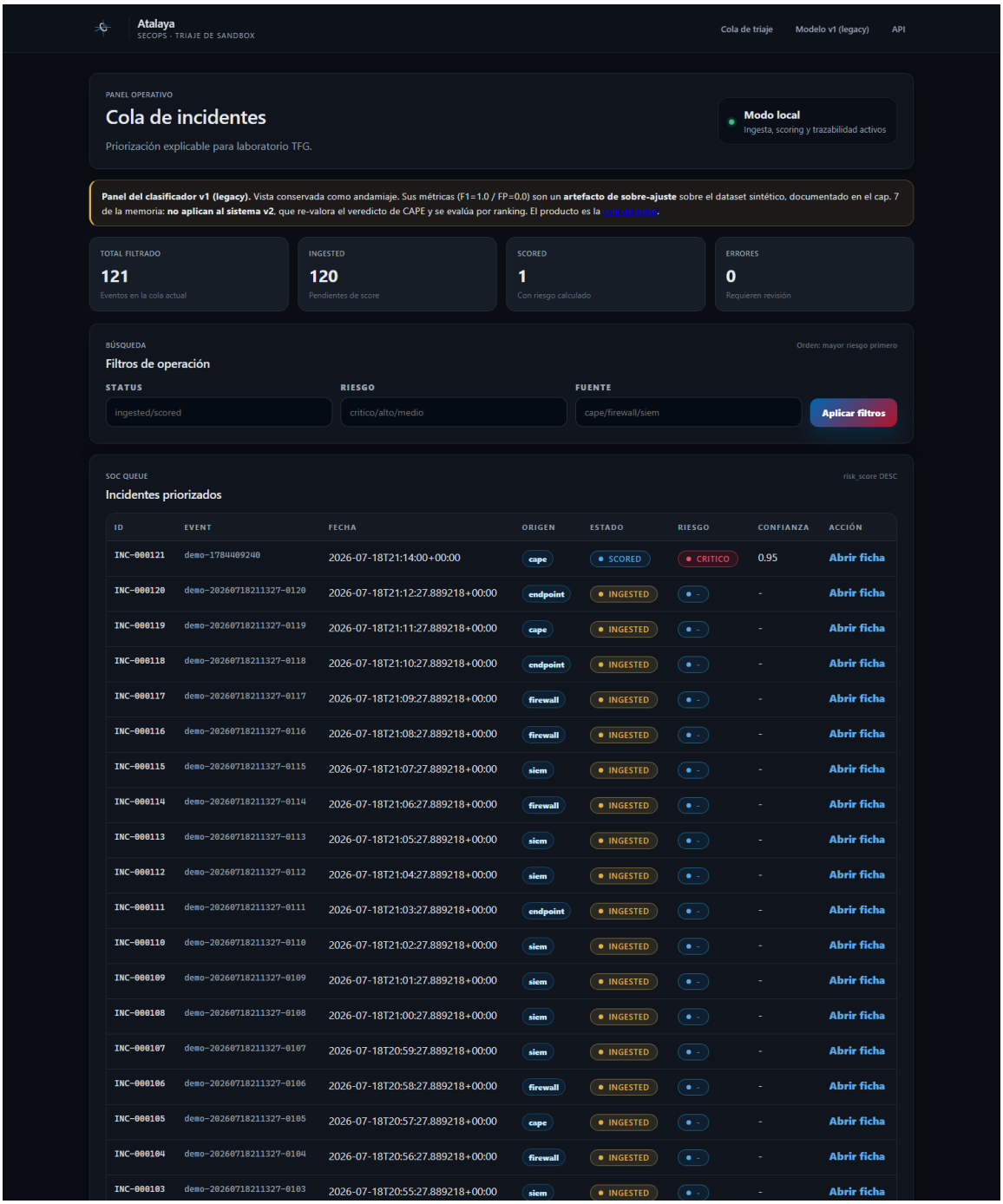


Figura D.3: Dashboard del prototipo v1, accesible en `/ui/legacy` fuera de la navegación principal y con su aviso de *legacy*. Se conserva sin modificar como evidencia de que el pivote a v2 no rompió el contrato anterior.